

中图分类号：TP316

论文编号：10006SY2106319

北京航空航天大学
硕士学位论文

面向广域虚拟数据空间的跨域数
据访问优化技术研究实现

作者姓名 杨锦乾

学科专业 计算机科学与技术

指导老师 肖利民 教授

培养院系 计算机学院

Research and Implementation of Cross-domain Data Access Optimization Technology for GVDS

A Dissertation Submitted for the Degree of Master

Candidate: Yang Jinqian

Supervisor: Prof. Xiao Limin

School of Computer Science and Engineering

Beihang University, Beijing, China

中图分类号: TP316

论文编号: 10006SY2106319

硕 士 学 位 论 文

面向广域虚拟数据空间的跨域数据访问优化技术研究与实现

作者姓名	杨锦乾	申请学位级别	工学硕士
指导教师姓名	肖利民	职 称	教授
学科专业	计算机科学与技术	研究方向	计算机系统结构
学习时间自	2021 年 09 月 01 日	起至	2024 年 06 月 05 日止
论文提交日期	2024 年 05 月 23 日	论文答辩日期	2024 年 06 月 05 日
学位授予单位	北京航空航天大学	学位授予日期	年 月 日

关于学位论文的独创性声明

本人郑重声明：所呈交的论文是本人在指导教师指导下独立进行研究工作所取得的成果，论文中有关资料和数据是实事求是的。尽我所知，除文中已经加以标注和致谢外，本论文不包含其他人已经发表或撰写过的研究成果，也不包含本人或他人为获得北京航空航天大学或其它教育机构的学位或学历证书而使用过的材料。与我一同工作的同志对研究所做的任何贡献均已在论文中作出了明确的说明。

若有不实之处，本人愿意承担相关法律责任。

学位论文作者签名：杨锦乾 日期：2024年06月07日

学位论文使用授权书

本人完全同意北京航空航天大学有权使用本学位论文（包括但不限于其印刷版和电子版），使用方式包括但不限于：保留学位论文，按规定向国家有关部门（机构）送交学位论文，以学术交流为目的赠送和交换学位论文，允许学位论文被查阅、借阅和复印，将学位论文的全部或部分内容编入有关数据库进行检索，采用影印、缩印或其他复制手段保存学位论文。

保密学位论文在解密后的使用授权同上。

学位论文作者签名：杨锦乾 日期：2024年06月07日

指导教师签名： 日期：2024年06月07日

摘 要

随着大数据、人工智能等领域应用蓬勃发展，单一数据中心逐渐不能满足领域应用的需求，多数据中心通过跨域数据访问实现数据资源共享，成为业界研究趋势之一。目前，针对各数据中心广域分散、存储资源隔离自治难以实现跨域数据访问的问题，课题组研发了广域虚拟数据空间系统，构建了基本的跨域数据访问通路。该通路包含调度、执行和缓存三个关键路径，提供了基础的跨域数据访问能力。

然而，在广域分布式环境中，网络带宽波动大、网络连接不稳定，广域虚拟数据空间系统中的数据访问面临着可靠性、公平性、性能等方面的挑战，直接影响了上层应用跨域数据访问的效率。具体来说，上述挑战包含广域网络波动，跨域数据访问请求难以完整执行导致可靠性差；广域带宽受限，大块跨域数据访问请求独占带宽资源导致公平性低；广域应用复杂，单一跨域数据预取策略时常失效导致性能不足三大问题。

因此，本文面向广域虚拟数据空间，研究跨域数据访问优化技术，针对跨域数据访问通路中存在的可靠性差、公平性低、性能不足等问题展开研究，优化跨域数据访问通路，提升跨域应用效率。本文的具体研究工作和主要成果如下：

(1) 针对跨域数据访问通路中面临的可靠性问题，设计并实现基于网络状况感知与时延预测的可靠跨域重传机制。首先，构建基于网络感知和时序分析的时延预测机制，通过长短期记忆网络与滑动窗口重传时机预测算法，计算目标节点往返时延预测值；其次，基于往返时延预测值，设计快速重定向机制，在网络链路出现异常时，依据预测值做出重定向决策，打断当前数据访问请求，并切换目标节点以完成后续请求，提升跨域数据访问请求可靠性和数据访问性能。实验结果表明，在异常的网络环境中，本方法相较于其他方法，平均带来了 48% 的读吞吐量提升和 175% 的写吞吐量提升。

(2) 针对跨域数据访问通路中面临的公平性问题，设计并实现基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制。首先，构建多应用 IO 队列，通过 IO 任务拆分，均衡跨域数据访问请求大小，并依据应用标识符对 IO 任务进行分组，赋予动态均衡权重；其次，引入虚拟运行 IO 作为带宽分配的决策依据，通过计算实际运行 IO 与应用权重的比值，更新虚拟运行 IO；最后，基于虚拟运行 IO 进行非抢占式与抢占式的跨域数

据访问请求调度，实现跨域数据访问带宽公平分配。实验结果表明，本方法相较于传统调度算法显著地提升了多应用带宽分配的公平性，同时相较于其它公平带宽分配方法带来了 8%左右的总吞吐量提升。

(3) 针对跨域数据访问通路中面临的性能问题，设计并实现基于跨域应用特征感知的自适应数据预取方法。首先，基于广域虚拟数据空间中的边缘缓存系统，构建了客户端侧的多级缓存机制，加速后续跨域访问请求；其次，面向顺序读和关联读场景，分别构建快速增量预取方法和关联数据预取方法，为特定跨域应用负载提供访问加速；最后，通过虚拟预取队列和滑动窗口机制，实现基于跨域应用负载感知的自适应数据预取方法，综合上述两种数据预取策略的运行效果，动态地为应用选取最佳的预取策略，提升缓存命中率。实验结果表明，本方法可以准确地识别出跨域应用特征，在顺序读和关联读混合的复杂负载场景下，相较于其它方法带来了至少 59%的缓存命中率提升。

综上，本文面向广域虚拟数据空间的跨域数据访问通路中面临的可靠性、公平性、性能问题，设计并实现 IO 任务可靠执行机制、多应用带宽公平分配机制、跨域数据预取策略，显著优化跨域数据访问通路，提升跨域数据访问效率。

关键词：高性能计算，虚拟数据空间，分布式存储系统，跨域数据访问，广域网络

Abstract

With the vigorous development of applications in fields such as big data and artificial intelligence, a single data center is gradually unable to meet the needs of field applications. Multiple data centers achieve data resource sharing through cross-domain data access, which has become one of the research trends in the industry. At present, in order to solve the problem of wide-area decentralization, isolation and autonomy of storage resources in various data centers, which make it difficult to achieve cross-domain data access, the research team has developed Global Virtual Data Space and established a basic cross-domain data access path. This path includes three key paths: scheduling, execution and caching, providing basic cross-domain data access capabilities.

However, in the wide-area distributed environment, network bandwidth fluctuates greatly and network connections are unstable. Data access in the Global Virtual Data Space system faces challenges in reliability, fairness, performance, etc., which directly affects the cross-domain data access of upper-layer applications. Specifically, the above challenges include three major problems: wide-area network fluctuations, and cross-domain data access requests are difficult to complete, resulting in poor reliability; wide-area bandwidth is limited, and large cross-domain data access requests monopolize bandwidth resources, resulting in low fairness; wide-area applications are complex, and single cross-domain data prefetching strategy often fail, leading to insufficient performance.

Therefore, this thesis focuses on the Global Virtual Data Space, studies the optimization technology of cross-domain data access, and conducts research on problems such as poor reliability, low fairness, and insufficient performance in cross-domain data access paths, optimizes cross-domain data access paths, and improves the efficiency of cross-domain applications. The specific work and main results of this thesis are as follows:

(1) To address the reliability problem faced in cross-domain data access paths, a reliable cross-domain retransmission mechanism based on network awareness and delay prediction is designed and implemented. First, a delay prediction mechanism based on network awareness

and time series analysis is constructed, and the target node round-trip delay prediction value is calculated through the LSTM and sliding window retransmission timing prediction algorithm. Secondly, based on the round-trip delay prediction value, a fast retransmission mechanism is designed. The mechanism makes redirection decisions based on predicted values when the network connection is abnormal, interrupts current data access request, and changes the target node to complete subsequent request, improving the reliability and performance of cross-domain data access. Experiments show that in abnormal network environments, compared with other methods, this method brings an average of 48% and 175% improvement in read and write throughput, respectively.

(2)To address the fairness problem faced in cross-domain data access paths, a multi-application cross-domain data access bandwidth fair allocation mechanism based on virtual running IO is designed and implemented. First, a multi-application IO queue is built, which balances the size of cross-domain data access requests by splitting IO tasks, groups IO tasks according to application identifiers, and assigns dynamic balancing weights. Second, virtual running IO is introduced as the basis for decision-making in bandwidth allocation, which is updated by calculating the ratio of actual running IO and application weight. Finally, non-preemptive and preemptive cross-domain data access request scheduling is performed based on virtual running IO to achieve fair allocation of cross-domain data access bandwidth. Experiments show that this method significantly improves the fairness of multi-application bandwidth allocation compared to traditional scheduling algorithms, and at the same time brings about an 8% increase in total throughput compared to other fair bandwidth allocation methods.

(3)To address the performance problem faced in cross-domain data access paths, an adaptive data prefetching method based on cross-domain application feature awareness is designed and implemented. First, based on the edge caching system in the Global Virtual Data Space, a multi-level caching mechanism on the client side is built to accelerate subsequent cross-domain data access requests. Secondly, the fast incremental prefetching method and related data prefetching method are built for consecutive read and associative read scenarios

respectively, which provide access acceleration for specific cross-domain application loads. Finally, through a virtual prefetch queue and sliding window mechanism, the adaptive data prefetching method based on cross-domain application load awareness is implemented. By combining the operating effects of the above two data prefetching strategies, the best prefetching strategy is dynamically selected for the application, improving the cache hit ratio. Experiments show that this method can accurately identify cross-domain application characteristics, and in complex load scenarios where consecutive reads and associative reads are mixed, it can increase the cache hit ratio by at least 59% compared to other methods.

In summary, this thesis focuses on the reliability, fairness, and performance issues faced in cross-domain data access paths for Global Virtual Data Space. Reliable IO task execution mechanisms, fair bandwidth allocation mechanisms for multiple applications, and cross-domain data prefetching strategies are designed and implemented, significantly optimizing cross-domain data access paths and improving cross-domain data access efficiency.

Key words: High Performance Computing, Virtual Data Space, Distributed Storage System, Cross-domain Data Access, Wide Area Network

目 录

第一章	绪论	1
1.1	课题来源	1
1.2	研究背景与意义	1
1.3	研究目标与内容	4
1.3.1	研究目标	4
1.3.2	研究内容	4
1.4	主要贡献与创新之处	6
1.5	论文组织结构	7
第二章	国内外相关研究	9
2.1	IO 任务可靠执行研究现状	9
2.1.1	面向数据持久化的可靠性机制	9
2.1.2	面向数据传输的可靠性机制	10
2.2	带宽公平分配研究现状	12
2.2.1	应用隔离公平分配方法	12
2.2.2	任务隔离公平分配方法	13
2.3	数据预取策略研究现状	14
2.3.1	顺序读负载预取策略	15
2.3.2	关联读负载预取策略	16
2.4	本章小结	17
第三章	基于网络状况感知与时延预测的可靠跨域重传机制	18
3.1	引言	18
3.2	问题与研究思路	18
3.3	详细方案设计与实现	21
3.3.1	符号描述	21
3.3.2	整体框架	22
3.3.3	网络感知模块	22

3.3.4	时序分析与预测模块	23
3.3.5	请求处理模块	24
3.4	实验与验证	25
3.4.1	实验环境	25
3.4.2	实验数据描述	26
3.4.3	高可靠跨域重传机制有效性对比测试	27
3.4.4	高可靠跨域重传机制时间复杂度测试	32
3.5	本章小结	32
第四章	基于虚拟运行 IO 的多应用跨域数据访问带宽分配机制	34
4.1	引言	34
4.2	问题分析与研究思路	34
4.3	详细方案设计与实现	37
4.3.1	符号描述	37
4.3.2	整体架构	37
4.3.3	多应用 IO 队列	38
4.3.4	带宽公平分配模块	40
4.3.5	基于虚拟运行 IO 的跨域数据访问请求完全公平调度算法	42
4.4	实验与验证	49
4.4.1	实验环境	49
4.4.2	带宽公平分配机制有效性测试	49
4.4.3	带宽公平分配机制性能测试	51
4.5	本章小结	54
第五章	基于跨域应用特征感知的自适应数据预取方法	55
5.1	引言	55
5.2	问题分析与研究思路	55
5.3	详细方案设计与实现	57
5.3.1	符号描述	57
5.3.2	总体架构	58

5.3.3	客户端侧的多级缓存机制	58
5.3.4	面向顺序读的快速增量预读方法	59
5.3.5	面向关联读的关联数据预取方法	61
5.3.6	基于跨域应用负载感知的数据预取自调优方法	62
5.4	实验与验证	64
5.4.1	实验环境	64
5.4.2	实验场景描述	64
5.4.3	实验结果与分析	65
5.5	本章小结	68
总结与展望		70
参考文献		72
攻读学位期间取得的成果		78
致 谢		79

图清单

图 1	GVDS 软件体系结构 ^[1]	1
图 2	GVDS 跨域数据访问通路示意图	2
图 3	跨域与域内数据访问性能差异图	3
图 4	论文研究目标	4
图 5	基于网络状况感知与时延预测的可靠跨域重传机制总体架构	22
图 6	时序分析与预测模块技术方案	23
图 7	跨域数据访问实验拓扑图	26
图 8	时序数据集前 500 条数据分布情况	27
图 9	快速重定向成功生效	28
图 10	不同访问延时读吞吐量	29
图 11	不同访问延时写吞吐量	29
图 12	不同延时波动读吞吐量	30
图 13	不同延时波动写吞吐量	30
图 14	高延时场景下不同任务大小读吞吐量	31
图 15	高延时场景下不同任务大小写吞吐量	31
图 16	正常网络环境中不同任务大小吞吐量	32
图 17	多应用带宽争用	35
图 18	基于虚拟运行 IO 的多应用跨域数据访问带宽分配机制总体架构	38
图 19	多应用 IO 队列基本结构	39
图 20	带宽公平分配模块基本结构	41
图 21	IO 任务拆分过程	42
图 22	10 个应用均衡场景下吞吐量分布	50
图 23	10 个应用倾斜场景下吞吐量分布	51
图 24	不同应用同访问负载性能测试结果	52
图 25	不同应用单请求递增访问负载性能测试结果	52
图 26	不同应用同访问大小递增任务数量性能测试结果	53
图 27	不同应用数量调度延迟	53

图 28	典型高性能计算应用文件访问负载占比图	56
图 29	基于跨域应用特征感知的数据预取方法总体架构	58
图 30	多级数据缓存示意图	59
图 31	面向顺序读的快速增量预读方法	60
图 32	面向关联读的关联数据预取方法	62
图 33	数据预取自调优方法示意图	62
图 34	顺序型负载数据预取实验结果图	66
图 35	关联型负载数据预取实验结果图	66
图 36	先顺序后关联负载数据预取实验结果图	67
图 37	先关联后顺序负载数据预取实验结果图	67
图 38	随机负载数据预取实验结果图	68

表目录

表 1	可靠跨域重传机制相关符号描述表.....	21
表 2	可靠跨域重传机制实验平台软硬件环境配置.....	26
表 3	时序数据集基本描述信息.....	27
表 4	模型训练参数表.....	28
表 5	带宽公平分配机制数据特征及描述表.....	37
表 6	本方法非抢占式调度数据结构表.....	43
表 7	本方法抢占式调度数据结构表.....	47
表 8	带宽公平分配机制实验平台软硬件环境配置表.....	49
表 9	带宽公平分配机制实验参数表.....	50
表 10	数据预取策略数据特征及描述表.....	57
表 11	数据预取策略实验平台软硬件环境配置表.....	64
表 12	自适应数据预取方法实验参数表.....	65

第一章 绪论

1.1 课题来源

本论文的课题源自国家重点研发计划“高性能计算”专项“高性能计算虚拟数据空间”项目。该项目攻克了国家高性能计算环境中统一存储空间构建、广域数据共享、数据与计算任务协同、多源数据处理等关键技术问题，形成了广域虚拟数据空间系统（Global Virtual Data Space, GVDS）^[1]。

GVDS 系统突破了跨域虚拟数据空间构建方法^[2]、广域环境数据高效迁移方法、广域环境存算协同调度方法^[3]、跨域高并发数据聚合处理技术等多项关键技术，实现了广域环境中分散自治异构存储资源的统一聚合，形成了全局数据统一访问视图。本课题面向 GVDS 系统的跨域数据访问开展研究，全面提升 GVDS 系统跨域数据访问的可靠性、公平性和性能。

1.2 研究背景与意义

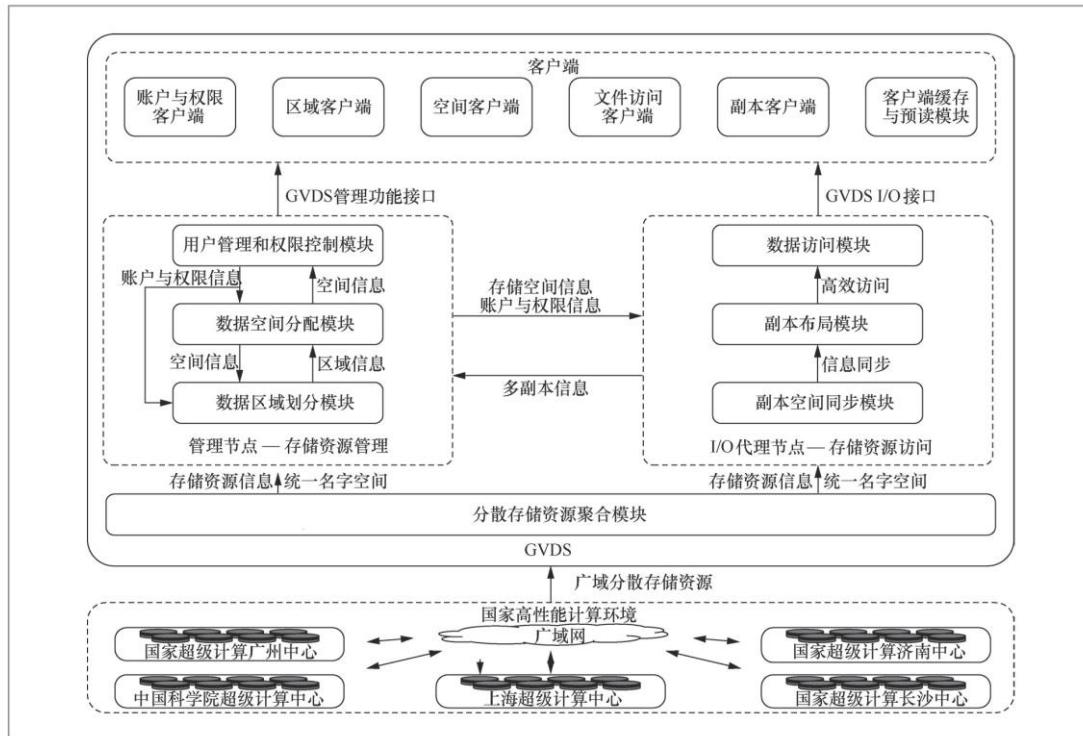


图 1 GVDS 软件体系结构^[1]

随着大数据、人工智能等领域应用蓬勃发展，单一数据中心逐渐不能满足领域应用的需求，多数据中心通过跨域数据访问实现数据资源共享，成为业界研究趋势之一。各大企业、研究机构纷纷在全国各地建设数据中心，其数据存储总规模达到了 PB、EB 甚至 ZB 的级别^[4]。

目前，跨域数据中心广域分散，存储资源隔离自治，跨中心协同计算应用在中心间大规模共享数据，迫切需要支持跨域统一访问的数据管理系统。如图 1 所示，GVDS 作为国内首个跨域分布式存储系统，通过多级异构资源聚合，隐藏了底层数据传输过程，形成了统一数据视图，实现了全局统一数据空间^[1]。

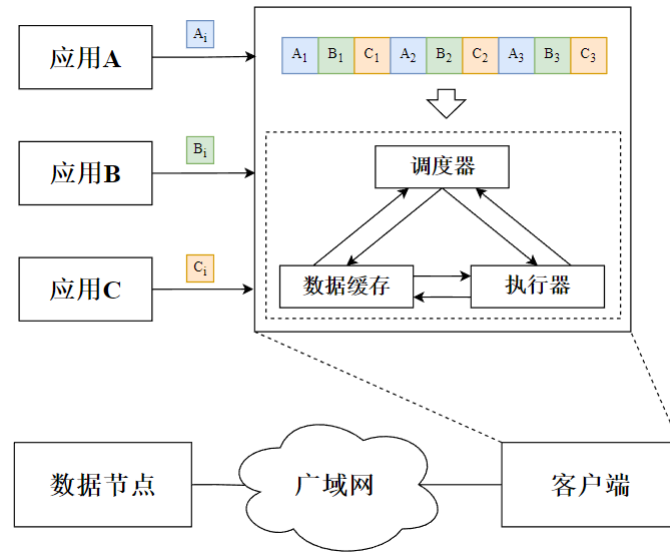


图 2 GVDS 跨域数据访问通路示意图

如图 2 所示，GVDS 已打造出一条基本的跨域数据访问通路。藉由这一访问通路，GVDS 具备了广域数据访问能力，实现了数据存储位置对上层应用的透明性。跨域数据访问通路包含三大关键路径，分别为调度、执行以及数据缓存。执行器模块负责管理网络 IO，通过多种传输协议、日志机制，为广域数据访问提供了灵活、可组合的基础传输能力；调度器模块负责管理广域数据访问任务，通过特定调度策略管理广域数据访问任务，提升广域数据访问效率；数据缓存模块负责管理数据块临时存储，通过可配置预取策略的数据缓存，突破广域网络延时造成的传输瓶颈。

如图 3 所示，本文使用基准测试工具 FIO^[5]分别测试了域内数据请求、跨域数据请求在不同块大小下的吞吐量表现。实验结果表明，受到广域网络的制约，跨域数据请求的吞吐量显著低于域内数据请求，无法满足大数据、人工智能等应用的跨域数据访问需

求。因此，研究面向 GVDS 的跨域数据访问优化技术具有重大意义。

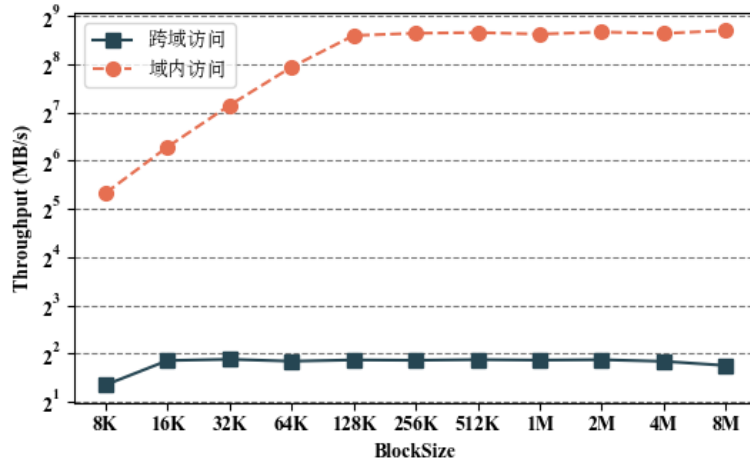


图 3 跨域与域内数据访问性能差异图

综合来说，广域分布式环境中跨域数据访问存在以下三大问题：

(1) 广域网络波动，跨域数据访问请求难以完整执行导致可靠性差。当发生短暂的网络中断时，基于重传的 IO 请求执行策略一方面会额外消耗大量的网络 IO 资源和 CPU 计算资源，另一方面会导致较长的请求执行时延；当达到重试阈值时，IO 请求会被丢弃，最终导致 IO 请求失败。所以，在 GVDS 中构建跨域数据访问的可靠执行机制，优化重传过程，提升跨域数据访问可靠性具有重要意义。

(2) 广域带宽受限，大块跨域数据访问请求独占带宽资源导致公平性低。不同应用的 IO 请求会竞争带宽资源，当采用自由竞争方式时，部分应用的 IO 请求占据大量带宽资源，导致其余应用出现“饥饿”现象；当采用串行执行方式时，跨域数据访问请求依次执行，导致长尾延时高且网络带宽利用率低。针对多应用场景下的跨域数据访问公平性问题，本文拟建立面向多应用的带宽公平分配策略，为各应用均衡地分配带宽资源。

(3) 广域应用复杂，单一跨域数据预取策略时常失效导致性能不足。数据预取是提升跨域数据访问请求性能的关键技术，当应用保持特定负载时，预取策略能够发挥显著优势；但当应用负载复杂多变时，单一的跨域预取策略不仅会产生无用的数据预取任务，而且会导致正常的跨域数据访问请求受到影响。所以，本文拟结合不同应用的数据访问特征，研究跨域数据访问的高效预取策略，提升跨域数据访问的性能。

综上所述，在 GVDS 中构建可靠执行机制、带宽公平分配机制、高效数据预取策略，以优化跨域数据访问通路是有必要的。

1.3 研究目标与内容

1.3.1 研究目标

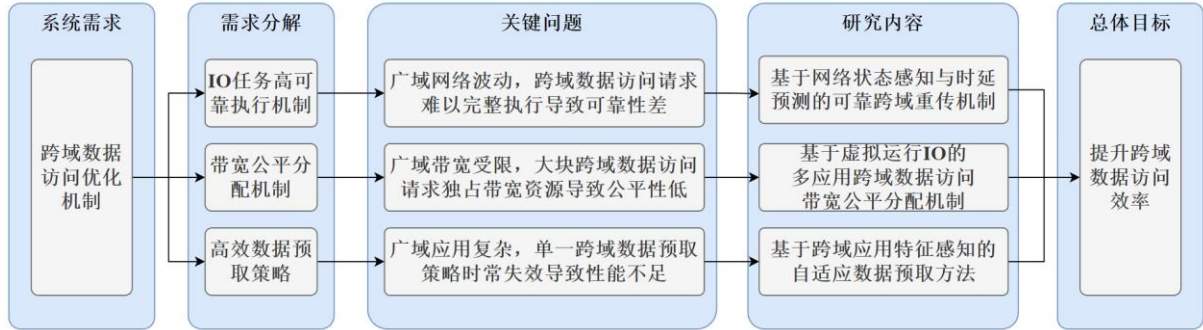


图 4 论文研究目标

如图 4 所示，针对广域虚拟数据空间系统（GVDS）跨域数据访问通路中存在的可靠性差、公平性低、性能不足等问题，本文拟研究 GVDS 跨域数据访问通路中的 IO 任务高可靠执行机制、带宽公平分配机制和高效数据预取策略等关键技术。最终面向多种广域应用访问负载，在 GVDS 中实现高效的跨域数据访问。

1.3.2 研究内容

论文拟研究面向广域虚拟数据空间系统的跨域数据访问优化技术，针对跨域数据访问通路中存在的可靠性差、公平性低、访问性能不足等问题展开研究，优化跨域数据访问通路，提升整体应用效率。具体的研究内容包括：

（1）基于网络状态感知与时延预测的可靠跨域重传机制

广域网络波动，跨域数据访问请求难以完整执行导致可靠性差。在网络异常时，现有的 IO 任务执行机制可能导致大量资源浪费、处理延迟及请求失败。因此，在 GVDS 中实现可靠的跨域数据访问机制以增强数据访问可靠性具有重要意义。

为了保障分布式存储系统中 IO 任务的可靠执行，研究人员主要针对持久化可靠性和传输过程可靠性两方面开展研究。在持久化可靠性方面，以快照备份（Snapshot）和预写日志（Write Ahead Log, WAL）为代表的事务操作，保障了 IO 持久化的可靠性，但未针对广域网传输延迟的做出针对性优化；在传输过程可靠性方面，以 ACK、重传为代表的传输机制，保障了数据包传输过程可靠性，但其主要针对单个报文完整提交场景，且存在一定的性能瓶颈。

针对上述问题与现状,本文感知广域分布集群之间的网络链路状态,预测网络异常,推定恰当的重传时机,并结合 GVDS 的多副本机制,实现 IO 请求快速重定向。最终,构建一套基于网络状态感知与时延预测的可靠跨域重传机制。

(2) 基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制

广域带宽受限,大块跨域数据访问请求独占带宽资源导致公平性低。不同应用的跨域数据访问请求会竞争带宽资源,现有的自由竞争方式导致了“饥饿”与长尾延时高的现象,使得多应用间公平性不足。因此,在 GVDS 中实现公平的带宽资源分配机制以增强应用的公平性是十分必要的。

目前,众多研究人员已经针对应用的带宽分配展开了广泛的研究,出现了许多不同的分配方法。其中,以 TACD^[6]为代表的隔离公平分配方法,利用带宽预留等手段保证资源的均等分配,但未能充分利用带宽资源;以完全公平调度^[7]为代表的任务隔离公平分配方法,通过请求调度协调 IO 任务执行顺序,在宏观上实现带宽公平分配,但未考虑应用数据访问负载的动态变化,且忽略了跨域数据访问请求慢启动的特性。

针对上述问题与现状,本文依据完全公平思想,引入虚拟运行 IO 作为决策依据,协调 IO 请求之间的执行顺序,实现多应用、高并发场景下,带宽资源的公平分配和充分利用。最终,构建一套基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制。

(3) 基于跨域应用特征感知的自适应数据预取方法

广域应用复杂,单一跨域数据预取策略时常失效导致性能不足。数据预取是提升跨域数据访问请求性能的关键技术,现有的单一数据预取策略在应对复杂场景时难以发挥效用,甚至可能因错误触发而导致性能下降。因此,在 GVDS 中实现灵活的数据预取策略,以应对复杂多变的应用访问负载对跨域数据访问的性能至关重要。

为提高分布式存储系统中缓存的命中率,研究人员提出了许多数据预取方法。其中,以 Pacaca^[8]为代表的顺序预取方法,利用快速增量、预取触发等手段大幅提升了顺序读场景下的访问性能;以 ALIS^[9]为代表的关联预取方法,通过访问路径与语义距离等方法发掘文件块之间的相关性,大幅提升了关联读场景下的访问性能。但大多数现有的预取方法通常只针对单一负载,在其它负载场景下无法发挥作用,甚至会由于额外的数据传输而导致系统整体性能下降。

针对上述问题,本文结合应用的不同访问负载特征,综合多种预取策略在不同访问

负载下的优势，通过自适应调优的手段选取准确、合适的数据预取策略，实现数据缓存的命中率提升，提升跨域数据访问性能。最终，构建基于跨域应用特征感知的自适应数据预取方法。

1.4 主要贡献与创新之处

本文面向广域虚拟数据空间，以提升广域分布式存储系统中跨域数据访问效率为目标，通过充分调研、分析和总结国内外相关工作，针对广域网络波动，跨域数据访问请求难以完整执行导致可靠性差、广域带宽受限，大块跨域数据访问请求独占带宽资源导致公平性低、广域应用复杂，单一跨域数据预取策略时常时效导致性能不足等问题，研究跨域数据访问优化方法。在跨域数据访问通路中执行器、调度器、数据缓存三大关键路经分别提出：基于网络状况感知与时延预测的可靠跨域重传机制，以保障跨域数据请求可靠性；基于虚拟运行 IO 的多应用跨域数据访问带宽分配机制，以保障多应用间带宽分配的公平性；基于跨域应用特征感知的自适应数据预取方法，以提升跨域数据访问性能。

本文的主要贡献与创新之处如下：

（1） 基于网络状况感知与时延预测的可靠跨域重传机制

本方法通过分析广域网络环境中的多维度负载信息与系统整体请求执行情况，采用机器学习的方法对网络状况信息进行时序分析，并预测出系统各数据节点的往返时延，以此推测出广域网络异常情况的发生；通过协同广域分布式存储系统中数据多副本分布情况，依据时延预测值，采用分段定时的方法监测请求执行过程，在网络异常发生时，迅速做出请求重定向，避免长时间受异常网络干扰，保障请求可靠执行。

与现有的 IO 任务可靠执行机制相比，本方法通过时序分析的方法准确预测网络异常的发生，可有效降低异常网络的干扰；同时，本方法通过请求重定向的方法，有效利用广域分布式多副本能力，避免请求性能受影响。综上，本方法在保障跨域数据访问请求可靠性的同时，带来了显著的性能优势，适用于广域分布式环境下的跨域数据访问。

（2） 基于虚拟运行 IO 的多应用跨域数据访问带宽分配机制

本方法通过 IO 任务拆分与动态均衡权重，有效衡量应用运行时的负载状况，以指导带宽比例公平分配决策；定义了虚拟运行 IO，更好地量化了 IO 任务的实际工作量，

并作为带宽公平分配的优先级依据；基于动态比例公平分配思想，根据多应用运行时动态权重大小，通过虚拟运行 IO 进行优先级调度，在宏观层面为多应用公平分配带宽资源，保障跨域数据访问公平性。

与传统的调度方法相比，本方法采用动态比例公平分配策略，显著地提升了带宽分配的公平性，并且带来的性能损失较少。与现有的公平分配机制相比，本方法引入了更契合 IO 任务的量化指标，带来了近似的公平性，并显著提升了系统总体吞吐量。综上，本方法在保障多应用跨域数据访问带宽分配公平性的同时，提高了带宽资源利用率。

（3）基于跨域应用特征感知的自适应数据预取方法

本方法通过分析目前典型计算应用的文件访问负载占比，确定应用的主要访问负载，并基于预取步长动态伸缩机制与数据块历史访问路径分析，构建了快速增量预取方法和关联数据预取方法，在针对性场景下提供访问加速；通过滑动窗口模拟预取的方法，动态感知跨域应用的实时访问特征，采取自适应调优方法，选取局部最佳的数据预取策略，保障跨域数据访问在复杂应用场景下的高性能表现。

与现有的数据预取策略相比，本方法通过数据预取策略自调优的方法，准确地感知跨域应用的访问特征，更好地应对复杂应用场景下数据预取的需求，做出合适的预取决策，同时考虑了无规则场景下预取策略的调整机制，避免系统资源浪费。综上，本方法为跨域数据访问带来了显著的性能提升，适用于广域分布式环境下的跨域数据访问。

1.5 论文组织结构

本论文总共分为六章，每一章节的具体内容安排如下：

第一章为绪论部分，首先介绍本论文的课题来源，其次阐述本论文的研究背景与意义，接着概括论文的研究目标和研究内容，最后阐述本文的主要贡献与创新之处。

第二章为国内外相关研究部分，主要包括分布式存储系统中 IO 任务可靠执行研究现状、带宽公平分配机制研究现状、数据预取策略研究现状，最后对本文研究点的国内外相关研究现状进行归纳和总结。

第三章介绍基于网络状况感知与时延预测的可靠跨域重传机制。首先介绍了目前 GVDS 中跨域数据访问的数据传输部分存在的主要问题，并进行了总体性的概括，之后从可靠机制、传输机制等方面对该问题进行了具体分析，接着根据模块划分思想对本论

文中提出的跨域可靠执行机制的方案设计进行了详细的描述和分析，最终通过实验验证了该方法在保证跨域数据访问可靠性的同时，优化了访问性能。

第四章介绍基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制。首先介绍了目前 GVDS 的 IO 任务执行现状，接着对本论文中提出的带宽公平分配机制进行了展开阐述，并对其中添加和调度两大关键步骤进行了详细描述，最终通过实验验证了本方法在保证多应用公平性的同时，提高了带宽的利用率。

第五章介绍了基于跨域应用特征感知的自适应数据预取方法。首先介绍了目前 GVDS 数据的缓存与预取模块现状和问题，接着从多级缓存机制、快速增量预取、数据关联性提取、预取策略自适应调整多方面对本论文提出的自适应数据预取方法的详细方案设计进行了描述，最终通过实验验证了该方法使得 GVDS 系统在应对不同的数据访问特征时均有较好的性能表现。

最后的总结与展望部分对本文的工作进行了总结概括，并指出了本文所做工作中存在的不足与局限性以及未来的工作规划。

第二章 国内外相关研究

本论文旨在设计并实现广域虚拟数据空间中更高效的跨域数据访问通路，针对跨域数据访问通路中的三大关键路径，进行可靠性、公平性、性能上的优化。本章节中，介绍并分析了目前国内外在跨域数据访问通路的三大关键路径上的相关研究，包括 IO 任务可靠执行、带宽公平分配、数据预取策略，并对目前的研究现状进行了总结。

2.1 IO 任务可靠执行研究现状

对于分布式存储系统来说，IO 任务执行的可靠性是十分关键的，这关系到使用该系统的用户体验与系统整体的可用性。IO 任务的执行主要包括数据持久化与数据传输两个步骤，研究人员主要从这两个方面来探索提升 IO 任务执行过程中可靠性的相关技术。

2.1.1 面向数据持久化的可靠性机制

现代分布式存储系统为了保证系统数据的一致性与可用性，在数据持久化的过程中经常采用各式日志机制保证可靠性与完整性。

快照（Snapshot）是一种基于写时复制的备份技术，通过定时将系统运行时的数据以压缩存储的形式进行备份，提供了系统数据持久化的可靠性，并用于故障时的快速恢复，在分布式存储系统中通常作为备用的保障手段。Redis 在其数据持久化过程中采用快照技术，以系统设定的间隔生成 Redis 数据备份文件，提供了一定容错性^[10]。HDFS^[11]的 Secondary NameNode 定期使用快照技术将系统数据进行持久化，以供 NameNode 宕机时快速恢复。

预写日志（Write-Ahead Log, WAL）是一种可靠事务日志技术，它在每次数据落盘之前，将该次数据操作抽象为一条指令，以追加的形式写入 WAL 中，用于系统出错时快速恢复。因其高可靠与高吞吐的特性，在目前主流分布式存储系统中得到了广泛应用。MySQL 通过 Redo/Undo Log 实现了 WAL，对于用户的每一次操作，存储引擎在实际数据更新前将该操作写入 Redo Log，这样即使发生系统掉电等异常情况，系统仍可以快速恢复，而 Undo Log 用于事务执行中途的快速回滚^[12]。ElasticSearch 在 WAL 的基础上引入了 translog，在每次操作时均进行了日志记录^[13]。Etcd 通过在数据目录中添加 wal 和 snap 两个子目录实现了 WAL，用于记录整个数据变化的全部历程^[14]。与大多数的分布

式系统相似, ZooKeeper 内置了 WAL, 对于每一个更新操作, ZooKeeper 使用 WAL 保证数据持久化过程的可靠性, 同时使用快照技术定期将目录树数据持久化到硬盘中, 加速故障恢复过程^[15]。

在上述系统当中, 对于系统的全部操作几乎都要写入 WAL, 给存储资源带来了压力, 部分无法忍受该开销的系统开始对 WAL 进行优化。文献^[16]提出硬件层面的 Undo-Redo 日志记录方案, 通过利用缓存中使用的写回-写分配策略来维护数据持久性, 但其依赖于特殊的 NVRAM 实现, 应用受限。IndexFS^[17]在元数据服务器中采用 WAL 日志记录变更操作, 用于崩溃时系统的恢复, 但仍旧需要日志落盘。G-Store^[18]对关键协议动作采用 WAL 记录, 用于节点崩溃后的恢复, 并且和其他的日志进行分离, 通过使用少量记录, 减少了性能损失, 但相应的恢复完整性也减弱了。bLSM^[19]采用 WAL 管理元数据和空间分配, 确保了崩溃时结构树的一致性版本可用。

2.1.2 面向数据传输的可靠性机制

在数据传输过程中, 目前已经有许多手段用于保证可靠性, 其中主要分为重传、冗余及混合 3 种方法。

重传是指在一方发出数据后, 通过应答和定时器机制确保在一定时间内收到另一方的应答, 否则触发重传直至成功或达到最大重传次数。重传的方法简单, 并且能达到较高的可靠性, 但是重传的时机需要通过准确的机制或算法来保证, 否则错误且过多的重传会消耗更多的网络资源。TCP 是一个可靠的传输层协议, 其使用以往发送报文往返时延加权和估计出的网络延迟作为重传时机的决策, 提供了对每个报文的可靠性保障。QUIC^[20]协议是一个构建在 UDP 协议之上的可靠传输层协议, 在提供 TCP 相当的可靠性保障的同时, 大幅优化了链接建立开销与传输速率。ESRT^[21]协议是一个无线传感器网络领域的包含可靠性和拥塞控制的传输协议, 其首先提出了“事件可靠性”, 目标节点能够将当前网络状况和期望可靠性作为决策依据, 以广播的形式控制源节点自适应调整, 从而保证了数据传输的可靠性而不至于产生拥塞。PSFQ^[22]协议通过多个定时器来保证传感器节点间数据传输的可靠性, 但其计算方法固定, 不能适应动态多变的广域网络环境。

冗余机制是通过在发送数据中添加一定量的冗余数据来达到较高的成功传输概率, 可分为前向纠错和多径路由两种方法。前向纠错是一种在可靠性较差的弱网环境下进行

数据传输时控制数据误码率的技术。通过前向纠错技术,可以减少重传时需要传输的数据量,但是额外的数据也会给网络增加一定的负载,HDFS 等分布式存储系统也将前向纠错纳入了传输技术栈中。多径路由^[23]是指网络传输路径中存在的多条路径,它通过将传输的数据通过分散的多条路径进行传输,从而达到均衡网络环境中负载的效果,同时也能够在一定程度上提高网络传输的可靠性和安全性^[24]。但广域网络不可控,无法由应用来控制选择路由的选择,对于广域虚拟数据空间的跨域数据访问通路并不适用。

为了达到满足期望可靠性的同时达到更高的能效和更低的时延,有研究者提出了将重传和冗余结合起来的方法,例如 RELAX^[25]协议提出了一种结合前向纠错的多径路由协议,该协议主要思想是在每次在传输数据包之前,RELAX 通过较小权重的 FEC 来增加冗余,以此提高协议的可靠性。文献^[26]将端到端的可靠传输问题按照网络层次进行子问题分解,并且分别选择一种复杂性较低的启发式算法,在保证端到端的可靠传输的同时,最大化了网络生命周期。

在主流的分布式存储系统中,例如 HDFS^[11]、Lustre^[27]、Ceph^[28]等,其面对的主要场景为单个数据中心内分散在机房各个机架之上存储服务器存储资源的共享和统一管理。这些服务器通过超高带宽的局域网进行连接,大幅缩短了彼此之间的通信时延,并且具有很强的抗干扰性。因此,它们更多的是从对磁盘的使用、对负载的分配等方面对数据持久化进行优化,极少需要考虑网络环境对于存储系统的影响。HDFS 通过构建物理机架的网络拓扑结构来智能化选取数据的放置位置,每次读数据时选择拓扑结构位置最近的节点进行读取,从而大幅度提升数据传输效率,但跨域多中心场景下这样的拓扑结构的构建和维护成本都是极高的。MadFS^[29]广泛使用远程直接内存访问(Remote Direct Memory Access, RDMA)技术,通过 InfiniBand 原子操作保证每一个数据访问请求的可靠性,同时大幅度优化了数据传输链路,降低延时,但是该技术有严格的距离限制,不适用于广域网场景下的数据访问。

综上所述,广域虚拟数据空间的跨域数据访问中受到广域网络的严重限制,提升数据传输的可靠性至关重要,因此本文从数据传输可靠性机制着手,研究跨域数据访问的 IO 任务可靠执行机制。

2.2 带宽公平分配研究现状

对于广域分布式存储系统，跨域数据访问请求直接依赖于网络带宽资源。多应用所产生的 IO 任务，在广域网络环境中并发执行，公平的带宽分配可以大幅提升系统的可用性和运行效率。随着众多学者多年的研究，目前已经出现了各式各样的带宽公平分配方法，依据其控制粒度，可以分为应用隔离公平分配方法和任务隔离公平分配方法。

2.2.1 应用隔离公平分配方法

应用隔离公平分配方法利用带宽预留等手段，保证应用在每时刻都能公平地分配到均等的资源。其中主要分为面向单资源的分配方法与面向多资源的分配方法。

对于单资源的分配，研究人员从资源的服务质量（Quality of Service, QoS）、利用率等多维度元素出发，为应用提供公平的分配建议。mClock^[30]依据比例分配的公平性，在 Hypervisor 中实现虚拟机之间公平的 IO 资源分配，其 QoS 控制在隔离虚拟机性能和提供更好的应用延迟非常有效，但在维持强隔离性方面存在较大的开销。SRP^[31]构建虚拟机中的存储资源池，通过分层隔离和共享机制实现虚拟机之间池级别的资源比例分配，但未实现自动化的控制手段。Triage^[32]通过自适应资源控制器实现统一存储基础架构下的多个负载之间性能隔离的资源分配，但其提供的公平性有限。Zygaria^[33]为大规模存储系统构建用户隔离的吞吐量池，通过分层令牌桶和 IO 调度实现池内资源预留和限制，保证了每个应用都能获得适当的性能水平，但导致系统中留存了一定的未使用资源，造成资源浪费。Parda^[34]在没有任何存储阵列的支持下实现了高效的分布式 IO 资源管理，提供了公平的端到端虚拟机资源分配，但无法自适应识别工作负载的显著变化。

对于多资源的分配，一些研究人员基于其主导资源进行合理的分配。DRF^[35]对于用户的差异化资源需求，提出了显性自愿公平性，通过一定的激励机制为用户提供更公平的资源比例分配方案。一些研究人员在 DRF 的基础上解决了 DRF 在系统整体瓶颈资源上的划分问题^[36]。Gutman 和 Nisan 将 DRF 扩展成了更通用的方案^[37]。DRFH^[38]在 DRF 的基础上实现了多服务器与多类资源之间的公平分配。上述方法实现了多种资源的比例分配，但由于采用资源限制的方法，导致整体的资源利用率较低。

为了提高资源的利用率，一些方法放宽了对于公平的限制。UAF^[39]考虑所有异构服务器之间的资源分配，在实现公平性的同时提高资源利用率。BAA^[40]是一种基于用户本

地瓶颈资源的定位方法,可以提高多层存储系统中的资源利用率,并且其分配满足 DRF 中描述的公平性,但是它的资源利用率会随着用户数量的增加而降低。TACD^[6]在异构存储系统中存储资源公平分配的问题上引入了经济博弈论的思想,提出了基于用户工作负载的吞吐量分配方法,保持了较好的资源整体利用率。但是上述方法仍没有突破资源预留所带来的空闲资源问题。

2.2.2 任务隔离公平分配方法

任务隔离公平分配方法通过请求调度协调任务的执行顺序,在宏观上实现资源的公平分配。依据其公平性的程度,主要分为绝对公平调度器、面向 QoS 的调度器、机会主义调度器。

绝对公平调度器是基于均分的思想,利用调度轮转的机制使得调度单元之间分配到绝对公平的资源。轮询^[41] (Round Robin, RR) 是最典型、最简单的调度方法之一,它为每个调度单元分配相同的时间片,而后在调度周期中,每个调度因子能够占用相同时长的全部资源以保障公平性,但这种绝对公平性使得规模较大的调度因子分配的宏观资源受限,执行时间被大幅拉长。公平队列^[42] (Fair Queuing, FQ) 通过划分队列,并对队列的链路容量进行限制来保证不同任务共享有限资源时实现公平性,防止一些规模较大的任务或大量小作业的任务占用更多的 CPU 时间,但这样的做法同时也使得一些任务的资源需求经常得不到满足。

面向 QoS 的调度器是以最大化应用 QoS 为目标,进行资源分配的调度算法,以保证应用之间的相对公平性。加权轮询^[43] (Weighted Round Robin, WRR) 在 RR 算法的基础上为每个调度因子添加了权重,而后依据其权重为每个调度因子分配相应比例的时间片,以保障因子间按照权重的比例公平性,但是其权重大多情况为静态分配,动态适应性不足。赤字轮询^[44] (Deficit Round Robin, DRR) 是 WRR 的一种变体,依据赤字机制动态调整每个调度因子的权重,并且允许负权重,保证了资源的充分利用,但是对于一些低延时需求的场景效果表现不佳。加权公平排队 (Weighted Fair Queuing, WFQ) 在 FQ 的基础上引入权重,使得其能够满足大规模任务的资源需求,但也面临着与 WRR 同样的需要静态分配权重的问题。最大最小公平 (Max-min fair, MMF) 在满足每个任务最小需求的资源之后,将剩余没有使用的资源均匀分配给资源需求大的任务,但是当所有任务都拥有相同的需求时,这类方法则会产生多余的资源分配操作。

机会主义调度器是一种被广泛应用于无线网络领域的调度方法，其基本思想是依据系统的瞬时状态来调整调度策略，主要目标是保证机会公平性。比例公平调度^[45]（Proportional Fair Scheduling, PFS）追求公平性与吞吐量的折中方案，其尽可能调度瞬时性能较好的任务，同时依据任务的等待时间调整优先级，以保证每个任务都能够得到调度机会。

除此之外，在一些实际系统中，学者们改造出了更有针对性的调度方法。文献^[46]通过虚拟化技术中的 Dom0 和 DomU 技术进行资源的绝对公平分配，但其依赖于虚拟化技术，并不适用于广域分布式存储系统。Themis^[47]基于完成时间公平与竞价调度的机制构造调度算法，其对于规模相似的多个任务之间有很好的效果，但不适用于分布式文件系统中大小差异很大的任务，适用范围有限。文献^[48]研究面向网络效用最大化的公平调度问题，基于 Lyapunov 框架，构造了低复杂度在线分布式算法，但它参数的调整过程对于广域环境下资源消耗过大。Garcia-Garcia^[49]与 Kim^[50]在内核级别修改 Linux 调度器，针对非对称多核调度场景，从公平性和一致性两方面研究公平调度机制，但它受限于场景，并不适用于传输任务的调度。

综上所述，应用隔离公平分配方法实现了强隔离的均等资源分配，但由于过多追求资源的隔离性和预留问题，使得资源整体的利用率较低；而任务隔离公平分配方法通过任务调度实现资源的宏观公平分配，但未考虑应用跨域数据访问负载的动态变化，且忽略了跨域数据访问请求慢启动的特性，在跨域数据访问的带宽分配上应用受限。因此，本文结合跨域数据访问的特点，并综合广域分布式应用访问负载的动态变化，设计适用于跨域 IO 任务的公平调度方法，实现多应用间带宽公平分配。

2.3 数据预取策略研究现状

在具有密集型 I/O 的现代分布式存储系统中，经常采用预取策略缓解分布式存储系统中的访问延时，利用空闲的带宽通过对用户行为模式的预测获取将来会被访问的数据，提升缓存的命中率和数据访问性能。值得注意的是，预取策略通常与缓存策略搭配使用，二者可以相互补充，因为缓存利用时间信息预测再次访问请求的对象，而预取策略则利用空间信息预测再次访问请求的对象。对于预取策略，根据其访问规律，可以分为顺序负载和关联负载两大类。

2.3.1 顺序读负载预取策略

顺序读是应用对存储系统的重要访问特征，较高的有效性和通用性使得它在众多常见分布式存储系统中得到广泛使用。

Ceph^[28]采用读缓存预取技术，以减少客户端和元数据服务器、数据服务器之间的网络通信开销，从而获得更好的访问性能。Ceph 客户端在请求元数据时，元数据服务器不仅会返回当前请求的元数据，还会返回相同目录下的部分元数据，通过元数据超量读取的方式弥合网络开销。Ceph 的数据预取只针对顺序读场景进行了设计，通过计算应用程序顺序读位置是否到达预取点和是否满足对齐要求等触发超量读预取。Ceph 中的预取策略实现比较简单，在一定程度上利用闲置的网络带宽弥合数据读取之间的速度差距，但是针对频繁变化的应用负载和特殊需求显得力不从心。

GFS^[51]是由 Google 首先提出并投入商用的大规模分布式文件系统，HDFS^[11]则是在 GFS 文章基础上的社区实现。上述两个系统的预取都依赖于简单的启发式策略，在顺序读时将数据请求进行拓展，使数据节点超量返回数据，从而实现更高的 I/O 吞吐量。但是，上述两个系统的预取策略并不能做到动态调整，且预取策略的启动依赖于用户调用的编程接口。这样的预取策略依赖于编程人员丰富的经验，对于代码编写者提出了更高的要求。

HopsFS^[52]是对 HDFS 的一个改进，元数据服务器使用关系型数据库极大地拓展了元数据预取的有效性。它使用新型的分布式关系型数据库作为元数据的存储方式，通过父节点 inode 的 ID 进行分区，这样同一个目录下的文件和目录会被分配到同一个分区中。在预取时，根据元数据的分区情况完成预取操作。

IndexFS^[17]和 LocoFS^[53]使用 KV 数据库处理分布式存储系统中的数据和元数据，他们通过 GIGA+分区方法分布元数据，并且设计了针对性的缓存策略。上述两个系统创新性地拆分了元数据，通过目录的形式预取元数据，但仅预取必要、常用的元数据，如文件名称、文件块位置等。这样使得一个请求中可以携带尽可能多的元数据，提升了系统整体的吞吐量。

GVDS^[1]采用了基于负载特征的数据预读方法，通过带宽负载和 IO 负载的自适应感知，基于预取触发块机制调整预读步长，显著提升了顺序读场景时的吞吐量，但在面对随机读场景时，预读触发块偶尔异常触发导致一定的性能损失。

2.3.2 关联读负载预取策略

关联读负载预取策略通过分析应用对数据块的访问路径,从概率学的角度提供预取决策,可以分为三大类:基于语义模式指导的预取策略、基于文件块关联关系指导的预取策略以及基于性能跟踪数据指导的预取策略。

基于语义模式指导的预取策略。Li 等人提出了 C-Miner^[54], 作者认为之前的许多关于预取的工作都依赖于简单的模式,如时间局部性、顺序性和循环引用,而没有充分挖掘存储系统中的语义模式。在 C-Miner 中,作者使用文件块 I/O 数据使用频率的序列进行匹配,通过块之间的相关性挖掘应该采用的预取模式,并且相比于之前的方法在可拓展性和空间效率方面都具有更大的优势。Kuenning 等人提出了 SEER^[55],其使用文件之间的语义距离来估计它们之间的相似度,并将尽可能多的相关文件预取到移动平台上,以便在网络断开后文件可用。

基于关联关系指导的预取策略。Hus 等人提出了 ALIS^[9],这是一种自动检测文件块局部性改进读取性能的预取策略。该方法通过分析块之间的相关性自动重新组织选定的块,从而按照顺序对相关对象进行聚类 and 预取。Zhang 等人^[56]认为传统的文件系统通常使用逻辑文件到物理元数据表示的一对一映射,这种映射损失了进行优化的空间,所以其提出了一种复合文件系统,允许文件到元数据的多对一映射,并且探索了不同映射策略优劣之处。此文件系统支持目录导向和基于嵌入式引用的合并。这种方法类似于预取,例如,当访问复合文件的元数据时,预取多个关联文件的元数据。在进行文件整合时,可以显著减少冗余元数据信息。因此,它提供了比普通预取更好的访问性能。

基于性能跟踪数据指导的预取策略。Pacaca^[8]是一个客户端缓存和预取框架,集成了对象集群、并行预取和成本感知缓存,以实现更高的云存储性能。然而,其不能动态地适应工作负载的变化;当面对多个工作负载时,它们无法隐藏网络通信造成的延迟。Jamilan 等人提出了 APT-GET^[57],其利用 Intel Last Branch Record (LBR) 等高效硬件支持,收集应用执行时的负载情况,并通过收集到的 Profile 文件表征负载的执行时间,并引入了一种新的分析模型,通过收集到的运行星系找到最佳预取距离和注入位置,从而实现高效的预取。

综上所述,现有大多数现有的预取方法通常只为应用程序提供针对单一负载的预取策略,而不考虑访问负载的动态变化,这种一刀切式的解决方案只能在其针对的负载场

景中发挥性能优势，而无法满足其他各式的应用需求，甚至会由于额外的数据传输而导致系统整体的吞吐率下降，造成负优化。对于广域网络环境中的广域虚拟数据空间，受限的带宽资源和复杂的应用特征，使得单一预取策略并不适用。因此，本文分析并感知跨域应用的访问特征，综合各类数据预取策略的优势，设计适用于跨域数据访问的智能数据预取策略，提升跨域数据访问性能。

2.4 本章小结

在本章节中，介绍了目前国内外在跨域数据访问通路的三大关键路径中的相关研究现状，其中包括 IO 任务可靠执行机制、带宽公平分配、数据预取策略，并且，本文分别对上述方向研究现状进行了分析和总结。对于广域虚拟数据空间中的跨域数据访问通路，目前主流的 IO 任务可靠执行机制、带宽公平分配机制及数据预取策略无法完全满足访问通路对于可靠性、公平性、性能的需求，需要构造一条适用于广域网环境下的高效数据访问通路。

第三章 基于网络状况感知与时延预测的可靠跨域重传机制

3.1 引言

跨域数据传输作为跨域数据访问通路的底层核心路径，对于跨地域的分布式存储系统来说至关重要。目前主流的分布式存储系统通常直接使用现有的网络协议栈完成数据传输工作，例如 TCP、UDP、QUIC、RDMA 等，这些网络协议在经过了一系列标准化工作之后在稳定的网络环境中有着优异的性能表现，但在面临复杂场景时缺少了充分的应变措施。

广域网环境具有带宽波动大、网络不稳定、带宽资源受限等特点，使得作为跨地域分布式存储系统的广域虚拟数据空间在跨域数据传输时面临着较大的网络瓶颈问题。由于网络短暂故障与略长中断在广域网中时常发生，IO 请求与数据在传输过程中时常传输失败并触发重传机制，而重传失败将会导致再次重传，直至超出重传次数上限被丢弃。为保证上层应用与用户数据不发生丢失，广域虚拟数据空间的跨域数据访问通路需要高可靠传输机制，减少重传耗时，并保证跨域数据访问的完整执行。

本文设计并实现了基于网络状况感知与时延预测的可靠跨域重传机制，该机制在保障了广域虚拟数据空间跨域数据访问过程中的可靠性的同时，较大幅度地提升了跨域数据访问性能。在本章节中将从问题与研究思路、详细方案设计、实验与验证三方面详细描述这一机制的技术细节。

3.2 问题与研究思路

对于跨域的虚拟数据空间，应用分布式运行在广域分布的计算集群当中。其数据的访问请求不同于通常的集群内存储系统，广域网高延迟、低带宽的特性，给跨域数据访问请求的可靠性带来了严峻的挑战。

在计算机网络领域，对于网络传输的可靠性保障主要通过冗余、重传或者二者混合的三大类技术手段提供。冗余机制是通过增加数据的数量来达到较高的成功传输概率，对于带宽资源丰富的局域网分布式存储系统来说，是一种合适的方法，但对于广域网数据传输，尤其是网络通信频繁的广域分布存储系统来说，加入的冗余数据会使得本就贫瘠的广域网带宽资源更加受限，使得数据访问性能降低。重传是指在一方发出数据后，

通过应答和定时器机制确保在一定时间内收到另一方的应答，否则触发重传直至成功或达到最大重传次数。这种简单的机制能达到较高的可靠性，但是其效果与重传时机的准确性强相关。当超时时间 RTO 较大时，会导致直接传输失败的请求等待许久才进行重传，使得系统运行效率低，性能较差。当超时时间 RTO 较小时，会导致请求并没有发生丢失就进行重发，进一步增加网络拥塞，并导致更多的超时发生，形成恶性循环。所以，设计合理且准确的重传时间决策机制对于重传机制的有效性至关重要。

目前主流的重传机制通常使用 TCP 协议进行实现，TCP 协议根据数据包的往返时延 (Round-Trip Time, RTT) 对超时重传时间 (Retransmission Timeout, RTO) 进行计算。如公式 3.1 所示，首先计算 RTT 的平均偏差，记为 $RTTVAR$ ；而后依据公式 3.2 计算 RTT 的平滑值，记为 $SRTT$ ；最后，依据公式 3.3 通过平均偏差与平滑值求出超时重传时间 RTO ^[58]。

$$RTTVAR = (1 - \beta) * RTTVAR + \beta * |RTT - SRTT| \quad (3.1)$$

$$SRTT = (1 - \alpha) * SRTT + \alpha * RTT \quad (3.2)$$

$$RTO = SRTT + \max(G, K * RTTVAR) \quad (3.3)$$

当再次超时，TCP 的策略是指数级加倍，即当遇到一次超时重传的时候，都会将下一次超时的时间间隔设置为先前值的两倍。而如果超过重传次数上限，这个传输请求就会被丢弃。从目前广域网数据访问中采用重传机制来保障可靠性的实践上来看，网络短暂中断会产生 1-2 次重传，略长中断会产生 3 次重传甚至超过重传次数上限，严重影响数据访问性能。主流分布式存储系统的数据访问请求通常采用客户端异步提交、服务端后续负责请求完成的机制来提升整体系统吞吐量。而超出重传次数的请求会被丢弃，直接影响系统可靠性。

主流的分布式存储系统，包括广域虚拟数据空间，都内置了十分完善的副本与同步模块，以此在保障了系统高可用性与一致性的同时，也分散了整体的负载情况。同时这也能够为跨域数据访问提供一个新的思路——重定向。对于存储于广域虚拟数据空间中的一个数据块，它可能至少会有两份一样的副本备份存储于不同的数据中心当中，副本的存储位置通常由元数据服务器进行监控和管理，并且在客户端对某一文件进行数据访问时，元数据服务器将会把该文件块所有副本的位置一并返回，并给出位置最近副本的建议。客户端便会根据一些系统内设定的负载均衡策略将数据访问请求分发给各个副本

节点，进行后续的数据获取。但对于其中的某一个数据访问请求，这一目标节点选定了之后，通常不会发生变化。当报文传输失败，则触发 TCP 的可靠重传机制，并始终以该节点为目标进行重传。

实际上，该做法并未充分利用副本的特性。对于跨域分布式存储系统，副本分散在相隔较远的不同数据中心之上，客户端到不同副本节点的网络路径近乎完全不同。因此当到达其中某一副本节点的网络路径发生异常时，其余副本节点的路径有大概率是良好的状态，此时可以将未完成的 IO 任务转而向网络状况良好的副本节点发起请求，即重定向。相比于在异常链路上反复重传，重定向到网络良好的节点能够极大幅度地提升数据访问性能。

当然，这一机制同样需要准确的重传时机把控。TCP 协议只是基于往返时延的单元素时间序列分析与预测，未综合考虑广域分布多数据中心的网络情况，并不适用于跨域数据访问请求的重传时机调优。因此，需要对广域多维度网络状况进行多元时间序列分析与预测，设置合适准确的超时重传时机与再次重传时机，才能降低系统重传次数，并提高系统可靠性与性能。

对于多元时间序列分析，目前主要包括 VARMA^[59]模型、VARX^[60]模型等数学模型，以及包括 LSTM^[61]、Transformer^[62]、Informer^[63]等的深度学习模型。其中数学模型需要极为繁琐的模型检验与参数估计过程，其计算量带来了较高的时间复杂度，而人工智能模型虽然需要较长的模型训练过程，但训练完成之后的推理过程耗时远小于复杂数学模型计算，其中比较适合小维度数据时序分析的是 LSTM 模型。因此本文使用 LSTM 模型来对跨域数据访问过程中的往返时延进行多元时序分析与预测，由此计算出合适的重定向时机。

面临单节点故障或网络原因导致的节点失联时，一方面会导致请求长时间无响应，发送多次无效重传请求，并占用有限带宽资源，另一方面如果持续等待请求的完成，则会阻塞应用的整体运行。所以，当请求失败发生时，需要建立快速重定向机制，将请求重定向到其余持有该数据副本的节点之中，由其完成请求的后续完成。

本章所研究问题的主要难点与挑战在于：

(1) 如何正确判断跨域数据访问请求的重传时机。重传时机的准确性将直接作用于跨域数据访问的性能，稍晚或稍早地触发重传都会引起吞吐量的剧烈变化。所以需要

综合考虑网络链路的多方面要素，并结合日志中请求完成的状态，做出合适的重传时机决策。

(2) 如何高效完成跨域数据访问请求的重定向。重定向需要保证整个请求对于上层的透明性以及请求完成的完整性，即需要正确地打断当前请求执行的流程并保存运行状态，并打开新的流程完成请求的后续内容。

针对以上问题，本文研究了一种基于网络状况感知与时延预测的可靠跨域重传机制。该机制综合网络链路情况与请求提交情况，结合时间序列分析与预测的技术，为跨域数据访问请求的重定向提供准确决策，保障跨域数据访问请求的完整执行，满足跨域数据访问的可靠性需求。

3.3 详细方案设计与实现

3.3.1 符号描述

本章所描述基于网络状况感知与时延预测的可靠跨域重传机制涉及到对于网络状况信息与请求状况信息的建模和分析。为简化模块分析和算法描述中对于参数的推算过程，现对模型相关参数的描述与符号定义如下表 1 所示：

表 1 可靠跨域重传机制相关符号描述表

符号	定义
TimeStamp	代表时间戳
Δt	时间序列数据采集时间间隔
DN	代表数据节点，是数据的实际存储位置和数据操作的实际执行位置
MN	代表元数据节点，是整体元数据维护的位置，包含系统元数据和文件元数据
Rcv	代表接收到的字节数
Trans	代表已经发送的字节数
PkgRcv	代表已接收的数据包数
PkgTrans	代表已发送的数据包数
ErrRcv	代表已接收数据包出错个数
ErrTrans	代表已发送数据包出错个数
RTT	往返时延，指请求从发送到接收确认的时间差
RTO	重传时间，指请求发送后到需要重传的时间
bw	网络带宽
bs	请求字节数

3.3.2 整体框架

本研究点的方案整体架构图如图 5 所示，首先需要在每一个数据节点之上建立网络状况感知模块，以对目前广域多集群中的网络状况信息进行采集，形成连贯的多元时间序列数据流。

时序分析与预测模块基于长短期记忆网络的重传时间预测算法，通过网络情况感知模块收集到的多元时间序列数据流进行训练，生成能够根据一定时间步长的历史往返时延预测失败重传时机的模型。

对于每个跨域数据访问请求，由 FUSE 进行请求的分割之后，异步提交到客户端上，而后由客户端对请求的完成提交与执行提供可靠性保障。客户端请求处理模块基于重传机制，采用基于网络感知的快速重定向算法，根据时序分析预测模块得到的重传时间预测结果，设置重传定时器，当请求超时，执行请求的快速重定向，直到整个请求的完整提交与执行。

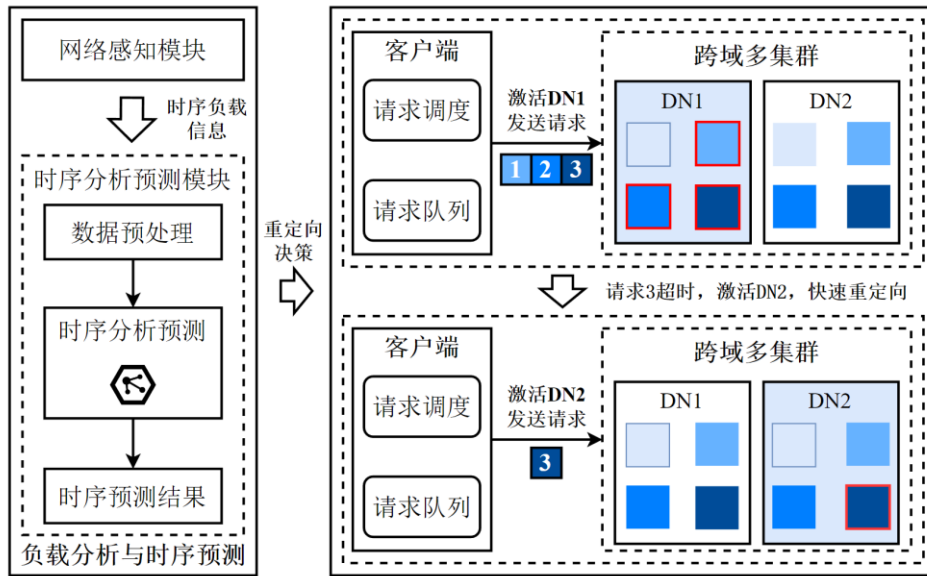


图 5 基于网络状况感知与时延预测的可靠跨域重传机制总体架构

3.3.3 网络感知模块

网络感知模块需要采集广域多集群中的网络状况信息进行采集，形成多元时间序列数据。最后得到一组时间序列数据，每个数据单元为

$$Net_t = \{TimeStamp_t, Rcv_t, Trans_t, PkgRcv_t, PkgTrans_t, ErrRcv_t, ErrTrans_t\} \quad (3.4)$$

此外，网络感知模块需要对跨域数据访问的请求执行情况进行统计记录，其中包括：

$$Req = \{TimeStamp_i, RTT_i, bs_i\} \quad (3.5)$$

3.3.4 时序分析与预测模块

根据以往跨域数据请求的往返时延来对后续请求的重传时机进行决策是时序分析预测模块的主要目标。而网络情况信息在时间维度上存在较强的前后依赖关系，长短期记忆网络（LSTM）是一种适合建模远距离序列依赖关系的神经网络模型，本模块将基于深度学习模型中的长短期记忆网络构建时序分析预测模块。包括数据预处理、时间序列模型训练及时序预测阶段，整体流程架构图如图 6 所示：

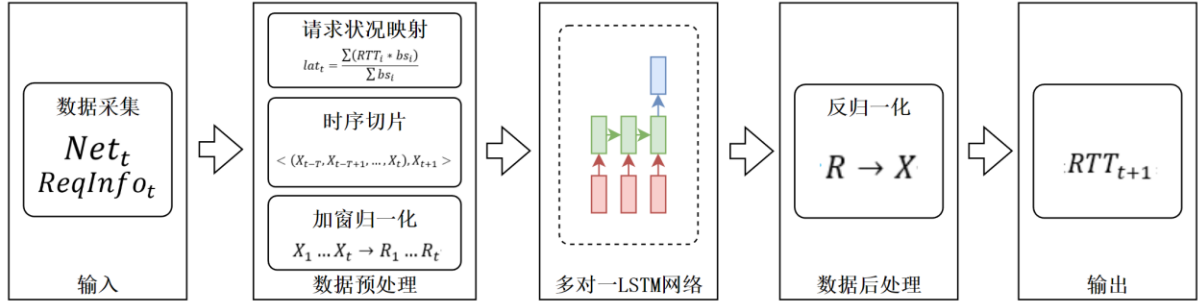


图 6 时序分析与预测模块技术方案

a. 预处理阶段

预处理阶段首先需要对请求数据进行时间映射，得到网络状况信息采集间隔内的平均响应时延，使其和网络状况信息组成一组数据。公式 3.6 给出时间映射方法，对 Δt 时间内的请求进行加权平均，作为该时间戳的响应时延 lat_t 。

$$lat_t = \frac{\sum(RTT_i * bs_i)}{\sum bs_i} \quad (TimeStamp_{t-1} < TimeStamp_i \leq TimeStamp_t) \quad (3.6)$$

于是，得到的单个数据为：

$$X_t = \{Net_t, lat_t\} \quad (3.7)$$

而后使用固定值的滑动窗口对时序数据进行切片，然后进行放缩得到待输入数据。由于模型的目的是通过前 T 个输入数据来预测后 1 个数据，所以每次的切片大小为 $T+1$ ，即：

$$<(X_{t-T}, X_{t-T+1}, \dots, X_t), X_{t+1}> \quad (3.8)$$

由于广域网中的网络情况具有很强烈的波动性，不同的输入时间窗内的数值会有很大的差距，这就导致模型收敛困难，为了消除不同时间窗内数值的差距，本研究点采用加窗归一化的方法，在全局归一化的基础上，引入相邻时间序列的相关性，从而减缓噪声数据的影响。

b. 模型训练阶段

本文采用基础的 LSTM 模型进行构建，出于网络状况的局部性特征，本研究点采用了多对一的长短期记忆网络结构，使用最后一个时间步的长短期网络的输出作为整个观测窗口的特征，这样可以利用长短期记忆网络的“有偏特性”让提取到的特征中包含更多的最近时序信息，从而让整体模型更关注最近点。

c. 时序预测阶段

在这一阶段，使用的为训练好的时序预测模型，为了和模型拟合时的数据处理方式保持一致，提升模型准确性，时序预测阶段同样采用窗口的方式进行模型推理。基于 LSTM 的滑动窗口重传时机预测算法如算法 1 所示：

算法 1 滑动窗口重传时机预测算法

Input:

$Net = \{Net_1 \dots Net_t\}$: 网络状况信息时序数据

$Lat = \{Lat_1 \dots Lat_t\}$: 响应延迟时序数据

RTT_RwLock : 预测结果更新的读写锁

RTT_Shared : 线程共享的预测结果

```

1   $X \leftarrow \{Net, Lat\}$ 
2  for  $i \leftarrow 1; i \leq T; i \leftarrow i + 1$  do
3       $X \leftarrow window\_normalization(X)$ 
4       $RTT\_Normal'_{t+i} \leftarrow model.predict(X)$ 
5       $RTT'_{t+i} \leftarrow window\_denormalization(RTT\_Normal'_{t+i})$ 
6       $RTT\_RwLock.writeLock()$ 
7       $RTT\_Shared \leftarrow RTT'_{t+i}$ 
8       $RTT\_RwLock.unlock()$ 
9       $Net_{t+i}, ReqInfo_{t+i}, RTT_{t+i} \leftarrow collect\ from\ network\ info\ collected\ module$ 
10      $Lat_{t+i} \leftarrow mapping\ ReqInfo_{t+i}\ and\ RTT_{t+i}\ to\ Lat$ 
11      $X.push\_back(\{Net_{t+i}, Lat_{t+i}\})$ 
12      $X.pop\_front()$ 
13 end for
```

算法执行过程中，每次通过数个数据预测下一个重传时间，而后将窗口后移，加入最新的网络信息与请求信息，进行下一轮预测。之后，将得到的预测结果交由请求处理模块，指导其进行请求重定向的决策。

3.3.5 请求处理模块

在请求处理模块中，本文提出基于网络状况感知与时延预测的可靠跨域重传算法，其算法流程如算法 2 所示。

算法 2 基于网络状况感知与时延预测的可靠跨域重传算法

Input:

$ReqInfo = \{Inode, Size, File\}$: 跨域数据访问请求
 RTT_RwLock : 预测结果更新的读写锁
 RTT_Shared : 线程共享的预测结果
1 $\{DN_1, DN_2, DN_3\} \leftarrow \text{datanode addr from metanode by inode}$
2 $\{RTT_1, RTT_2, RTT_3\} \leftarrow \text{RTT of these three datanode}$
3 $DN \leftarrow \text{choose datanode with the min RTT}$
4 $DoneSize \leftarrow 0$
5 **while** $DoneSize < Size$ **do**
6 $RTT_RwLock.readLock()$
7 $RTT \leftarrow RTT_Shared$
8 $RTT_RwLock.unLock()$
9 $Timer \leftarrow \text{set a timer with timeout of RTT}$
10 $size \leftarrow \text{deal with request}$
11 $DoneSize \leftarrow DoneSize + size$
12 **If** $Timer.timeout()$ **then**
13 $DN \leftarrow \text{choose another datanode}$
14 **end if**
15 **end while**

对于每个跨域数据访问之前，客户端会预先向元数据服务器请求所需数据及副本存储的数据节点位置 $\{DN_1, DN_2, DN_3\}$ ，首先对三个节点同时发起连接请求，得到每个节点的往返时延： $\{RTT_1, RTT_2, RTT_3\}$ ，并选择往返时延最小的数据节点作为本次请求的目标节点。接着客户端向数据节点发送数据请求信息，并根据时序分析预测模块得到的往返时延预测结果 RTT_{t+1} ，设置计时器。当请求在计时器超时前执行完成，则不触发超时重传。当请求超时，则将当前未完成的数据部分，并再获取新的往返时延预测结果，向其余数据节点中较优的节点执行相同的数据请求流程，直到完成请求。请求过程中，以日志的形式，向时序分析预测模块更新当前的请求状态，使得预测窗口滑动。

由于写请求中对系统容错性的需求，在数据节点和客户端中维护请求的预写日志，通过 Redo 和 Undo 机制实现节点单点故障时请求的可靠恢复。

3.4 实验与验证

3.4.1 实验环境

在本小节中将对本文所提出基于网络状况感知与时延预测的可靠跨域重传机制进行实验验证。为方便自定义调整网络环境，实验在通过网络相连的四台不同服务器上进行，

其拓扑图如图 7 所示。为模拟出真实的广域网络环境，本节使用 Linux 系统中的流控工具 TC (Traffic Control) 对网络情况进行人工干扰，这一工具支持对网络延时、发送速率、接收速率、丢包率等一系列网络指标的自定义修改。

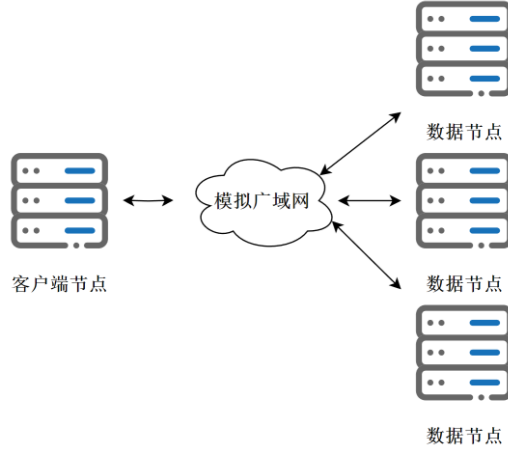


图 7 跨域数据访问实验拓扑图

实验所用实验平台软硬件环境配置如表 2 所示，每台服务器的配置基本一致，并且拥有独立的 IP 以便相互访问。

表 2 可靠跨域重传机制实验平台软硬件环境配置

软硬件	详细描述
CPU	Intel(R) Xeon(R) Silver 4110
内存	64GB
磁盘	AVAGO MR9361-8i 4TB
网络带宽	1000M (测速约 124MB/s)
操作系统	Ubuntu 20.04.4 LTS
操作系统内核	5.4.0
Rust	1.73.0-nightly
Python	3.10.11
Pytorch	py3.10_cpu

3.4.2 实验数据描述

首先，为了构造本方法所需的时序分析模型，本文在上述的实验平台中采集原始训练数据。本文通过 TC 调整网络实验为 $80 \pm 30ms$ ，并持续 6 个小时让客户端向数据节点发起 IO 请求，数据采集的时间间隔为 100ms。在此过程中，根据公式 3.4 与 3.5 所示数据描述采集时序数据，并按照数据预处理方法转换之后，输出到文件当中。图 8 展示了此时序数据前 500 条的分布情况，表 3 给出了该数据集的基本描述信息。

表 3 时序数据集基本描述信息

	RTT	Rcv	Trans	PkgRcv	PkgTrans
最小值	50	0	0	0	0
最大值	116	72974	29341	78	55
平均数	80.42	300.02	426.09	2.99	4.58
中位数	80	210	356	2	4
方差	17.30413892	442.8773574	460.6385779	2.211391142	2.109003132
标准差	299.4332237	196140.3537	212187.8994	4.890250782	4.44789421

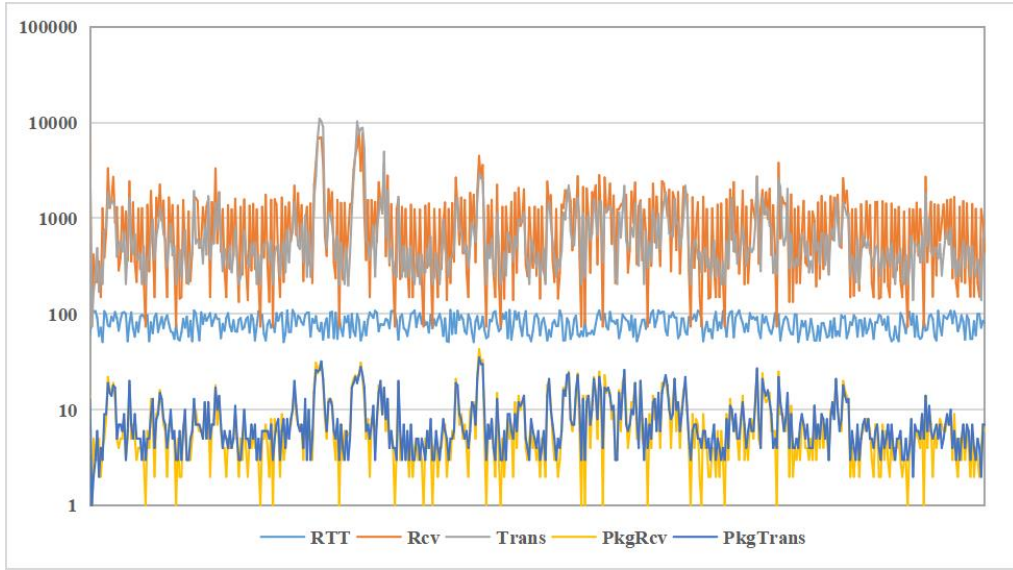


图 8 时序数据集前 500 条数据分布情况

接着，本文通过 Pytorch^[64]实现了简易的 LSTM 网络模型，其基本参数描述如表 4 所示。最后得到了本方法所需的基本时序分析模型，用于在跨域数据传输过程中给出重传时机决策。

3.4.3 高可靠跨域重传机制有效性对比测试

首先，为了验证本方法有效性，本文通过客户端节点发起了 10GB 大小文件的读请求任务，在任务执行过程中通过 TC 命令为客户端节点与所选数据节点之间添加了 $80 \pm 30ms$ 的往返时延，并实时采集了该任务执行过程中的吞吐量，结果如图 9 所示。该图纵坐标表示请求的瞬时吞吐量，每隔 4MB 数据计算一次所耗时间，并以此计算吞吐量，横坐标表示任务的执行时间条，单位为 4MB 数据的执行间隔。从图中可以看到，任务起初保持了高吞吐量的传输速率，但在添加了高时延之后，受网络状况影响，任务的吞吐量出现了骤降。而后本方法发挥效果，将该任务进行快速重定向到另一个数据节点中，

以此恢复高吞吐量的数据传输，最终完成该任务。这进一步说明了本方法在面临广域网延迟波动影响时的有效性。

表 4 模型训练参数表

参数项	描述	参数值
epochs	训练轮数	100
layers	LSTM 层数	2
input_size	输入特征维度	5
hidden_size	隐藏层维度	128
learning_rate	学习率	0.001
sequence_length	预测窗口大小	10
batch_size	训练批大小	64
dropout	丢弃率	0.2
损失函数	用于量化模型有效程度	MSE 均方误差
优化器	反向传播中指引收敛方向	Adam

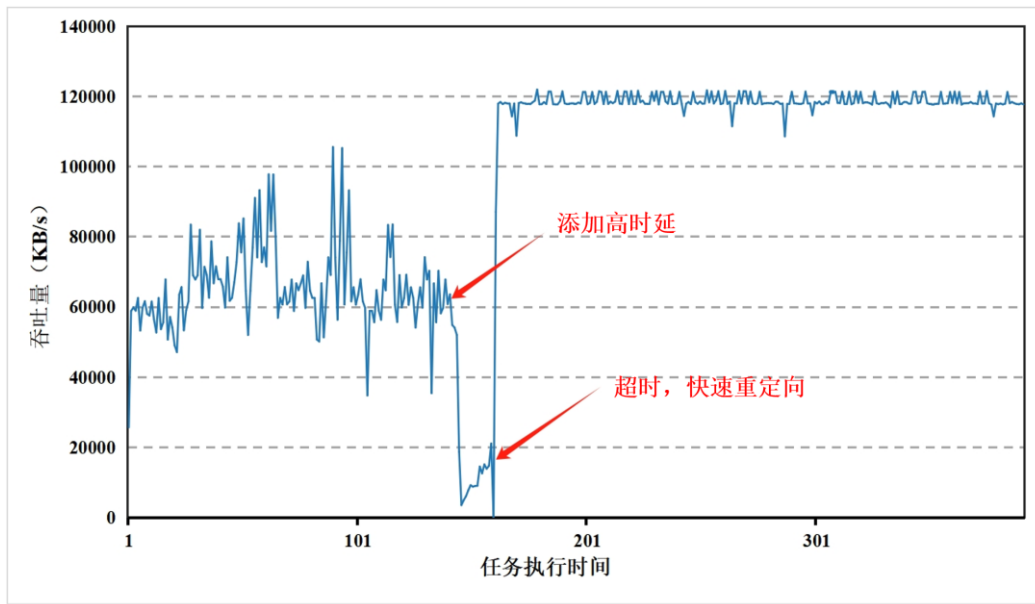


图 9 快速重定向成功生效

接着，本方法选取了 HDFS 与 baseline 作为对照，从多角度进行了对比试验，验证了本方法的有效性。HDFS^[11]是目前工业界与学术界使用最广泛的分布式文件系统之一，为大数据应用提供了底层支撑，本文选取的版本为目前最新版 3.3.6，而本方法的 baseline 为关闭了缓存功能之后的 GVDS 基本版本。为了便于观察实验效果，后续的实验中本文令首次数据节点的选择固定，并使用 TC 命令修改其时延，以测试不同情况下的性能表现。

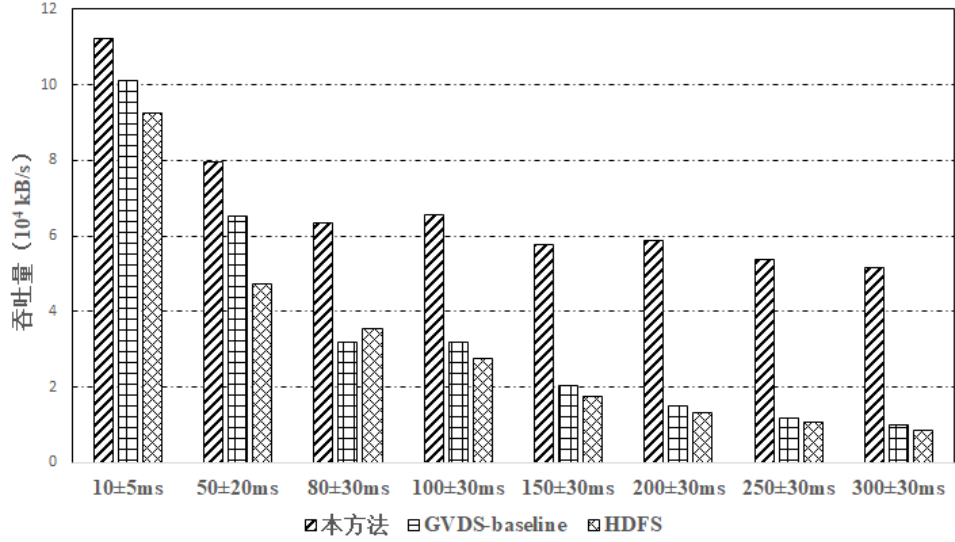


图 10 不同访问延时读吞吐量

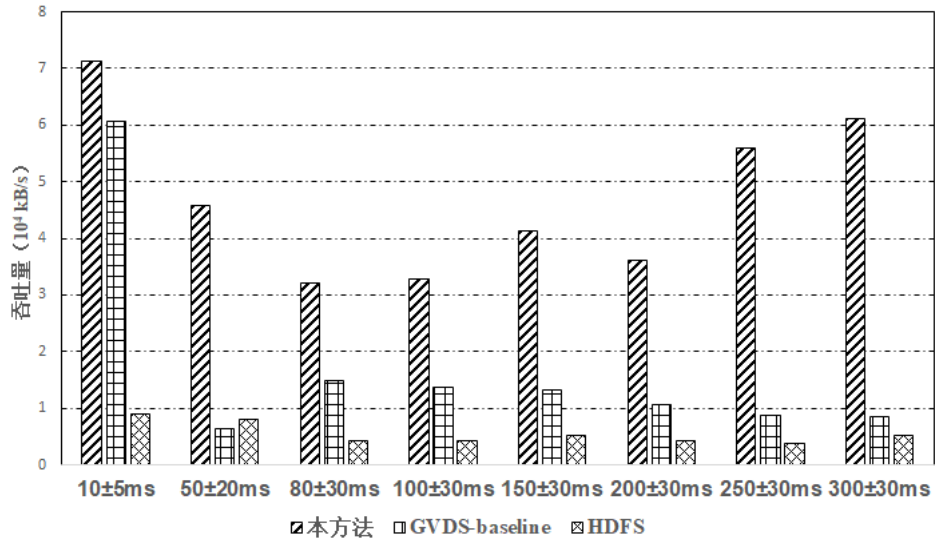


图 11 不同访问延时写吞吐量

图 10、图 11 给出了不同访问延迟环境中，本方法与 HDFS 和 GVDS-baseline 在 1GB 大小 IO 任务下读写吞吐量的实验数据，其中每个时延均附加了一定的波动。实验结果表明，本方法为 GVDS-baseline 带来了巨大的性能提升，并且随着时延的增加，性能提升越显著，在 $300\pm30\text{ms}$ 场景下读写性能分别提升了 4.2、6.2 倍。此外，随着访问时延的上升，三者的吞吐量都有明显的下降趋势，但相比于 GVDS 和 HDFS，本方法的下降幅度更小，并且有保持性能平稳的趋势。这是因为当时延上升时，TCP 的超时重传率也随之上升，此时可能触发 1-2 次重传，降低整体吞吐率；而本方法会预先感知网络

波动，提前进行快速重定向，减少重传时间的浪费，弥补一定的吞吐量，但由于本方法基于固定大小数据片的计时来判定网络状况，所以仍会受到一定的影响。值得一提的是，本方法在超高延时下（300ms），写请求的性能逐步回升，而 HDFS 由于其数据流一致性机制，使得写吞吐量有更大幅度的下降，处于基本不可用的状态。

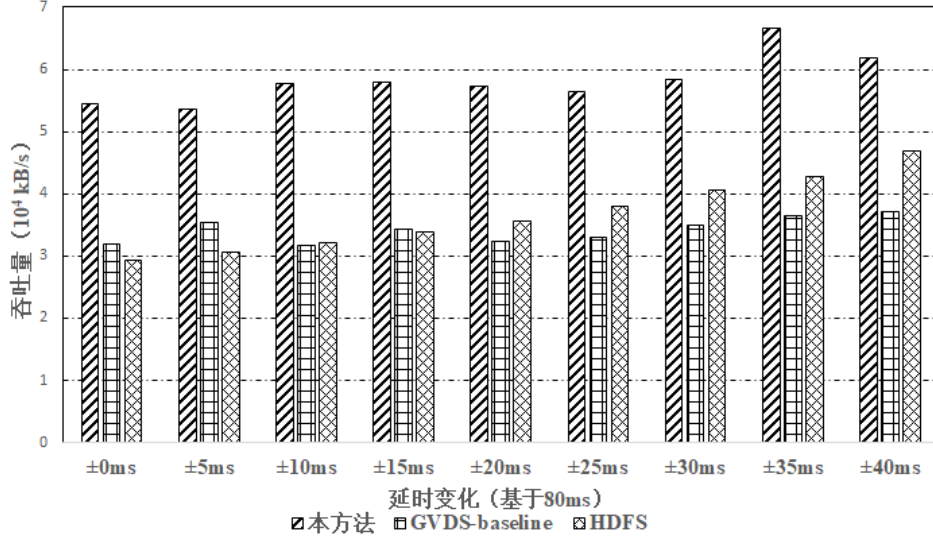


图 12 不同延时波动读吞吐量

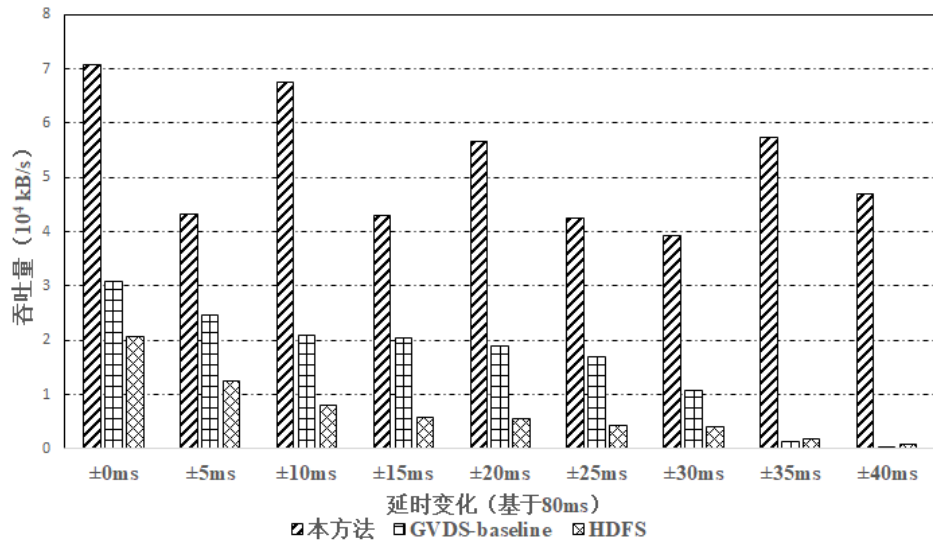


图 13 不同延时波动写吞吐量

为了测试本方法应对不同网络波动的能力，本文通过调整延时的波动值分别测试不同方法在 1GB 大小 IO 请求时的吞吐量，图 12、图 13 分别给出了读写请求的实验结果。实验结果表明，本方法相较于其他方法在各种延迟波动下都有较大的性能提升。并

且，从实验表现中也能发现，随着时延的波动越大，每种方法的读吞吐量都有一定的提升，这是因为在高延迟波动时，其往返时延的上界与下界也随之扩大，使得一些数据包能够在低延时情况下传输，提升了吞吐量。但是，随着延时波动的增大，GVDS-baseline 和 HDFS 的写请求吞吐量随之下降，而本方法也产生了一定的性能波动，但仍保持着高水平，这是因为延迟波动导致副本同步过程中 TCP 重传时机判断不准确，产生了多次重传，造成了性能骤降，而本方法在感知到网络波动时快速重定向到拥有良好网络环境的副本节点中，避免受到该链路的后续干扰。

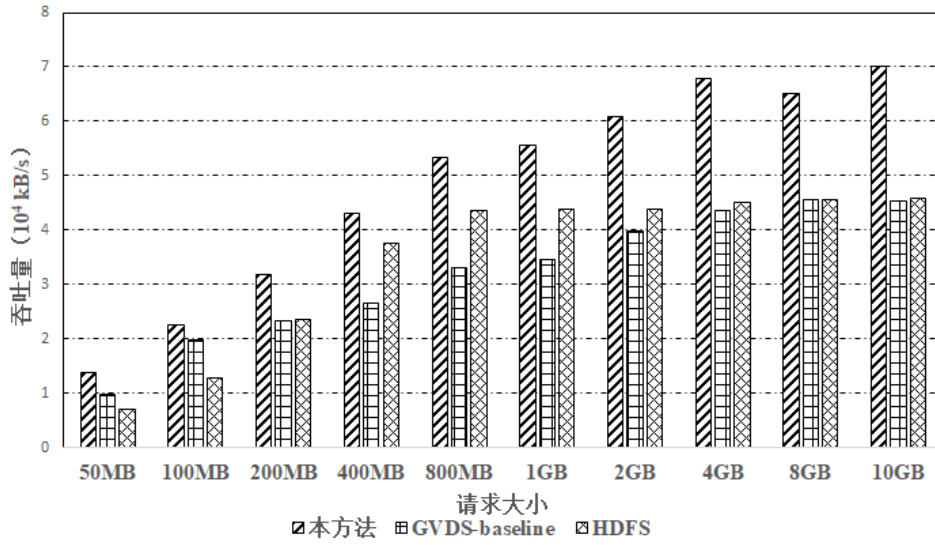


图 14 高延时场景下不同任务大小读吞吐量

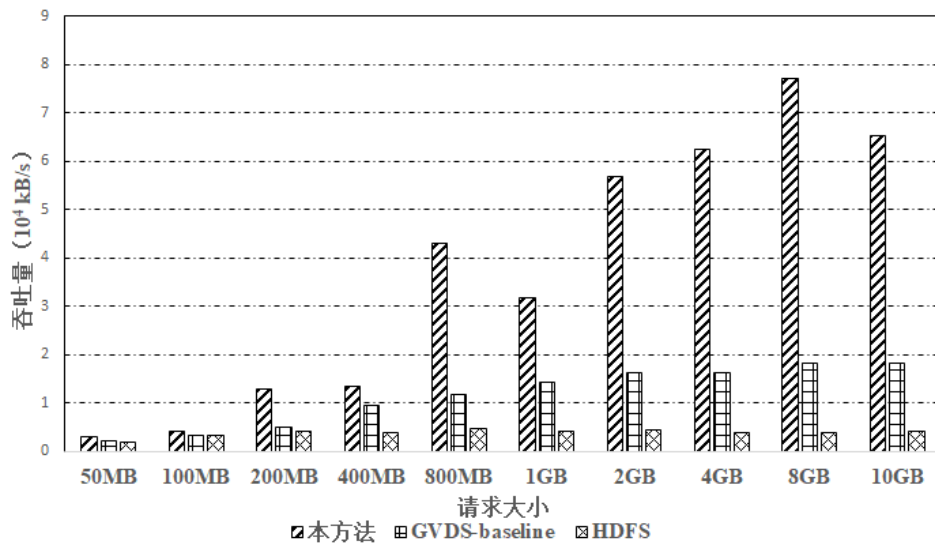


图 15 高延时场景下不同任务大小写吞吐量

接着, 本文测试了 $80 \pm 30\text{ms}$ 场景下, 本方法与其他方法在处理不同大小 IO 任务的性能表现, 图 14、图 15 分别给出了读写请求的实验结果。实验结果表明, 随着 IO 任务的增大, 三种方法的吞吐量都随之增大, 最终趋于平缓, 这是因为 IO 任务有启动的过程, 在该过程中完成元数据的确认和与数据节点网络连接的建立, 而随着 IO 任务增大, 这一过程的占比可以忽略不计。并且, 实验同样证明了本方法相较于其他方法带来了巨大的性能提升。

3.4.4 高可靠跨域重传机制时间复杂度测试

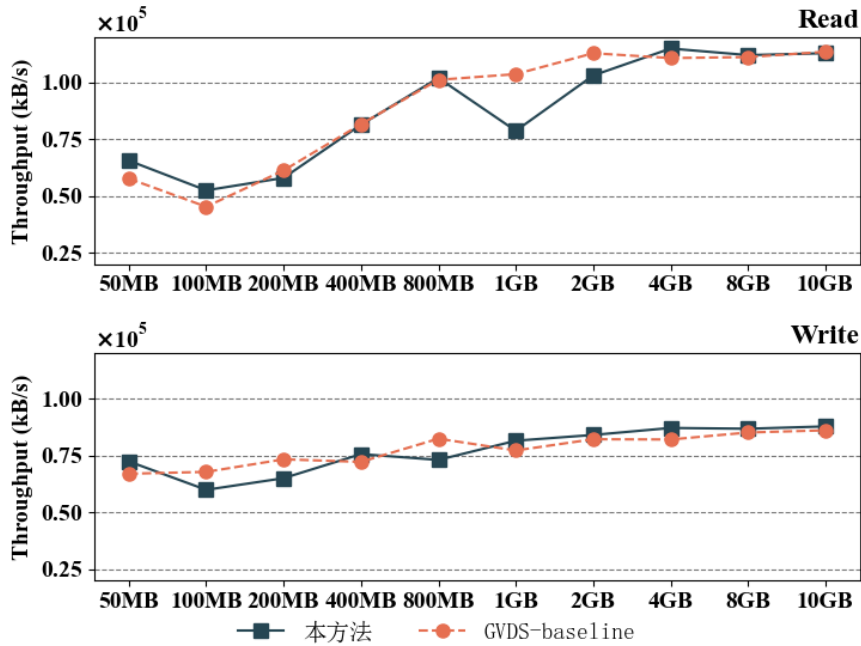


图 16 正常网络环境中不同任务大小吞吐量

为了测试本方法所带来的额外开销, 本文对正常广域环境下 ($10 \pm 0\text{ms}$) 本方法与 GVDS-baseline 的性能表现, 图 16 给出了读写请求吞吐量实验结果。实验结果表明, 本方法与 GVDS-baseline 在正常网络环境中的性能表现基本一致, 说明本方法对于系统运行状态下的性能损耗几乎可以忽略, 这是因为本方法中的核心工作部分均以异步的形式完成, 产生额外开销的地方在于获取预测结果的读锁开销与重定向时的切换开销。

3.5 本章小结

针对广域虚拟数据空间的跨域数据访问通路中面临的可靠性问题, 本章节设计并实现了基于网络状况感知与时延预测的可靠跨域重传机制, 综合广域网络链路情况与请求

提交情况，结合时间序列分析与预测的技术，为跨域数据访问请求的重定向提供准确决策，在保证跨域传输可靠性的同时，提升了数据传输性能。主要的工作如下：

（1）设计并实现了基于网络状况感知与时延预测的可靠跨域重传机制。首先，构建基于网络感知和时序分析的时延预测机制，通过长短期记忆网络与滑动窗口重传时机预测算法，计算目标节点往返时延预测值；其次，基于往返时延预测值，设计快速重定向机制，在网络链路出现异常时，依据预测值做出重定向决策，打断当前数据访问请求，并切换目标节点以完成后续请求，提升跨域数据访问请求可靠性和数据访问性能。本方法在保障跨域数据访问请求可靠性的同时，大幅优化传输性能。

（2）构建了实验系统。在仿真模拟的弱网环境中实现了本章节设计的可靠执行机制，并通过实验对比了本方法和其余方法的性能表现。实验结果表明，本方法能够很好地识别网络异常的发生并做出正确的重定向决策。在面对异常的网络环境时，相较于其余方法，本方法平均能带来 48% 的读吞吐量提升和 175% 的写吞吐量提升，并保证了请求的可靠完整执行。同时实验表明，本方法对系统整体运行开销带来的影响可以近乎忽略不计。

第四章 基于虚拟运行 IO 的多应用跨域数据访问带宽分配机制

4.1 引言

目前,广域虚拟数据空间系统已经通过对广域环境中异构存储资源的多级聚合,形成了数据访问的统一视图,已具备跨域数据访问能力。作为跨域分布式存储系统,其部署在跨地域分散的多个数据中心之上,聚合并管理其底层的存储资源。在此之上,多应用所产生的计算、存储请求任务,在分布式系统中并发执行,若对请求的任务进行正确地调度,可以极大地提升系统整体的运行效率。

但是,对于分布式存储系统,多应用所进行的并发数据访问传输任务之间可能发生带宽等资源竞争问题,而目前主流的分布式存储系统包括广域虚拟数据空间系统采用的自由竞争方式会导致其中部分任务近乎独占整个网络信道,而其余任务则近乎停滞。这对于用户或是应用来说,都是不公平的现象。为了提升多应用跨域数据访问请求之间的公平性,需要建立面向多应用的带宽公平分配策略,均衡各应用的带宽资源。

本文设计并实现了一种基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制,该机制结合完全公平的思想,实现跨域数据访问的公平调度,以保证多应用带宽公平分配,并提升带宽的利用率。在本章节中将从问题分析与研究思路、详细方案设计、实验与验证三方面详细描述这一机制的技术细节。

4.2 问题分析与研究思路

在广域虚拟数据空间之上,运行着许多不同类型的数据密集型应用,这些应用每时每刻都会产生使用数据的需求,并向广域虚拟数据空间发起 IO 任务。如同计算机中不同进程进行调度时可能发生的计算核心争用一样,不同的跨域数据访问请求的广域网络传输之间会产生严重的带宽争用。如图 17 所示,多个 IO 密集型应用同时运行在广域虚拟数据空间之上时,某一应用会占用近乎 90%的带宽资源,而其余应用则分抢剩余的 10%,甚至一些应用一度获取不到带宽资源的分配。造成该现象的两个主要原因有:

一方面,和主流的分布式存储系统一样,广域虚拟数据空间系统采用的是自由竞争的调度方式,这种方式会直接导致一些竞争力小的应用直接陷入饥饿状态。对于单个数据中心内使用局域网连接的分布式存储系统来说,这种方式固然是一种十分高效的方法。

其调度过程几乎不产生时间损耗,并且由于局域网带宽资源十分充足且稳定,每个 IO 任务的执行都十分迅速,对于后续 IO 任务的等待时间影响都很小。但是对于面临高延迟、低带宽广域网的广域虚拟数据空间并不适用,以分钟计时完成的 IO 任务将带来漫长的排队等待过程。

另一方面, Linux 系统中维护了多个不同的数据包队列,而由广域虚拟数据空间客户端统一发起的 IO 任务请求会被分配给同一个数据包队列,应用单个调度算法。Linux 中 Qdisc 默认支持的调度算法为 FIFO 算法,即其发送顺序完全取决于上层应用的调度顺序。这样完全倾斜的带宽资源分配使得在某一时间段内,一些应用完全分配不到带宽资源,这也直接导致该应用发起的 IO 任务吞吐量接近于 0,无论是 QoS 还是系统可用性,对于一个分布式存储系统来说都是不可接受的。

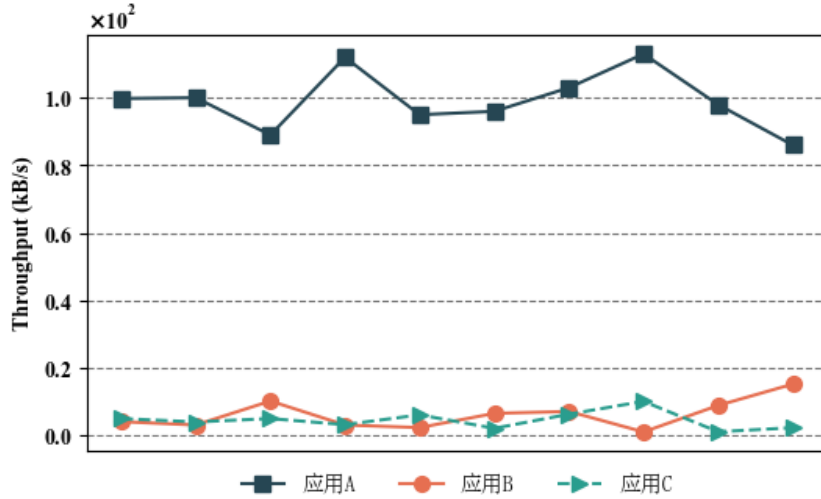


图 17 多应用带宽争用

因此,在跨域数据访问中,对并发运行的多应用进行合理公平的带宽分配,对于一个分布式存储系统来说是至关重要的。目前,学者们已经提出了许多带宽公平分配方法,依据其分配方式可以分为应用隔离和公平调度两大类。

应用隔离是在应用系统外通过使用 Linux 中的 cgroup 和 namespace 两个系统调用对应用进程进行带宽限制,使得每个进程以及其子进程在物理上分配相同的带宽资源。但静态的应用隔离在分配时为了保证后续的新应用公平性会对带宽资源留有剩余,无法完全地利用带宽资源,而动态的应用隔离在新应用到来时对其余应用的资源调整涉及大量的系统调用,额外开销较大。

公平调度通过一些公平的计量策略,调整每个调度因子的运行顺序与执行时间,以实现多应用在系统宏观运行过程中的资源公平分配。这种方法往往由于本身时间复杂度过大或过于追求公平性而忽略了性能问题。Linux 内核中的完全公平调度算法取得了一个较好的平衡,其引入的完全公平思想,借由虚拟运行时间,保证了不同进程之间分配时间片的公平性,同时通过静态权重为高需求任务分配稍多的资源,对于计算资源的公平分配效果显著。

然而不同于普通的计算任务,IO 任务调度时的上下文切换开销较大,且存在网络连接建立时的慢启动阶段,所以调度的频率问题不能和进程的调度方法一概而论。而且不同的应用之间差异显著,其涉及到的数据量规模天差地别,所产生的 IO 任务之间的数量、频率、时间点、目标数据节点等同样存在差异;同一个应用同样存在 IO 阶段和计算阶段,两阶段所产生的 IO 负载截然不同。这样迥异的应用并发产生着跨域数据访问请求,使得广域虚拟数据空间中存在的 IO 负载混杂且难以摸索其规律,对 IO 任务的调度带来了较大挑战。完全公平调度算法静态的权重分配特性无法完全满足跨域数据访问请求的带宽公平分配需求,但是其完全公平的调度思想对于 IO 任务仍具有借鉴的意义。

因此,在多应用并发进行跨域数据访问时,应结合 IO 任务的特点,识别各应用的运行过程中的负载状态,基于完全公平的思想,通过 IO 任务调度动态地为应用公平分配带宽资源,满足应用的运行需求。

本章所研究问题的主要难点与挑战在于:

(1) 如何公平且高效地调度不同应用的 IO 任务。调度的过程中需要为每个应用提供带宽资源公平性的保障,同时也需要根据应用的负载分配合适大小的带宽资源,此外也要保证调度过程本身的耗时不能超过预期。

(2) 如何动态调整以面对应用的不同运行阶段。应用处于 IO 密集态和计算密集态所需要的带宽资源截然不同,需要识别应用的运行阶段,并动态调整对其的带宽资源分配。

针对以上问题,本文研究了一种基于虚拟运行 IO 的多应用跨域数据访问带宽分配机制,综合 IO 任务的特点,基于完全公平的思想,引入虚拟运行 IO 作为带宽分配的决策依据,通过对跨域数据访问请求进行调度运行,实现应用带宽公平分配,最终满足跨域数据访问的公平性需求,并提升带宽资源的利用率。

4.3 详细方案设计与实现

4.3.1 符号描述

在本方法中所涉及到的数据特征及其描述如下表 5 所示：

表 5 带宽公平分配机制数据特征及描述表

符号	定义
aid	应用唯一标识 id
req_id	IO 任务唯一标识 id
vio	虚拟运行 IO
weight	代表应用的权重
queue	代表 IO 任务队列
sz	代表 IO 任务的大小
offset	代表 IO 任务的偏移量
schedule_cycle	代表调度周期
load	代表负载值

4.3.2 整体架构

本研究点的方案整体架构图如图 18 所示。具体来说，本方法由三个组件构成。

首先需要在客户端节点之上建立负载监控机制，采集存储集群目前的带宽、延时、计算存储负载、能耗等情况以及通过日志获取历史 IO 任务信息，构建运行时负载模型，在广域虚拟数据空间系统运行时输出系统实时的运行负载值。

多应用 IO 队列通过队列整形机制，为广域虚拟数据空间中运行的每个应用构建一条 IO 任务队列。在 IO 队列中，根据队列的数据量大小，为每一个应用赋予一个动态变化的权重，以此为依据进行完全公平的带宽分配。

带宽公平分配模块基于负载监控信息与多应用 IO 任务队列，依据完全公平的思想，使用基于虚拟运行 IO 的跨域数据访问请求完全公平调度算法，对每个应用的 IO 任务进行合适、高效的调度，以达到均衡各应用之间带宽分配，并最大化利用带宽资源的目的。本方法并不调整 Linux 系统中对于网卡的默认 IO 调度方法，即 FIFO 调度算法。因此，IO 任务实际执行顺序会和由本方法所调度出来的 IO 任务顺序一致。并且，在系统运行时，应用 IO 队列的权重与整体调度周期均会进行动态变化，以适应不同的运行时状况，保证本方法的实时性与有效性。

本方法以 IO 任务调度的形式对带宽在应用层进行宏观地分配，在一段时间内，每

个应用之间的 IO 吞吐量是完全公平的。

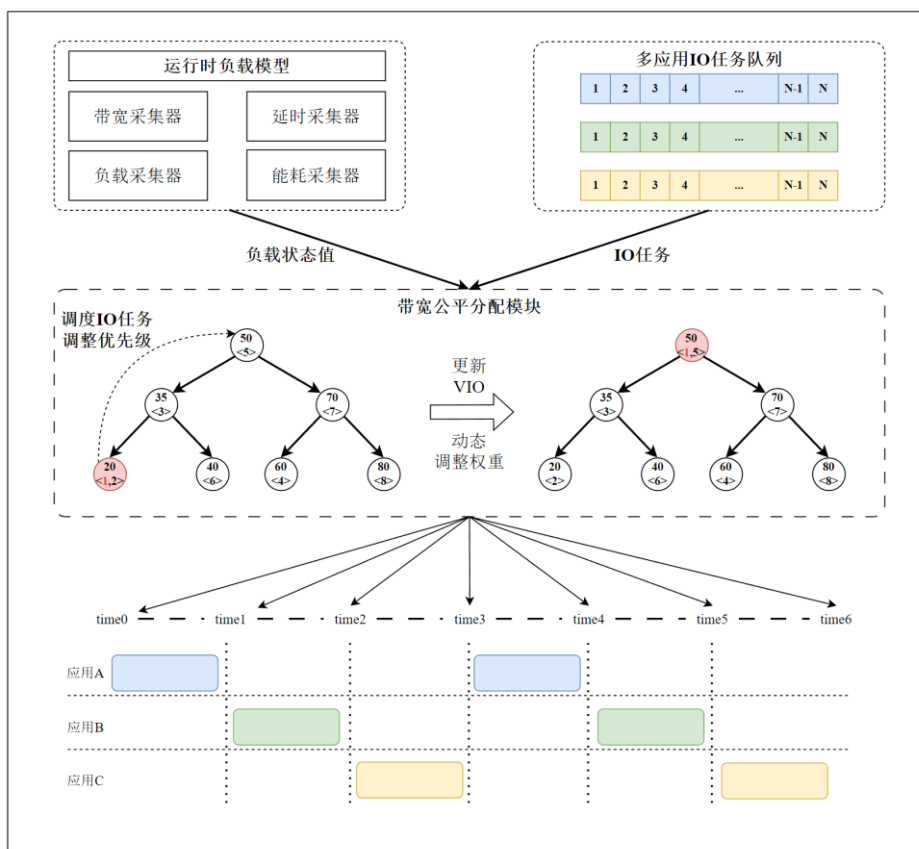


图 18 基于虚拟运行 IO 的多应用跨域数据访问带宽分配机制总体架构

4.3.3 多应用 IO 队列

在 Linux 系统的完全公平调度算法中，每个线程是一个调度单元，尽管完全公平调度算法成功将调度过程的时间复杂度降低到了 $O(\log n)$ ，但面临数以万计的调度单元所产生的调度时延开销对于操作系统来说也是巨大的。一个比较有效的解决方法是对这些调度单元进行分组，在组别之间使用完全公平的调度算法，在组别内使用另外的调度算法或继续使用完全公平算法，这样所产生的时延开销会大幅降低。

本方法中同样采用了这样的思想。在本方法中，IO 任务依据发起该任务的应用进行分组，每个应用在广域虚拟数据空间之中会有一个唯一标识 aid。通过 aid，所属于该应用的 IO 任务均会被分到这一组，无论是该应用所附属的任意进程或线程发起的。不过，这一划分是针对于使用广域虚拟数据空间的上层应用统一访问视图入口在发起数据访问请求时额外携带的属性。广域虚拟数据空间同样对外暴露了符合原生 POSIX 标准的接口，普通用户可以直接使用该接口对系统中的存储数据进行相关操作，从该入口产生的数据访问请求中的所属类别会以该进程的 pid 表示，当然这样所产生的应用 IO 队列

会和上层大型应用的 IO 队列差异显著。

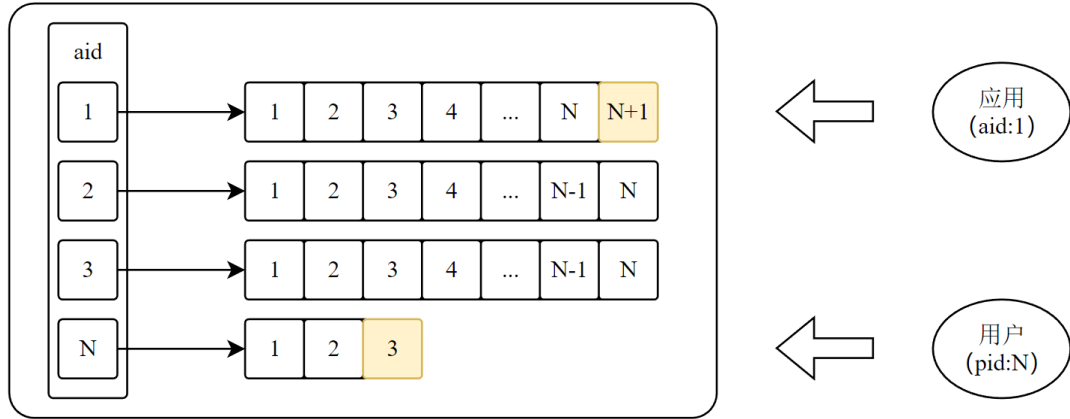


图 19 多应用 IO 队列基本结构

如图 19 所示，算法使用了 aid 与该应用所属 IO 队列构建映射关系。调度时每次先判断应用执行的优先顺序，再从 IO 队列中依据队列内的调度方法选择所需要调度的 IO 任务，以此保证整体系统运行的有序性。由于广域虚拟数据空间中跨域数据访问与域内数据访问之间的吞吐率相差显著，所以本方法将两者在调度队列中拆分，域内数据访问交由原本的简单调度方法进行调度，而跨域数据访问使用本方法提供的调度机制进行调度决策。本方法将对跨域数据访问的调度优化进行效果评估。

在本方法中，依据每个应用的权重，为该应用进行完全公平的带宽分配，所以需要为每个应用赋予一个权重。不同于进程后续运行的不可预测性，IO 任务的大小是一个确定值，并且其执行时间是与大小成正比的，所以根据 IO 队列中 IO 任务的总体大小对该应用赋予相应的权重是合理且可行的。但是直接地根据该项指标对应用进行赋权会产生严重的权重倾斜问题，庞大且 IO 密集的应用权重可能达到低频应用的权重数十倍甚至百倍之多，使用这种差异过大的权重进行带宽划分同样是一种不公平的策略。因此，应当适当缩小应用之间的倍数差异，并为应用赋予最低限度的权重，使得应用的带宽资源得到基本满足，同时让 IO 密集应用适当获得较多资源以提升整体吞吐率。

此外，本方法动态对应用的权重进行调整，以匹配其运行时的负载状态。当应用处于 IO 密集的状态时，理应赋予更高的权重，而计算密集时，应当赋予较低的权重。所以，本方法根据调度过程中，每个 IO 队列里依据 IO 任务的新增与调度对其权重进行动态调整，换句话说，当某个应用产生新的 IO 任务时，根据其 IO 任务大小，对其权重进行适当增长，而当调度某个应用的 IO 任务被调度执行时，同样根据其 IO 任务大小，对

应用权重适当降低，以匹配应用的负载状态。通过动态的权重划分，可以更好地为不同的应用分配适宜的带宽资源，避免静态划分中出现的负载资源不匹配现象。

4.3.4 带宽公平分配模块

带宽公平分配模块基于负载监控信息与多应用 IO 任务队列，采用任务隔离的公平分配方法，对每个应用的跨域 IO 任务进行合适、高效的调度，以达到均衡应用之间带宽分配，并尽可能最大化利用带宽资源。

在带宽公平分配模块中，本方法构建了公平调度机制。本方法引入了虚拟运行 IO 的概念，即一个应用所进行的 IO 任务的虚拟大小。在一个调度周期当中，本方法保证了每个应用的虚拟运行 IO 都是一致的，即保证了应用的完全公平性。虚拟运行 IO 是根据每个应用的实际运行 IO 计算而来的，其增长速度与应用的权重呈反比。这样的增长方法也就意味着，权重越高的应用，其虚拟运行 IO 的增长速度越慢，也就能够获得更多的实际运行 IO，在一个调度周期内，能够分配略多一些的带宽资源。当一个 IO 任务被调度运行时，其实际运行过程中产生的 IO 字节数被视为实际运行 IO 大小，而后根据其权重计算出本次调度所产生的虚拟运行 IO 更新值，累加到调度之前的值之上，作为新的虚拟运行 IO 值，用于后续调度过程的优先级判断。通过这样的完全公平机制，在保证应用一定公平性的同时，尽可能利用带宽资源。

在每次的调度过程中，本方法选择虚拟运行 IO 最小的应用，并从其 IO 队列选择 IO 任务进行调度执行。为了提升调度机制的性能和更好地利用完全公平调度机制，本方法基于红黑树这一基本数据结构来维系不同应用之间的调度顺序关系。红黑树是一棵近乎平衡的二叉搜索树，使得其具有高效的增删改查特性，其查询的时间复杂度为 $O(\log n)$ ，并且相较于完全平衡二叉树，其插入和删除时造成的二叉树旋转次数要少得多，大幅提升了整体运行性能。

如图 20 所示，在多应用 IO 队列之上，使用红黑树结构维护了应用之间的调度顺序关系。红黑树中以键值对的形式进行存储，其中键为应用的虚拟运行 IO，值为应用 aid 的集合，即不同的两个应用可能拥有相同的 vio 值。通过红黑树的排序之后，最左下的元素即虚拟运行 IO 最小的元素值，红黑树的特性使得每次获取最小元素的时间复杂度为 $O(\log n)$ 。为消除这一时间消耗，本方法为这一节点构建了索引，索引始终指向该优先级树中的最小节点，并且在每次调度过程的间隙以异步的形式更新索引的指向，以

此达到每次获取最小节点时 $O(1)$ 的时间复杂度。

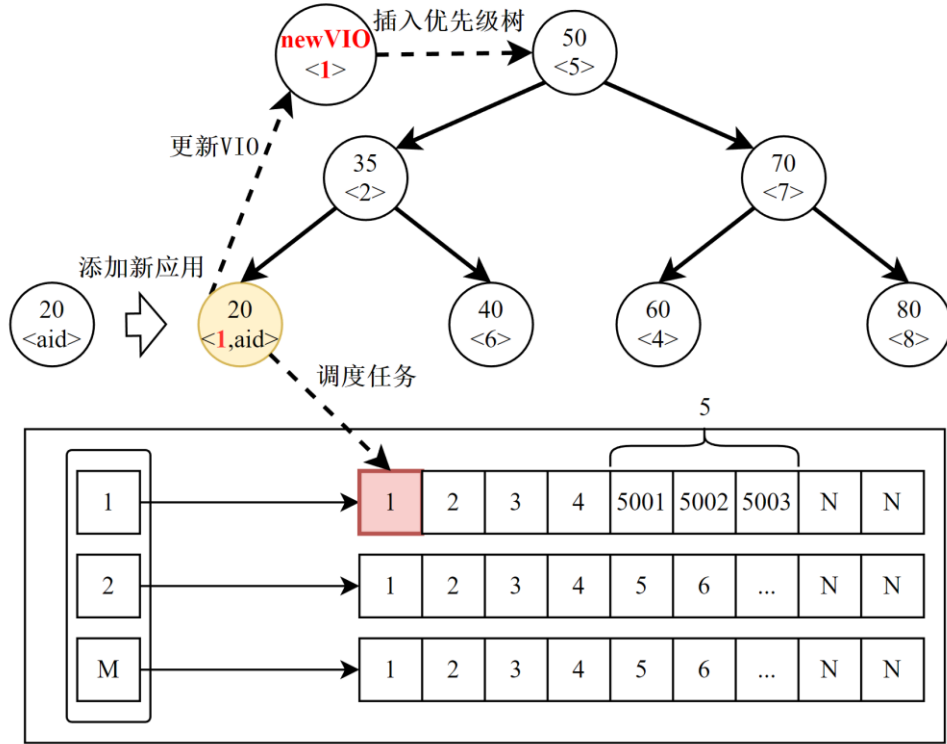


图 20 带宽公平分配模块基本结构

在选择了本次调度的应用之后，需要从其 IO 队列中选择一个 IO 任务进行真正的调度执行。在本方法中，对于队列内的 IO 任务的调度方法采用的是先到先服务，即 IO 任务按照到达时间有序排列在 IO 队列之后，每次从队列的队首获取一个 IO 任务调度执行。先到先服务虽然简单，但是对于应用内的 IO 任务来说有以下优势：一方面，一个应用内所产生的 IO 任务本身由于应用的执行特性就是有序地提交到广域虚拟数据空间客户端之中的，使用先到先服务调度能够直接保证其执行顺序能够符合应用的顺序期望。并且在 IO 任务中分为读任务和写任务，保证正确的读写顺序是保证分布式存储任务中数据一致性的关键手段，同时也可以避免许多不必要的文件读写句柄加锁开销，读写乱序会严重影响数据一致性与系统的可用性。另一方面，简单且高效的先到先服务调度具有最小的时间复杂度，缩短调度时延能够最大化地利用所分配的带宽资源。因此，在本方法中，每个应用的 IO 队列基本构成为双端队列，双端队列可以高效地在队尾添加和在队首删除元素。

本方法的基本目标是在一个调度周期内，所有应用都能够按照其权重获得一个相对公平的带宽分配。所以对于一个在系统运行时新增的应用来说，赋予其一个合适的虚拟

运行 IO 初值是十分重要的。在本方法中，遇到全新的应用时，令其虚拟运行 IO 为当前优先级树中的最小虚拟运行 IO 值，一方面这样可以利用索引直接快速获取最小值，另一方面这样可以使得新的应用能够直接纳入本次调度周期当中，使其能够快速被调度。

为适应系统不同的负载状况，本方法依据负载采集模块的负载情况动态调整调度周期，当分布式文件系统运行时负载状况超过阈值或低于阈值时，调整单个调度周期中调度 IO 任务的总数量，以适当平衡每个应用之间调度的等待时间，提高响应率，并最大化提升整体带宽利用率。由于 IO 任务的慢启动特性，调度周期不宜过于频繁。

4.3.5 基于虚拟运行 IO 的跨域数据访问请求完全公平调度算法

以上 4 小节中从整体架构到具体模块的设计思路，描述了本文中提出的基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制的基本思想。在实际的实现过程中，和大多数的调度算法一致，本方法也分为抢占式与非抢占式两个版本，由于其调度过程的实时性差异，所以在调度周期、调度实际过程的一些具体细节也不尽相同，二者各有优劣势，所以对于实际系统，两种都是可选的调度算法。在本小节中将分别描述本文所提出调度算法的抢占式、非抢占式版本的设计细节与具体执行流程。

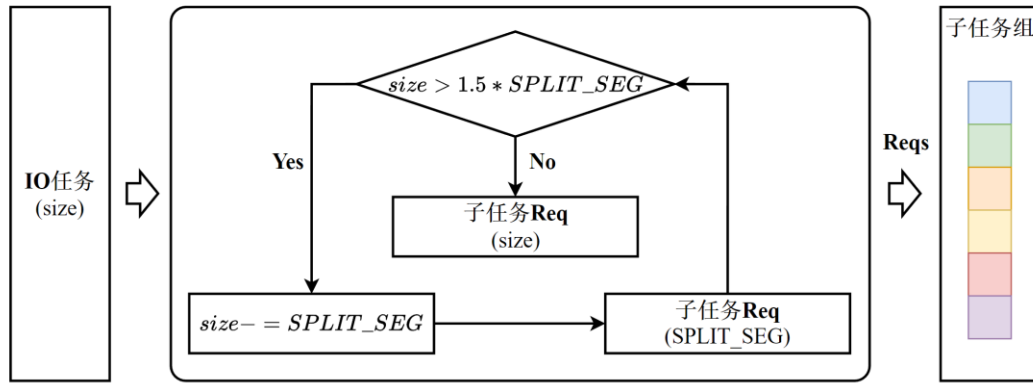


图 21 IO 任务拆分过程

(1) 非抢占式

非抢占式的调度在每次调度之后需要等到前一个调度单元执行完成才会进行下一步的调度策略，因此如果前一个调度单元的执行时间过长则很可能导致后续的调度单元陷入长时间等待状态。而在跨域数据访问的 IO 任务当中，这一点就尤为显著。因为在分布式文件系统当中，不同应用所产生的 IO 任务之间大小差异显著，部分 IO 任务可能在数 MB 左右，而部分 IO 任务可能达到数 GB 甚至 TB 级别。在广域网贫瘠的带宽资

源限制下，其执行时间显得十分漫长。因此对于非抢占式的 IO 任务调度，需要尽可能避免这一现象发生。

表 6 本方法非抢占式调度数据结构表

成员名称	类型	描述
req_map	HashMap<int, Deque<Req>>	应用映射，每一个应用持有一个 IO 任务队列，键为应用唯一 id，值为应用所持队列，队列由双端队列组成，其中包含了 IO 任务的基本信息，例如 size, offset 等；
weight	HashMap<int, int>	权重映射，记录了每个应用的权重，键为应用唯一 id，值为权重，均为整型；
total_weight	int	调度器总权重，为所有应用权重之和，为整形；
prio_tree	RBTree<int, ArrayList<int>>	优先级树，根据每个应用的优先级对应用进行排序，每次调度时选择键最小的应用进行调度，即虚拟运行 IO 最小的应用。键为虚拟运行 IO，值为应用列表，包含了虚拟运行 IO 为当前值的所有应用 id；
req_map	HashMap<int, Deque<Req>>	应用映射，每一个应用持有一个 IO 任务队列，键为应用唯一 id，值为应用所持队列，队列由双端队列组成，其中包含了 IO 任务的基本信息，例如 size, offset 等；

表 6 给出了本方法非抢占式调度使用的数据结构，主要包括多应用 IO 队列 req_map，多应用权重 weight，优先级树 prio_tree 以及调度其中的总权重 total_weight。在本方法中，使用任务拆分的队列整形方法对 IO 任务进行预先处理，将其拆分成大小近似的多个子任务，再添加到 IO 队列当中。如图 21 所示，在拆分过程中，根据系统设定的分割阈值对 IO 任务进行分割，如果剩余 IO 任务大小超出阈值的 1.5 倍，则拆分出一个大小和阈值等同的子任务，插入到 IO 队列的末尾，直至最后一个不大于阈值 1.5 倍的任务部分，将剩余部分直接添加到 IO 队列当中。由于 IO 任务的特殊性，每次 IO 任务都会有一个任务启动的过程，这一阶段具有耗时大、吞吐小的特点，所以需要设置适宜的分割阈值以避免出现过大大或小 IO 任务的问题，导致多应用间的公平性降低。

通过 IO 队列整形，每个应用 IO 队列中的 IO 任务大小都相当，每个应用中的任务数量能够很大程度反映出该应用的 IO 负载规模，所以在非抢占式版的 IO 任务完全公平调度算法中，会以此对应用进行赋权。如 4.3.3 小节所述，直接根据规模对应用赋权会导致应用之间产生指数级权重差异，应适当缩小权重倍数差异，才能保证基本公平性。

所以本方法根据应用中任务数量的指数级别对应用进行赋权，如公式 4.1 所示。例如一个应用有 10 个任务，则该应用的权重为 1，若该应用有 100 个任务，则该应用的权重为 2，以此类推。并且，该权重会在调度运行中进行调整，若由于某一次调度或新增任务导致应用队列的总数上升或下降一个数量级，其权重也会上升或下降 1，以此来动态进行任务调度。

$$Weight_{aid} = \log_{10} queue_{aid}.len() \quad (4.1)$$

算法 3 非抢占式跨域 IO 任务公平调度算法——添加 IO 任务

Input:

Req: 新增 IO 任务信息，其中包含字段应用 *aid*，任务大小 *size*，偏移量 *offset*

scheduler: 调度器

SPLITSEG: 划分阈值

```

1  sz ← Req.size
2  offset ← Req.offset
3  while sz > 0 do
4      if sz > SPLITSEG * 1.5 do
5          req ← new Req(aid, SPLITSEG, offset)
6          scheduler.req_map[Req.aid].push_back(req)
7          sz ← sz - SPLITSEG
8          offset ← offset + SPLITSEG
9      else do
10         req ← new Req(aid, sz, offset)
11         scheduler.req_map[Req.aid].push_back(req)
12     end if
13 end while
14 deltaWeight ← calculateWeight()
15 scheduler.weight[aid] ← scheduler.weight[aid] + deltaWeight
16 scheduler.total_weight ← scheduler.total_weight + deltaWeight
17 if !scheduler.req_map.contains(Req.aid) do
18     scheduler.prio_tree.front().push_back(Req.aid)
19 end if
```

算法 3 显示了添加 IO 任务的算法过程。首先是对所添加的 IO 任务进行拆分，其拆分过程如上文所述。接着 calculateWeight 即根据所添加的任务数量对应用权重进行更新，如果新添加的任务导致应用的任务数量跨越了一个新的数量级，则应用的权重加 1，并需要同步更新到调度器中的总权重。最后，如果新增的 IO 任务为一个新的应用，则使用优先级树中最小的键作为该应用目前的键，并将其插入到优先级树当中，以便该应用能够快速投入调度流程中。

算法 4 非抢占式跨域 IO 任务公平调度算法——调度 IO 任务

Input:*scheduler*: 调度器**Output:***Req*: 本次调度选择的 IO 任务

```

1  vIO, aidList  $\leftarrow$  scheduler.prio_tree.poll_front()
2  aid  $\leftarrow$  aidList.poll_front()
3  scheduler.prio_tree.insert(vIO, aidList)
4  Req  $\leftarrow$  scheduler.req_map[aid].poll_front()
5  deltaVIO  $\leftarrow$  (Req.size * Req.total_weight) / (SPLITSEG * scheduler.weight[aid])
6  newVIO  $\leftarrow$  vIO + deltaVIO
7  if scheduler.prio_tree.contains(newVIO) do
8      scheduler.prio_tree[newVIO].push_back(aid)
9  else do
10     scheduler.prio_tree.insert(newVIO, new array(aid))
11 end if
12 deltaWeight  $\leftarrow$  calculateWeight()
13 scheduler.total_weight  $\leftarrow$  scheduler.total_weight - deltaWeight
14 scheduler.weight[aid]  $\leftarrow$  scheduler.weight[aid] - deltaWeight
15 return Req

```

算法 4 给出了调度 IO 任务的算法过程。首先，调度时每次从优先级树中获取键值最小的应用进行调度，每个应用持有自己的调度队列。接着，从该应用的 IO 任务队列中以先到先服务的策略获取一个待调度 IO 任务，由于 IO 任务队列采用的是双端队列进行实现，所以直接 *poll_front* 即可。之后，重新计算该应用的虚拟运行 IO，根据本次调度所选 IO 任务的大小，以及该应用的当前权重与调度器的总权重，计算本次调度会产生虚拟运行 IO 变化值，将该变化值加上先前值即作为新的虚拟运行 IO，并插入到优先级树中。最后，由于是非抢占式调度，本次所调度的 IO 任务会一直执行直到结束，因此在此处即可直接计算权重变化值。依据本次调度是否会对应用 IO 任务数量的数量级造成影响来判断是否更新应用权重与总权重，若应用的任务数量降低到了下一个数量级，则应用的权重减 1。随后将所选 IO 任务返回，交予执行器进行实际 IO 操作，至此完成了本次的调度。

通过队列整形，本方法将应用的规模转化为了应用 IO 任务队列的任务数量，并以此为依据为应用赋权。为契合其特性，本方法以任务数量作为非抢占式调度中调度周期的表现形式，即定义每一个调度周期中所要调度的任务总数。理想中的调度周期为一个周期内满足所有应用的需要，即调度周期与调度器所有应用权重的总和，如公式 4.2 所

示。但当系统运行时负载出现过低或过高的现象时，适当地调整调度周期，能够缓解系统运行状况，更好地发挥系统性能。所以在本方法中，依据负载监控模块输出的系统当前负载状况，若负载状况超过某个阈值或低于某个阈值时，调整调度周期，即调整每个周期内调度 IO 任务的数量，以适当平衡每个应用之间调度的等待时间，提高响应率，并最大化提升整体带宽利用率。

$$schedule_{cycle} = \begin{cases} \frac{CYCLE}{2}, & load \leq 0.2 \\ CYCLE, & 0.2 < load \leq 0.7 \\ CYCLE * 2, & 0.7 < load \end{cases} \quad (4.2)$$

综上所述，本方法的非抢占式调度方法能够通过任务拆分队列整形从一定程度上降低非抢占式所带来的后续任务等待问题，并且通过完全公平的策略保障了应用之间的公平性，但是其实时性仍旧有所欠缺，任务拆分所带来的额外任务上下文切换开销也是不可忽视的损耗。

(2) 抢占式

在抢占式的调度中，每次执行调度时，无论之前的调度单元是否已经完成，都会即时地抢占其带宽资源，若之前的任务还未完成，则将其停止并退回调度队列当中。这一过程，本身就是对 IO 任务的一次执行拆分，因此，在本方法的抢占式调度中，不需要使用拆分的手段将 IO 队列进行整形，单一且大规模的 IO 任务能够最大程度发挥抢占式的优势。

与之相对的，抢占式也失去了依据任务划分所形成的天然权重表示形式。在抢占式中，本方法使用应用队列中 IO 任务的总大小的函数值作为应用的权重，其映射形式如公式 4.3 所示。如此得到的应用权重同样和应用规模成正比例关系，并且使用指数级削弱了应用之间权重的倍数差异。

$$Weight_{aid} = \log_{10} \frac{\sum queue[i].sz}{SPLITSEG} \quad (4.3)$$

表 7 给出了本方法抢占式调度使用的数据结构描述。与非抢占式的相比主要新增了两个数据成员，一是指向先前调度 IO 任务的指针，另一个是该 IO 任务上一次调度时的大小。指针的作用是记录先前调度的状态，并作为索引指向先前 IO 任务，便于后续的加速访问，而 `pre_size` 的作用是用于记录先前 IO 任务的执行状态，便于计算两次调度之间所发生的 IO 操作大小，以此计算虚拟运行 IO 的大小。

表 7 本方法抢占式调度数据结构表

成员名称	类型	描述
req_map	HashMap<int, Deque<Req>>	应用映射，每一个应用持有一个 IO 任务队列，键为应用唯一 id，值为应用所持队列，队列由双端队列组成，其中包含了 IO 任务的基本信息，例如 size, offset 等；
weight	HashMap<int, int>	权重映射，记录了每个应用的权重，键为应用唯一 id，值为权重，均为整型；
total_weight	int	调度器总权重，为所有应用权重之和，为整形；
prio_tree	RBTree<int, ArrayList<int>>	优先级树，根据每个应用的优先级对应用进行排序，每次调度时选择键最小的应用进行调度，即虚拟运行 IO 最小的应用。键为虚拟运行 IO，值为应用列表，包含了虚拟运行 IO 为当前值的所有应用 id；
pre	*Req	IO 任务的指针，指向调度器上一次调度的 IO 任务，为空则表示还未调度任务；
pre_size	int	调度器上一次调度的 IO 任务开始时的大小；

算法 5 抢占式跨域 IO 任务公平调度算法——添加 IO 任务

Input:

Req: 新增 IO 任务信息，其中包含字段应用 aid，任务大小 size

scheduler: 调度器

SPLITSEG: 划分阈值

- 1 $scheduler.total_size \leftarrow scheduler.total_size + size$
 - 2 $scheduler.total_weight \leftarrow \log_{10}(scheduler.total_size / SPLITSEG)$
 - 3 $scheduler.req_map[aid].push_back(aid)$
 - 4 $scheduler.req_map[aid].total_size += size$
 - 5 $scheduler.weight[aid] \leftarrow \log_{10}(scheduler.req_map[aid].total_size / SPLITSEG)$
-

算法 5 给出了本方法抢占式调度方法的添加 IO 任务算法过程。在抢占式调度中，省去了任务划分的步骤，其余与非抢占式基本一致。不同之处在于，本次添加 IO 任务所产生的应用权重变化适用于公式 4.3，直接将当前 IO 任务的大小增加到调度器总大小之上，而后以 IO 任务大小的指数级别判定是否需要更新应用的权重与调度器的总权重。由此得到的动态均衡权重能够直接反应应用的当前访问负载，便于进行合适的带宽资源分配。并且，省略了任务划分的步骤之后，抢占式调度算法添加 IO 任务步骤的时间复杂度要小于非抢占式，任务在调度时的权重变化也更加细粒度，使得资源分配更为准确，这是抢占式调度的优势之一。

算法 6 给出了本方法抢占式调度方法的调度 IO 任务算法过程。在抢占式中，需要对先前调度任务的运行状态进行维护，并更新整体的调度策略，因此，首先判断先前调度的任务是否已经完成。若为已完成状态，则将该任务的大小作为上一次调度所产生的实际运行 IO，将其与应用的权重与调度器总权重共同计算出这一次调度所产生的虚拟运行 IO 变化值，并更新到优先级树中，同时将该任务从 IO 任务队列中移除；若为未完成状态，则根据该任务当前的大小，与调度器中所记录的先前值计算差值，作为上一次调度所产生的实际运行 IO，而虚拟运行 IO 的计算方式和上述一致。随后，根据实际运行 IO 对应用的权重完成更新。与非抢占式不同的是，在抢占式中由于一次调度可能无法完成整个任务，所以不直接将该任务从调度队列中移出，而是以索引的形式将该任务与其当前大小记录在调度器的运行上下文中，以便访问。至此完成了本次的调度执行。

算法 6 抢占式跨域 IO 任务公平调度算法——调度 IO 任务

Input:

scheduler: 调度器

Output:

Req: 本次调度选择的 IO 任务

```

1  if pre != null do
2      if *pre.state == RUNNING | PENDING do
3          *pre.abort()
4          deltaSize  $\leftarrow$  pre_size - *pre.size
5      else do
6          deltaSize  $\leftarrow$  pre_size
7          scheduler.req_map[*pre.aid].pop_front()
8      end if
9      deltaVIO  $\leftarrow$  deltaSize * scheduler.total_weight / scheduler.weight[*pre.aid]
10     newVIO  $\leftarrow$  deltaVIO + *pre.vio
11     if scheduler.prio_tree.contains(newVIO) do
12         scheduler.prio_tree[newVIO].push_back(*pre.aid)
13     else do
14         scheduler.prio_tree.insert(newVIO, new array(*pre.aid))
15     end if
16 end if
17 vIO, aidList  $\leftarrow$  scheduler.prio_tree.poll_front()
18 aid  $\leftarrow$  aidList.poll_front()
19 scheduler.prio_tree.insert(vIO, aidList)
20 Req  $\leftarrow$  scheduler.req_map[aid].front()
21 pre, pre_size  $\leftarrow$  &Req, Req.size
22 return Req

```

由于实时追踪 IO 任务的执行大小所产生的资源损耗巨大，所以在抢占式中，和主流的调度算法一样，以时间片为调度周期的表现形式。对于每个 IO 任务，本方法每次调度时给其分配相同的时间片，IO 任务在给定的时间片中执行实际的 IO 请求，并记录已完成的 IO 大小。由于 IO 任务的特殊性，每次 IO 任务都会有一个任务启动的过程，这一阶段具有耗时大、吞吐小的特点，因此给每个 IO 任务分配的时间片不宜过小，降低启动阶段在整体时间阶段的占比，能够提升整体吞吐率。当然，在抢占式调度中同样需要依据系统运行时负载动态调整调度周期，其调整的形式即增加或缩短时间片的大小，同样的，调度周期会有一个最小值。

综上所述，抢占式调度具有实时性高的特点，同时不需要进行任务拆分，减少了额外的网络连接建立开销，但是其调度周期的设定对 IO 任务的运行性能影响巨大，频繁的调度决策也会带来许多额外的系统运行开销。

4.4 实验与验证

4.4.1 实验环境

表 8 带宽公平分配机制实验平台软硬件环境配置表

软硬件	详细描述
CPU	Intel(R) Xeon(R) Silver 4110
内存	64GB
磁盘	AVAGO MR9361-8i 4TB
网络带宽	1000M (测速约 124MB/s)
操作系统	Ubuntu 20.04.4 LTS
操作系统内核	5.4.0
Rust	1.73.0-nightly

在本小节中将对本文所提出基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制进行实验验证。在本实验中，采用 3.4.1 小节中相同的实验拓扑图构建的模拟环境中进行，以方便自定义调整网络环境。实验所用实验平台软硬件环境配置如表 8 所示。

4.4.2 带宽公平分配机制有效性测试

为了测试本文所提出带宽公平分配机制的有效性，本文实验并评估其在不同并发负载应用场景下各应用的带宽资源分配情况。本方法的目标是让各个应用能够公平地分配满足应用需求的带宽资源，并且为了满足高负载应用的运行需求，应适当为其分配稍多

的带宽资源。因此，本文选取了以下两个应用场景进行实验：一是多个应用之间拥有完全相同访问负载的均衡场景，即各应用的 IO 请求大小、数量均一致，以测试该场景下算法是否为各个应用分配相同的带宽资源；二是多个应用之间拥有较大差异访问负载的倾斜场景，即某一应用负载极高，而其余应用拥有一致且较小的访问负载，以测试算法是否为高负载应用分配稍多的带宽资源，并为其他应用保持公平分配。

表 9 带宽公平分配机制实验参数表

任务划分阈值 (SPLITSEG)	本方法-抢占式调度周期	CFS 调度周期
800MB	500ms	50ms

为了对比算法的运行效果，本文同时对本方法的抢占式和非抢占式两个版本进行实验评估，并选取了 IO 任务的常用调度算法作为对照组，包括 CFS 和 GVDS-baseline。其中的实验参数设置如表 9 所示。

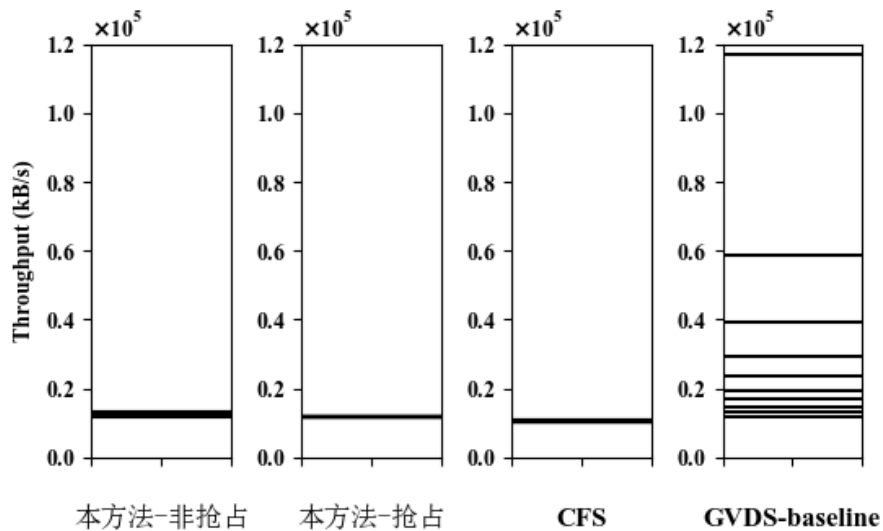


图 22 10 个应用均衡场景下吞吐量分布

图 22 给出了 10 个应用在均衡场景下各应用的吞吐量分布情况。从中可以看出，在本方法和 CFS 的作用之下，各应用所分配的带宽资源几乎一致，保证了良好的公平性，尤其是抢占式的方法能够更接近于 CFS 的公平性表现，这是因为频繁的调度过程能够使得调度周期窗口变小，在整个应用运行的宏观过程当中能够有更加均衡的表现。而 GVDS-baseline 则是完全忽视了公平性问题，产生了极大差异化的带宽分配。

图 23 给出了 10 个应用在倾斜场景下各应用的吞吐量分布情况，其中一个应用拥有其余应用访问请求之和的负载，图中红色虚线为该高负载应用的吞吐量。由于本方法

对调度单元的 IO 大小所采取的完全公平调度策略，本方法的两个版本均能为高负载应用分配稍多的带宽资源，并且其余相同负载的应用能够公平地分配相同的带宽资源。而 CFS 是对任务的执行时间进行静态权重的完全公平调度，所以其仍旧表现出绝对公平的带宽分配。同样的，GVDS-baseline 呈现出杂乱的带宽分配表现。

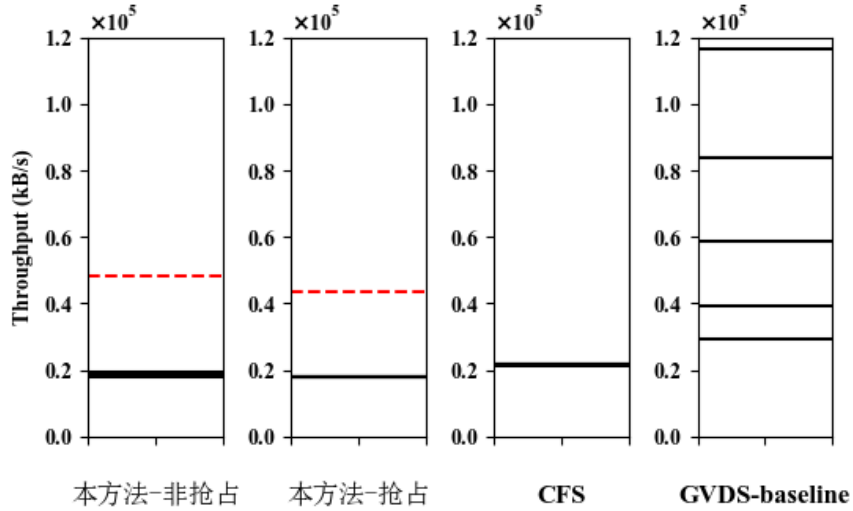


图 23 10 个应用倾斜场景下吞吐量分布

综上所述，本方法能够在一定程度上保证应用带宽资源分配的公平性，并适当调整分配策略以满足高负载应用的运行需求，做出合适、准确的调度决策。

4.4.3 带宽公平分配机制性能测试

为了测试本方法的性能，本文将 CFS 和 GVDS-baseline 作为对照，从多角度场景进行了对比实验，选取了应用执行过程中的吞吐量的均方根误差、总吞吐量以及调度时延作为测试指标。均方根误差是反映带宽资源分配公平程度的重要量化指标，均方根误差越低，意味着为应用提供了更高的公平性保障；总吞吐量为广域虚拟数据空间中应用启动到所有应用完成时全过程的整体吞吐量，一定程度上能反映调度策略对于带宽资源的利用率；调度延迟为调度算法在执行过程中的平均耗时，代表了调度过程的开销大小。

图 24 给出了不同应用数量下，本方法与 CFS 和 GVDS-baseline 在各应用均为单次 10GB 文件读负载特征的总吞吐量与均方根误差实验数据。实验结果表明，本方法无论是抢占式还是非抢占式都有与 CFS 几乎一致的公平性表现，并且本方法的抢占式算法具有比 CFS 更低的均方根误差，而非抢占式则稍高一些，这是因为在抢占式中，较高的调度频率能够在牺牲一定吞吐量的情况下均衡调度周期内的带宽资源分配差异。这一点

在吞吐量表现中也可能看出，本方法能够达到近乎 baseline 的性能，并比 CFS 有更好的性能表现。在 50 个应用的场景下，本方法的抢占式与非抢占式比 CFS 性能分别提升约 7% 和 8%。这是因为 CFS 是根据任务执行时间来计算各个调度因子的虚拟运行时间，以此来进行完全公平的调度，因此需要保持较高的调度频率来维持系统整体公平性，而由于 IO 任务存在性能较低的启动期，所以过快的调度过程会使得启动期占比增大，从而影响执行性能，故而 CFS 总体吞吐量较低。而本方法采取任务的 IO 大小与动态权重的完全公平调度，可以在采取更长调度周期的同时也能保证系统的公平性。

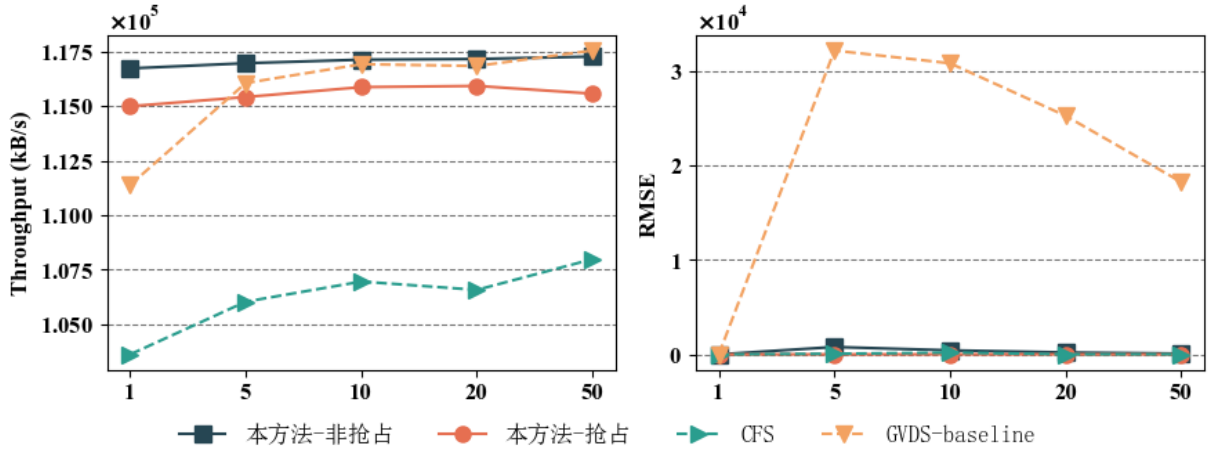


图 24 不同应用同访问负载性能测试结果

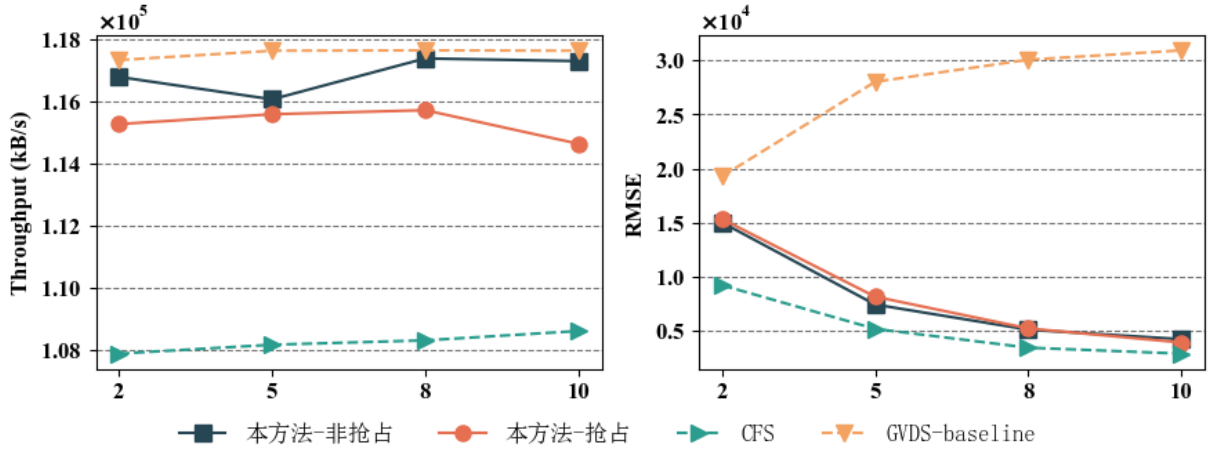


图 25 不同应用单请求递增访问负载性能测试结果

接着，本文测试了不同应用数量下，本方法与其他方法在应对多应用差异访问负载时的性能表现。图 25 给出了多应用单访问请求递增访问大小场景下的总吞吐量与均方根误差实验结果，而图 26 给出了多应用同访问大小递增任务数量场景下的总吞吐量与均方根误差实验结果。实验结果表明，随着任务数量或任务大小的增大，即访问负载的

增加, GVDS-baseline 的不公平性愈发显著, 而本方法与 CFS 能够表现出更好的公平性, 而本方法的均方根误差要比 CFS 稍高一些, 这是因为本方法会为高负载应用分配稍多一些的带宽资源, 均方根误差高是正常现象。并且, 和均衡负载时的表现一致, 本方法能取得接近于 baseline 的总吞吐量。在两种场景下, 相对于 CFS, 本方法抢占式分别能提升约 5.6%、7.1% 的性能, 而非抢占式能提升约 8.0%、8.4% 的性能。

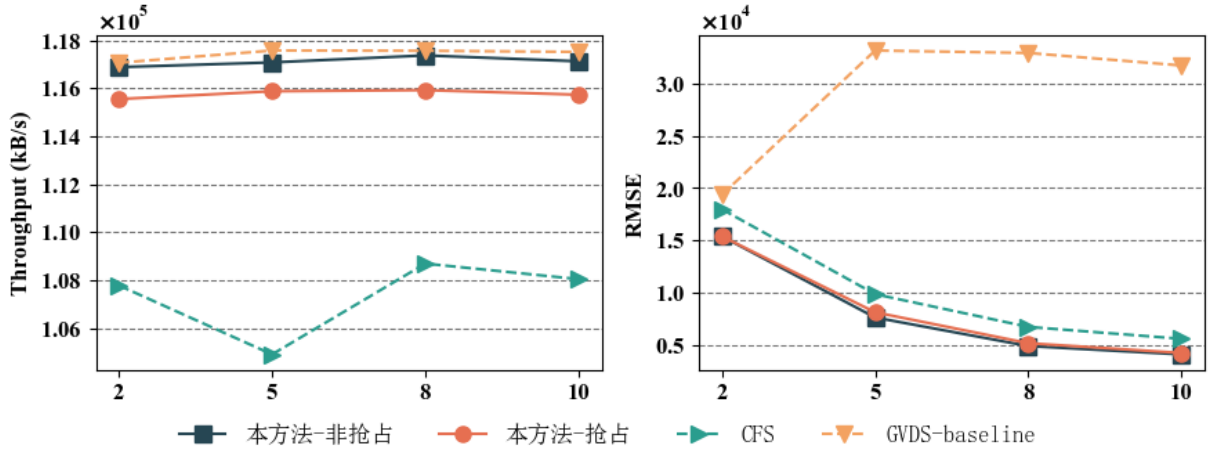


图 26 不同应用同访问大小递增任务数量性能测试结果

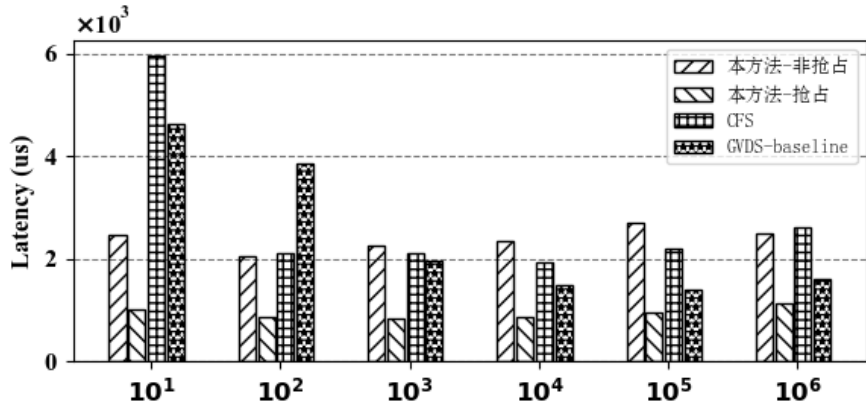


图 27 不同应用数量调度延迟

此外, 本文测试了本方法和其它方法的调度时延表现。图 27 给出了各方法在不同应用数量时的平均调度时延。实验结果表明, 随着应用数量的提升, 各方法的调度时延均趋于平稳, 本方法的抢占式算法的调度时延相比 CFS 降低了 57%, 而非抢占式与 CFS 的差异不大。从图中也可以看出非抢占式的算法拥有较高的调度时延, 这是因为非抢占式算法调度过程中对调度因子的读写锁争用导致的线程等待, 从而产生了多余的时间开销。总的来说, 这四种算法所带来的调度过程开销, 对于高耗时的 IO 任务来说影响微

乎其微。

4.5 本章小结

针对广域虚拟数据空间的跨域数据访问通路中面临的公平性问题，本章节设计并实现了基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制，基于完全公平的思想，综合多应用 IO 任务队列，为高并发的多应用 IO 任务提供合适、准确的调度决策，在保障多应用间带宽分配公平性的同时，提升了带宽资源的利用率。主要的工作如下：

（1）设计并实现了基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制。首先，构建多应用 IO 队列，通过 IO 任务拆分，均衡跨域数据访问请求大小，并依据应用标识符对 IO 任务进行分组，赋予动态均衡权重；其次，引入虚拟运行 IO 作为带宽分配的决策依据，通过计算实际运行 IO 与应用权重的比值，更新虚拟运行 IO；最后，基于虚拟运行 IO 进行非抢占式与抢占式的跨域数据访问请求调度，实现跨域数据访问带宽公平分配。本方法在保障多应用间带宽分配公平性的同时，提升了带宽资源的利用率。

（2）构建了实验系统。在仿真模拟的并发环境中实现了本章节设计的 IO 任务调度算法，并通过实验对比了本方法和其余方法的性能表现和公平性表现。实验结果表明，本方法能够在一定程度上均衡各应用之间的带宽分配，做出合适、准确的调度决策。同时对比实验表明，本方法的非抢占式与抢占式调度算法拥有比 **baseline** 调度算法显著的公平性提升，同时比完全公平调度算法带来了 8%左右的总吞吐量提升。

第五章 基于跨域应用特征感知的自适应数据预取方法

5.1 引言

广域虚拟数据空间构建在跨地域分散的多数据中心之上,对其异构存储资源进行统一管理 & 共享访问。作为基于网络的分布式存储系统,广域虚拟数据空间将客户端与服务端解耦合,并通过网络进行互相通信。为加速数据访问,一些基于网络的存储系统通过构建高效的缓存系统来在一定程度上屏蔽网络的带宽限制。广域网呈现出带宽资源受限、延时大、带宽波动大的特点,广域虚拟数据空间在近客户端构建了边缘缓存系统,旨在为跨域数据访问提供加速效果。

由于缓存的命中率对于缓存系统的运行性能至关重要,许多学者们致力于探索更准确的缓存换入换出策略。数据预取作为文件系统中常用的优化策略,对于提升缓存系统的命中率有显著作用。现有存储系统中的预取策略通常只针对于特定的工作负载,而面对其他负载时没有增益或甚至产生负面影响,无法满足多个工作负载的各种预取要求。

针对目前预取策略实时性不足导致跨域数据访问效率下降的问题,本文提出了一种基于跨域应用特征感知的自适应数据预取方法。该方法结合应用的不同访问负载特征,通过自适应调优的手段选取准确、合适的数据预取策略,以提升客户端数据缓存的命中率,提升跨域数据访问性能。在本章节中将从问题分析与研究思路、详细方案设计、实验与验证三方面详细描述这一方法的技术细节。

5.2 问题分析与研究思路

广域虚拟数据空间由一个元数据服务器、多个 IO 数据节点和多个客户端组成,通过客户端,可以与 IO 数据节点建立通信隧道,并开始数据传输。作为一个跨域的分布式文件系统,其跨域数据访问请求在总请求中占了很大一部分,但贫瘠的带宽资源和不稳定的弱网环境使得跨域数据访问的吞吐率相较于域内数据访问有巨大性能差异。

为在一定程度屏蔽时延影响,广域虚拟数据空间构建了边缘缓存系统,该系统利用近客户端节点的空余存储资源为客户端提供访问加速能力^[65]。其接收所有的最近 IO 访问请求作为缓存换入内容,并以改良的 LRU 缓存换出机制对外提供缓存能力。这种策略对于简单重复的 IO 访问负载能起到良好的效果,但对于复杂的应用来说其缓存命中

率大幅下降，并且其本身所占用的额外系统资源也是一个不可忽视的开销。例如数据密集型应用中常见的批处理计算模式，这一种模式通常对于单个文件只会访问一次，而后通过计算产生新的文件，再依据新的文件来做后续的计算，采用上述的缓存策略会导致大量的无效缓存，浪费系统资源。因此，应当构建恰当的缓存机制以匹配应用的访问负载模式，才能有效利用缓存系统的资源，提升缓存命中率，并提升整体运行性能。

客户端数据预取常用于广域网存储系统，因为它可以有效地隐藏网络延迟。在跨域数据访问中，数据预取即在满足正常的 IO 访问请求的基础上，根据对 IO 路径的预测结果预先将后续可能访问到的文件数据块伴随正常的请求结果一同返回，并将其缓存起来。如此，后续有对该数据块的访问请求时就可以直接从内存或近端节点直接获取，而避免了高延时低吞吐的广域网链路。

目前，学者们已经提出了许多数据预取方法，根据其访问规律，可以分为顺序负载和关联负载两大类，其中关联负载包括语义模式指导的预取策略、基于文件块关联关系指导的预取策略以及基于性能跟踪数据指导的预取策略。大多数现有的预取方法通常只为应用程序提供针对单一负载的预取策略，而不考虑访问负载的动态变化，这种一刀切式的解决方案只能在其针对的负载场景中发挥性能优势，而无法满足其它各式的应用需求，甚至可能会由于额外的数据传输而导致系统整体的吞吐率下降，造成负优化。现实中的分布式存储系统中往往支撑多种应用类型，在应用运行前凭经验人为手动地配置预取策略也给系统管理人员带来了巨大的挑战。

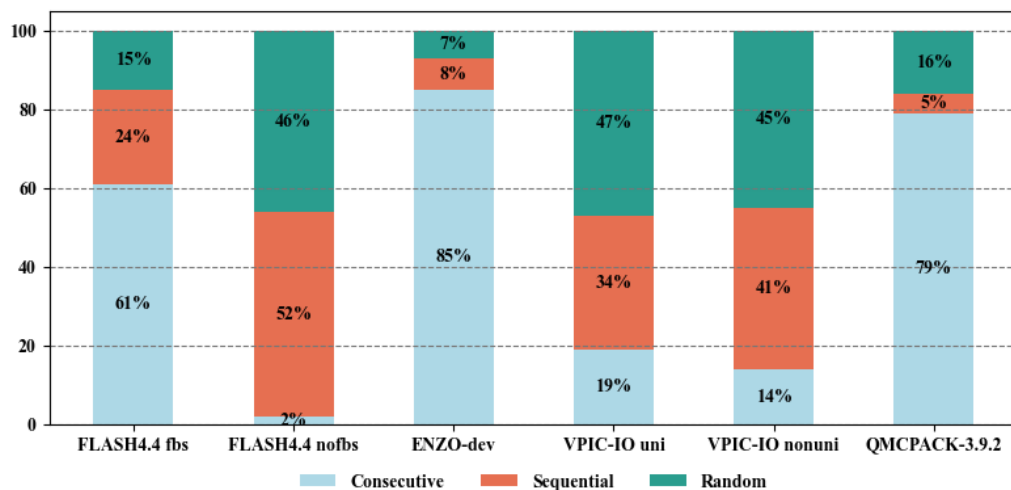


图 28 典型高性能计算应用文件访问负载占比图

并且，不同应用之间的负载特征差异是显著的，根据 Recorder^[66]中采集的高性能计

算应用的 trace 数据，图 28 展示了典型高性能计算应用的文件访问工作负载百分比，其中工作负载分为三大类，consecutive（即顺序读，在物理上连续），sequential（即关联读，符合一定规律的连续），random（即纯随机读）。从图中可以看出，以某一类负载为主要占比的应用为少数，大部分的应用为多种负载融合的访问模式，以单一负载为目标的数据预取策略在此类应用中往往收效甚微。因此，在应用运行过程中动态地选择恰当的数据预取策略，才能最大限度地发挥多种预取策略各自的优势，提升存储系统整体的高效性。

本章所研究问题的主要难点与挑战在于：

（1）如何准确地动态感知系统中应用的跨域数据访问特征，并自适应地调整预取策略的选择，以提升缓存的命中率。不同的预取策略能够满足其所针对的访问负载模式，而对其他模式没有效果甚至产生负面效果。因此，为应用的动态特征采取自适应的预取策略可以有效发挥各自的性能优势，提高预取的准确性。

（2）如何无额外开销地在系统中引入多种预取策略，以免系统正常数据通路受到干扰。简单地同时添加多种预取策略会产生翻倍的额外数据传输开销，并且可能污染有限的数据缓存空间。因此，需要构建相应机制，使得每时刻有且仅有一种预取策略发挥实际作用。

针对以上问题，本文研究了一种基于跨域应用特征感知的自适应数据预取方法，该方法使用快速增量数据预取方法和关联数据预取方法分别为顺序读和关联读场景提供数据预取决策，并引入模拟预取的机制与自调优方法，为应用提供无额外开销且自适应的预取策略，提高客户端数据缓存的命中率，最终提升系统跨域数据访问的性能，满足跨域数据访问的性能需求。

5.3 详细方案设计与实现

5.3.1 符号描述

表 10 数据预取策略数据特征及描述表

符号	定义
block	文件数据块
window	滑动窗口
winsize	滑动窗口大小

本章方法所涉及数据特征及其描述如表 10 所示。

5.3.2 总体架构

本方法的总体架构如图 29 所示。具体来说，本方法总共分为四大部分。

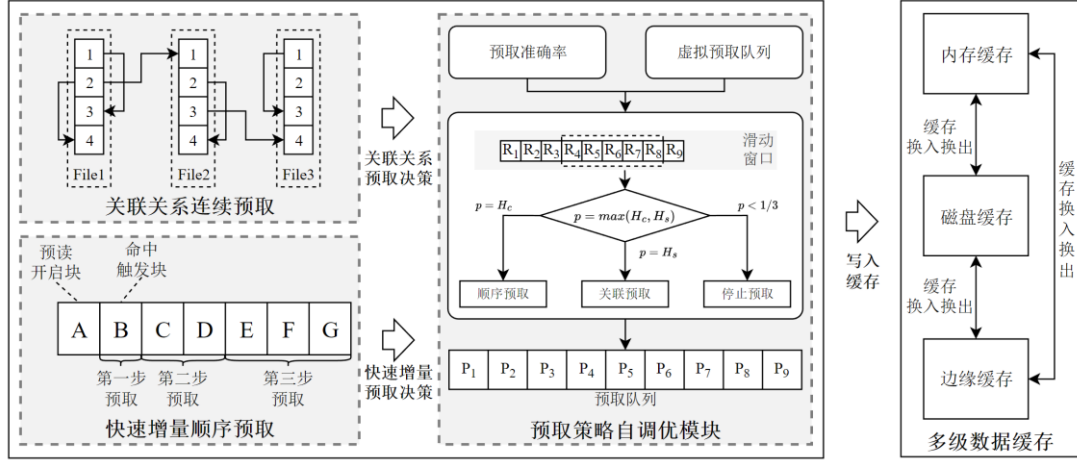


图 29 基于跨域应用特征感知的数据预取方法总体架构

首先，基于广域虚拟数据空间中的边缘缓存系统，构建客户端侧的多级数据缓存。通过客户端内存、客户端节点磁盘、边缘缓存三级的缓存结构，使用 LRU 缓存换出策略将跨域数据访问的数据与预取数据进行缓存，以加速后续访问请求。

其次，针对顺序读的工作负载，构造快速增量顺序预取方法。该方法将每次所访问文件数据块物理上连续的其余数据块作为预取结果返回，并且依据预取数据的命中情况进行增量预取，即在判定所预取数据命中后自适应调整预取的步长，控制预取数据量。

随后，针对关联读的工作负载，构造关联关系连续预取方法。该方法通过基于文件内数据的关联关系与基于元数据布局模型的文件间数据的关联关系构造统一的文件数据关联模型，并以此为数据预取提供指导决策。

最后，本方法构建客户端预取策略自调优模块。该模块为两种预取策略构建虚拟预取队列，依据策略的预取准确率，以滑动窗口的形式感知应用特征，自调优地选取当前最佳预取策略，并生成对应策略的预取任务加入到预取队列之中。

5.3.3 客户端侧的多级缓存机制

广域虚拟数据空间中构建了边缘缓存系统，其通过在同一数据中心中的近客户端侧节点上部署缓存节点来加速跨域数据访问，并内置了完善的数据同步机制来保证数据的一致性。

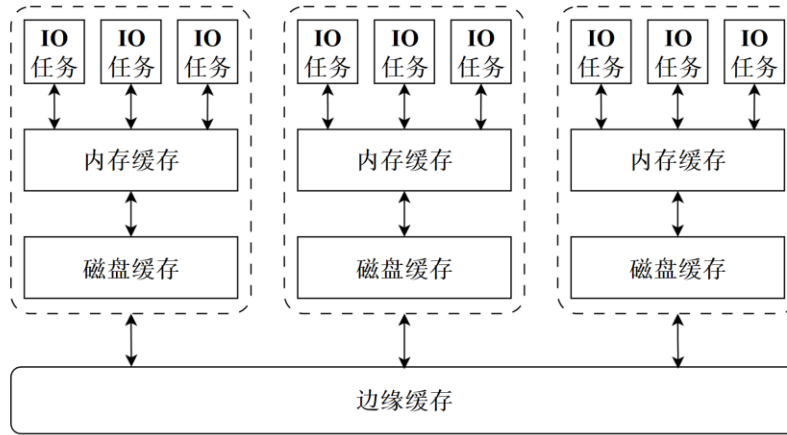


图 30 多级数据缓存示意图

如图 30 所示，本方法以边缘缓存为底座，在客户端节点的内存、磁盘上分别构建缓存机制。内存的访问速度快，但可用的空间有限；边缘缓存系统的访问速度慢，但是可用的空间巨大；而客户端节点的磁盘介于两者之间。

依据三者访问速度与可用空间的差异，本方法构建了多级缓存机制。换入的数据首先放入内存的空间中，而从内存中换出的数据则换入磁盘中，磁盘中换出的数据则换入边缘缓存系统中。出于简单高效的设计思想，本方法中的多级缓存系统均采用 LRU 作为缓存的替换策略进行实现。

并且，本方法设计了类似于快表的数据结构，在内存中记录了数据块的基本信息以及在多级缓存系统中的存储位置，便于以索引的形式访问缓存中的数据，避免多级缓存中的多次查找。

该缓存机制作为数据预取策略的基础能力，将本方法中所预取的文件数据缓存起来，后续的跨域数据访问请求命中时，即可起到屏蔽延时差异，提升访问性能的作用。

5.3.4 面向顺序读的快速增量预读方法

根据空间局部性原理，程序在运行时，最近被引用过的内存位置以及其临近的内存位置在之后的程序行为中更有可能被再次访问，这一点对于分布式文件系统来说也同样适用。在广域虚拟数据空间所管理的存储资源中，如果一个文件数据块被访问，那么这一文件数据块及其临近的文件数据块在之后的数据访问请求更有可能被再次访问。同时，在各类数据密集型应用的数据访问特征中，顺序读的访问负载均有不小的占比，文件系统的常用编辑操作中也有大量的遍历整个文件的操作。因此，顺序读访问负载加速对于

优化跨域数据访问至关重要，而面向顺序读的数据预取策略是加速访问的关键手段。

本方法构建了面向顺序读的快速增量预读方法，当应用发起了对某一文件数据块的跨域数据访问请求时，本方法将以数据块为粒度将与该数据块相邻的数据块进行预取，并存入缓存系统中。在本方法中，通过每个文件所关联的文件缓存控制块来记录该文件的历史信息，其中包括文件的 inode、数据节点地址、缓存数据块、历史预取数据块及命中率等多维度信息。本方法为文件中的块设置一项预取步长标识位，当发起对某一些块的预取任务时，将当前的预取步长赋予这些块的标识位，用以确定之后的预取步长。当正常的数据访问请求到来时，将该请求所涵盖数据块中标识位的最大值作为预取基准步长，并根据该请求的命中率在基准步长基础上动态伸缩。

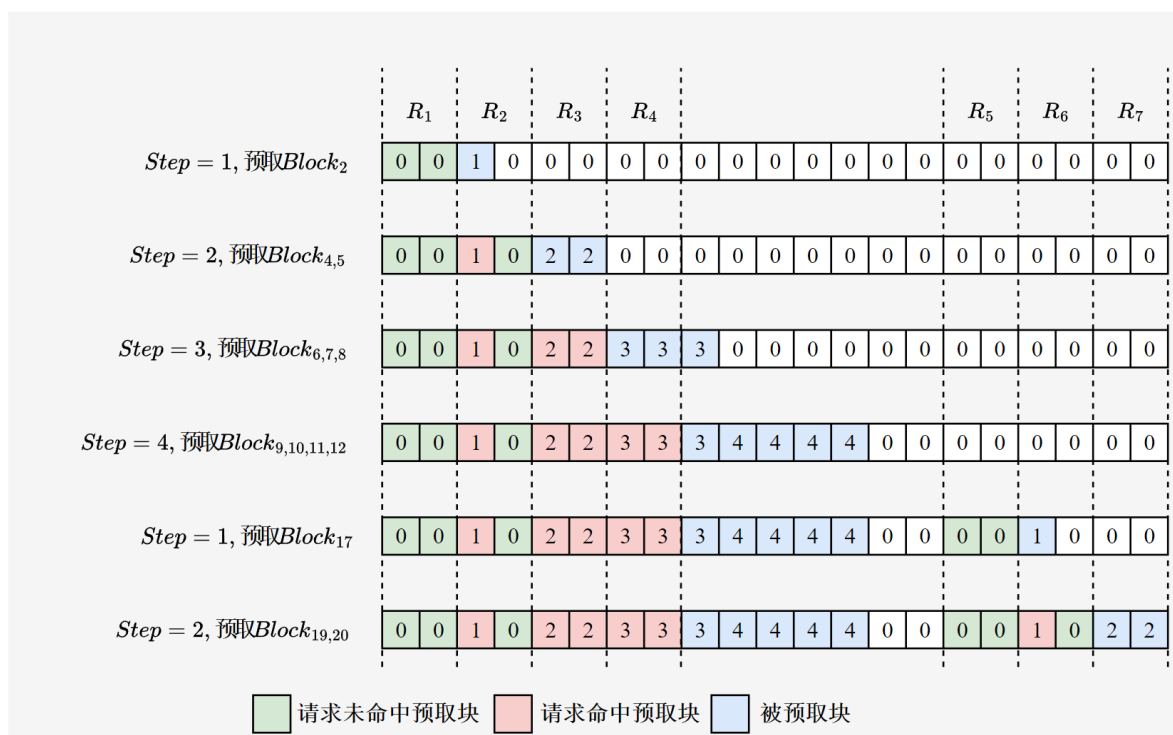


图 31 面向顺序读的快速增量预读方法

图 31 给出了本方法触发预取机制时的示意图。以每次获取两个数据块为例，首先数据请求为 block0 和 block1，两个数据块中步长标识位均为 0，即未被访问过，此时触发初始预取，步长为 1，发起预取任务获取 block2。而后数据请求为 block2 和 block3，命中了被预取的数据块 block2，而 block3 未命中，此时的预取基准步长为 1，并且命中率超过了阈值，因此预取步长增长为 2，发起预取任务获取 block4 和 block5。接着数据请求为 block4 和 block5，均命中被预取的数据块，因此在预取基准步长 2 之上增长为

3, 发起预取任务获取 block6、block7 和 block8。而当应用调整访问游标获取之后的数据块时, 则会覆盖标识位为 0 的数据块部分, 此时便会重新从步长为 1 开始数据预取。通过这样动态调整预取的步长, 可以使得预取策略更灵活地面对应用访问目标的变更, 避免错误预取过多不必要的数据块, 造成系统运行资源浪费。

5.3.5 面向关联读的关联数据预取方法

在工程实践中, 依据软件工程中的系统设计要素, 数据密集型应用的运行通常符合一定的计算模式, 而这也使得其所产生的文件系统数据访问请求通常具有一定的规律可循, 这样的 IO 访问路径也为文件中的数据之间赋予了路径关联性。通过分析数据之间的关联性, 即可以此为依据对应用的访问路径做出预测, 为数据预取提供决策, 提升预取的准确性。

本方法构建了面向关联读的关联数据预取方法, 通过 IO 访问路径发掘文件数据块之间的关联性, 并结合元数据布局模型探索文件间数据块的关联性, 以此构建广域虚拟数据空间中的数据关联性模型, 为关联读数据预取方法提供决策依据。一方面, 本方法通过对系统中持续运行应用的访问路径进行记录, 标识路径之中相连数据块的关系, 以此作为系统中基础的关联关系模型。另一方面, 出于元数据和数据之间的强耦合关系, 即当应用访问了某一文件的数据时, 与该文件处于一个目录下的其他文件会更有可能被访问, 本方法从广域虚拟数据空间中区域-空间-路径的元数据布局中探索了文件间数据块的关联关系。

通过上述的文件内与文件间的关联关系, 本方法即可构造出广域虚拟数据空间中的数据关联模型, 以此作为数据预取的决策依据。图 32 给出了本方法基本架构图。根据数据关联模型, 本方法构建出数据块之间的映射关系表, 表中系数即为块之间的权重, 代表了数据块关系的强弱程度。当应用对文件进行数据访问时, 依据映射关系表, 将与当前数据块关系最紧密的数个数据块从数据节点预取至缓存系统中, 以加速后续的数据访问请求。由于这本身是一种概率上命中的行为, 该初始映射表的准确性仍有待提升, 且不同的应用对于同一文件的访问特征也存在差异, 需要对其在运行时动态调整, 以提升预取准确率。因此, 本方法会在应用的持续运行过程中, 依据访问数据块的先后顺序, 对映射关系表进行更新, 以更贴合目前应用的实际工作负载。

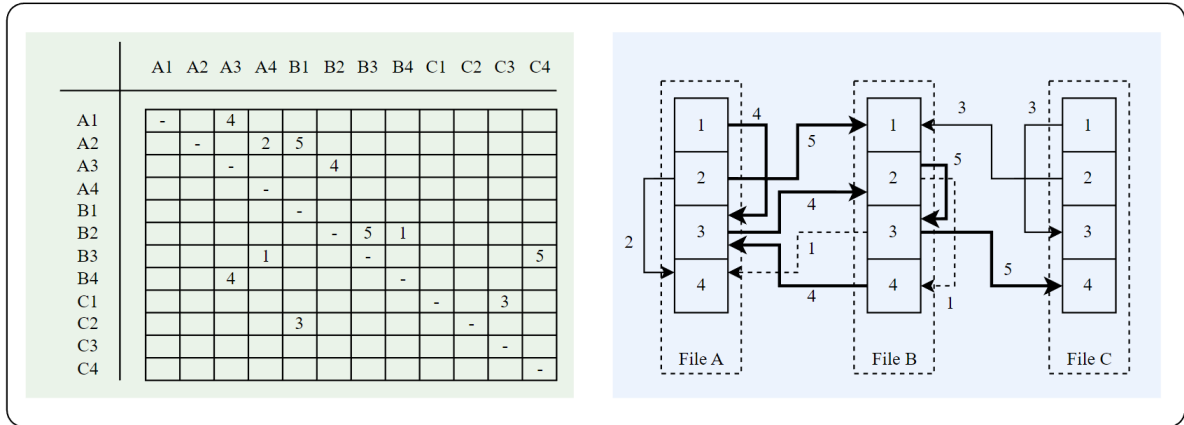


图 32 面向关联读的关联数据预取方法

5.3.6 基于跨域应用负载感知的数据预取自调优方法

在 5.3.4 和 5.3.5 两小节中，介绍了本章节所设计的两种数据预取方法，其分别针对顺序读和关联读两种不同的场景做出了针对性的设计，提供数据访问加速能力，但对于对方的场景可能产生负面的影响。因此，本文提出了基于跨域应用负载感知的数据预取自调优方法，通过分析应用的访问特征与不同预取方法的命中率，选取当前情况最合适的数据预取方法，以适应应用访问负载的变化。

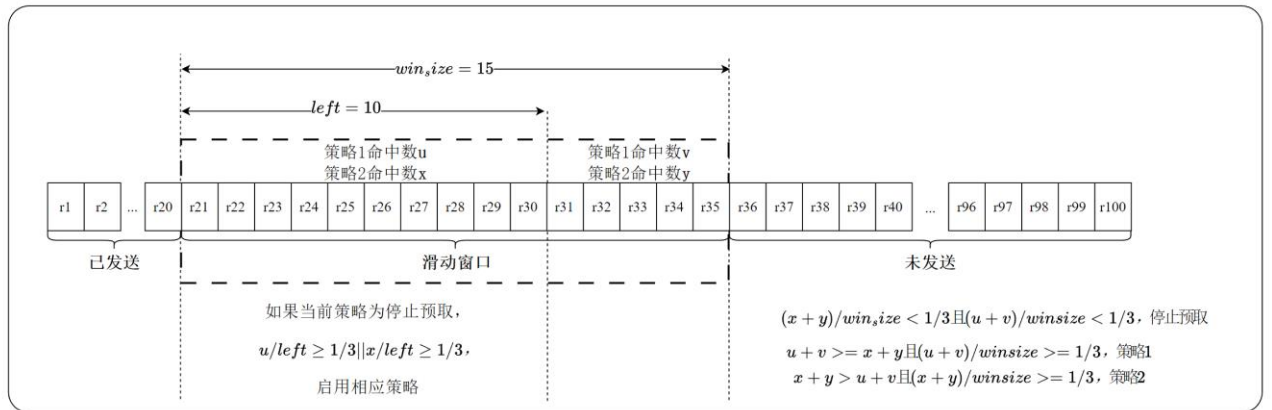


图 33 数据预取自调优方法示意图

为能够清晰地展现出不同预取策略的命中率，本方法引入了滑动窗口的结构。本方法通过滑动窗口来维护对最近访问任务的分析，滑动窗口中的数据信息即应用对于文件的访问流，每个节点为一个 IO 任务。滑动窗口被分为三大部分，已完成请求部分、窗口内部分以及未发送部分，通过计算窗口内部分两种预取方法的缓存命中率，来为后续的预取策略选择提供决策依据。

算法 7 基于跨域应用特征感知的自适应数据预取方法

Input:*Req*: 新增 IO 任务信息, 其中包含字段应用 *aid*, 任务大小 *size**Strategy*: 当前预取策略*Window*: 滑动窗口*Win_size*: 窗口大小**Output:***Prefetch*: 新增的预取任务

```

1  prefetchcon, hitcon  $\leftarrow$  ConsecutivePrefetch(Req)
2  prefetchseq, hitseq  $\leftarrow$  SequentialPrefetch(Req)
3  Window.add({hitcon, hitseq})
4  totalhitcon, totalhitseq  $\leftarrow$  Window.curWindow()
5  if Window reaches right do
6      threshold  $\leftarrow$  Win_size / 3
7      if totalhitcon < threshold | totalhitseq < threshold do
8          Strategy  $\leftarrow$  NonFetch
9      else if totalhitcon  $\geq$  totalhitseq do
10         Strategy  $\leftarrow$  Consecutive
11     else do
12         Strategy  $\leftarrow$  Sequential
13     end if
14 else if Strategy == NonFetch do
15     if totalhitcon  $\geq$  totalhitseq & totalhitcon  $\geq$  Window.left / 3 do
16         Strategy  $\leftarrow$  Consecutive
17     else if totalhitseq > totalhitcon & totalhitseq  $\geq$  Window.left / 3 do
18         Strategy  $\leftarrow$  Sequential
19     end if
20 end if
21 switch Strategy do
22     case NonFetch return None
23     case Consecutive return prefetchcon
24     case Sequential return prefetchseq
25 end switch

```

图 33 给出了本方法的示意图, 在窗口内依据应用的数据访问请求, 运用当前选取的预取方法进行实际的预取操作, 满足应用 IO 任务数据需要的同时, 计算缓存的命中次数。而对于未选取的预取方法, 本方法构建了虚拟预取队列, 按照窗口内的访问任务, 根据该预取方法, 模拟一遍数据预取的路径, 记录到虚拟预取队列之中, 但不进行实际的数据获取, 以此便可以计算出采用该预取方法时的命中次数。

算法 7 显示了数据预取自调优方法的算法过程。当数据访问流到达时, 两种预取方

法的命中次数均设置为 0，出于顺序读占比更高的考虑，初始预取策略选定为增量顺序预取方法。算法维护了窗口的起始位置和末尾位置，在窗口内使用顺序预取和连续预取两种方法进行虚拟预取，而后根据当前选择的预取策略将预取任务返回。当到达窗口的末尾位置时，根据窗口内两种预取策略的模拟命中次数进行统计比较，来为下一个窗口中的请求选取合适的预取策略：如果两种策略的命中率均达不到预取阈值（本文设置为 $1/3$ ），则表示当前的应用访问特征无明显特征，应停止预取；而如果策略 1 的命中数大于等于策略 2，则下一段窗口内选取策略 1 作为预取策略，反之则选取策略 2。为了提高预取策略的响应度，当本方法选取停止预取时，如果窗口内请求发生过程中出现某一预取策略达到了预取阈值，则迅速切换为该策略进行实际预取流程。

5.4 实验与验证

5.4.1 实验环境

在本小节中将对本文所提出基于应用特征感知的自适应数据预取方法进行实验验证，在本实验中，采用 3.4.1 小节中相同的实验拓扑图构建的模拟环境中进行，以便自定义调整网络环境。实验所用实验平台软硬件环境配置如下表 11 所示。

表 11 数据预取策略实验平台软硬件环境配置表

软硬件	详细描述
CPU	Intel(R) Xeon(R) Silver 4110
内存	64GB
磁盘	AVAGO MR9361-8i 4TB
网络带宽	1000M（测速约 124MB/s）
操作系统	Ubuntu 20.04.4 LTS
操作系统内核	5.4.0
Rust	1.73.0-nightly

5.4.2 实验场景描述

为了验证本方法的有效性和性能，本文对实际生产环境中运行的数种应用的 trace 进行采集，通过 trace 回放的方法评估各预取方法的缓存命中率与总吞吐量。本方法所采用 trace 如下：

顺序型负载，通过在五个大小为 10GB 的文件中运行 MapReduce 任务，并记录其每次进行 IO 操作时涉及的文件块信息并形成 trace 文件，其运行的大体应用特征为顺序遍

历整个文件并交替进行数据处理。

关联型负载，通过在五个大小为 100GB 的文件中运行 FileSearch 任务，并使用 Recorder 将其对文件系统的 IO 操作记录下来形成 trace，其运行的大体应用特征为连续有规律地访问文件数据。

合并型负载，分为先顺序后关联和先关联后顺序两种，即顺序地合并顺序型负载和关联性负载。这一负载用于模拟测试两个不同应用顺序执行时的性能表现，其在大型计算环境中同样常见。

随机型负载，即以随机数的形式生成对文件系统的访问负载，包含文件信息、块信息等，用以模拟测试本方法在完全无规律场景下的表现。

本文选取了快速增量预取（Consecutive）、关联数据预取（Sequential）、单文件关联数据预取（SeqSingle）和不预取（NonFetch，即简单使用缓存的 LRU 能力）作为本方法的对照方法，其实验参数入下表 12 所示。

表 12 自适应数据预取方法实验参数表

块大小	内存缓存大小	磁盘缓存大小	窗口大小	顺序预取最大步长	连续预取最大步长
4MB	16	256	20	5	5

5.4.3 实验结果与分析

图 34 显示了本方法与其他方法在顺序型负载场景下的缓存命中率和总吞吐量的性能表现。从图中可以看出，在该应用特征中，系统整体的访问负载较为平稳，随着访问块的增多，各预取方法的缓存命中率基本没有变化。本方法和快速增量预取能够保持近乎一致的缓存命中率与总吞吐量，这是因为本方法选择快速增量预取作为初始预取方法，而随着 trace 持续回放，本方法始终识别该方法为目前最优方法直至结束，因此在这一过程中实际上本方法等同于快速增量预取方法。由于该负载近乎为从头到尾遍历文件内容，因此关联数据预取和不预取两种方法均很难预取到准确的数据块，从而展现出极低的缓存命中率和吞吐量。相比于不包含顺序预取的其他方法，本方法取得了平均 311% 的吞吐量提升。

图 35 显示了本方法与其他方法在关联型负载场景下的缓存命中率和总吞吐量的性能表现。与顺序型负载场景下的表现有着类似的特征，在当前场景下本方法能够取得和关联数据预取方法一致的缓存命中率，而快速增量预取和单文件关联数据预取则表现较

差。值得一提的是，此时不预取也能够有一定的缓存命中率，这是因为该负载与顺序型负载不会访问历史数据块这一点不同，LRU 仍有机会发挥一定作用。相比于快速增量预取和不预取，本方法的总吞吐量分别提升了 299%和 133%。

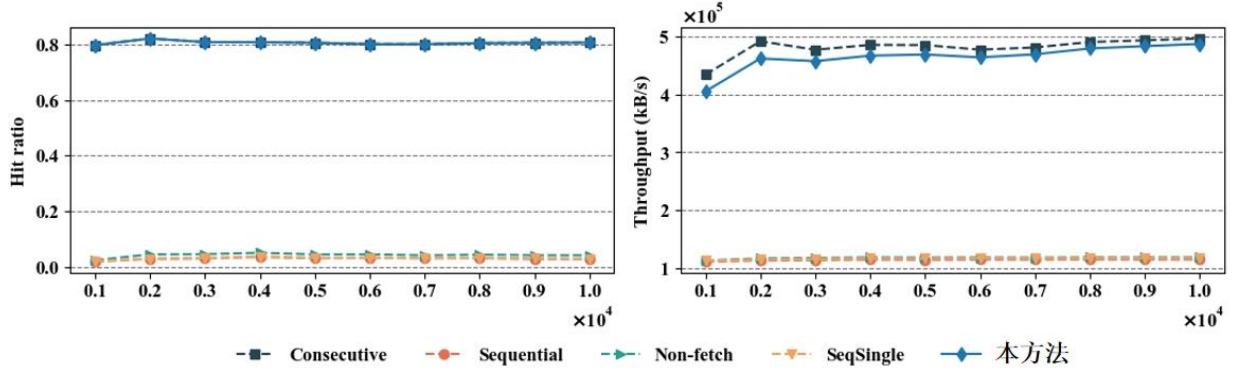


图 34 顺序型负载数据预取实验结果图

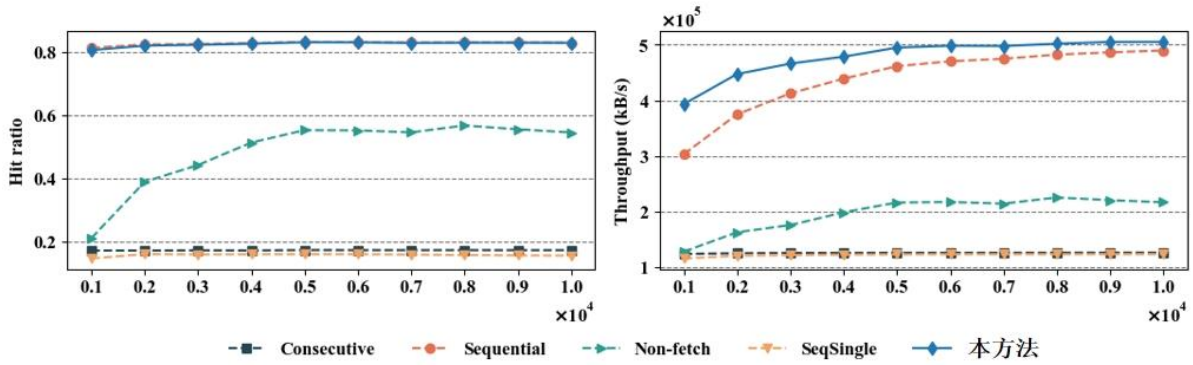


图 35 关联型负载数据预取实验结果图

图 36 显示了本方法与其他方法在先顺序后关联负载场景下的缓存命中率和总吞吐量的性能表现。此时由于夹杂了两种迥异的访问负载，该负载下在中间的分水岭处均产生了显著变化。因为先进行顺序型的访问负载，因此起初的 10000 个数据块中快速增量预取方法表现出显著的性能优势，但随着负载特征的变化，该方法的缓存命中率出现了急剧的下降，而总吞吐量也随之降低。而关联数据预取在变化前性能表现不佳，在应用特征变化之后才开始逐渐发挥性能优势，由于不预取方法在关联型负载中仍有一定发挥空间，因此其性能表现与关联数据预取相似。但是，本方法在该负载的整个过程中始终保持着优秀的性能表现，这是因为本方法的自适应特性，使得在面临应用特征的变化时，本方法能够识别并迅速决策出最符合当前负载特征的预取方法，因此本方法的性能表现即为每个时间段最佳预取方法的性能表现。在 20000 个数据块时，本方法相较于单纯的快速增量预取、关联数据预取方法分别提升了 62%、92%的缓存命中率和 141%、163%

的总吞吐量。

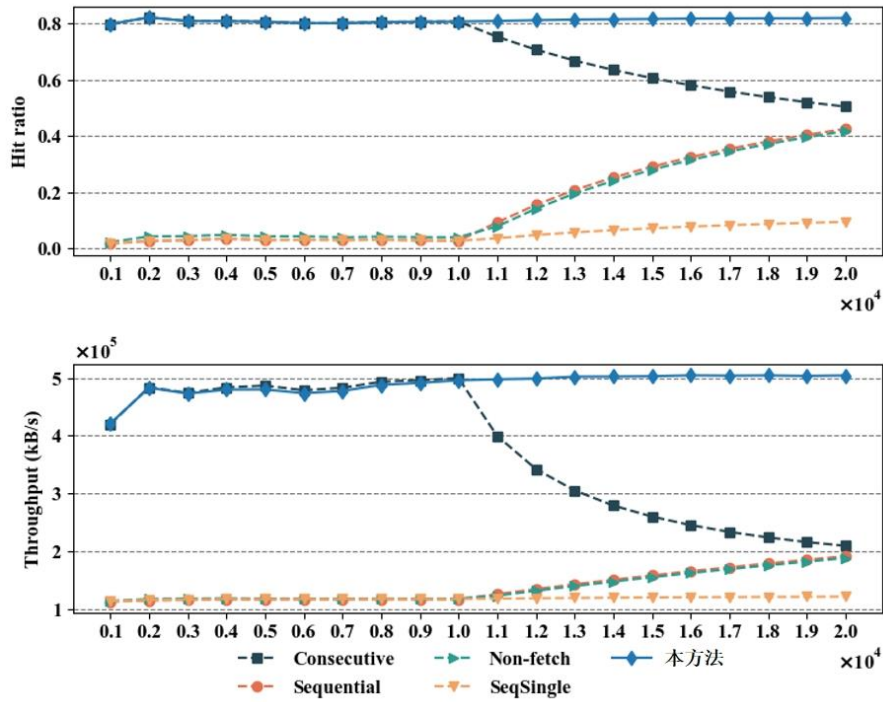


图 36 先顺序后关联负载数据预取实验结果图

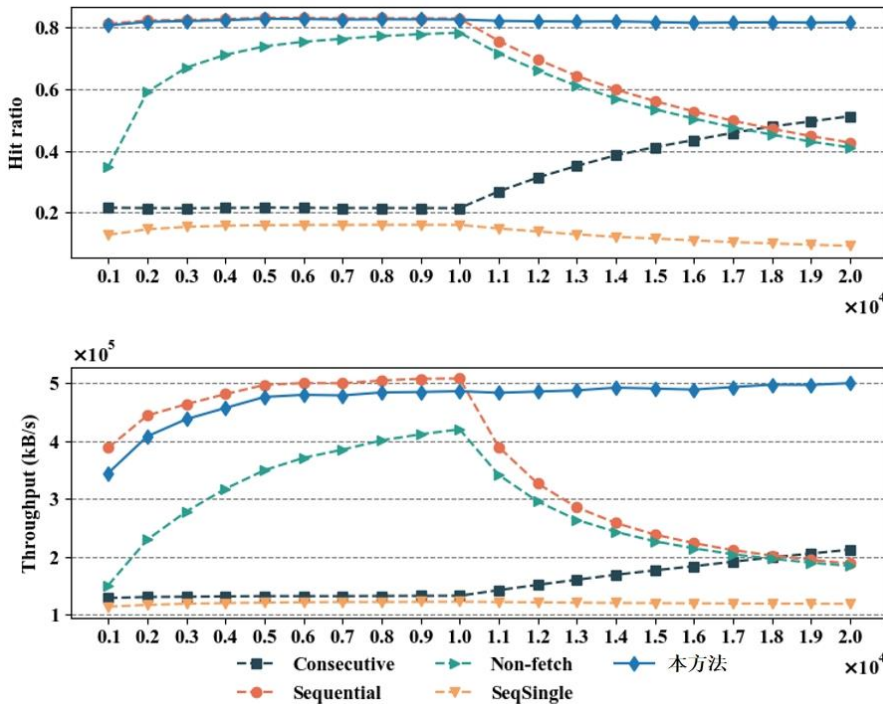


图 37 先关联后顺序负载数据预取实验结果图

图 37 显示了本方法与其他方法在先关联后顺序负载场景下的缓存命中率和总吞吐量的性能表现。和先顺序后关联类似，快速增量预取和关联数据预取在应用特征发生变

化时出现了显著的性能变化，而本方法能够始终保持较高的性能表现。在 20000 个数据块时，本方法相较于单纯的快速增量预取、关联数据预取方法分别提升了 59%、92% 的缓存命中率和 135%、165% 的性能提升。

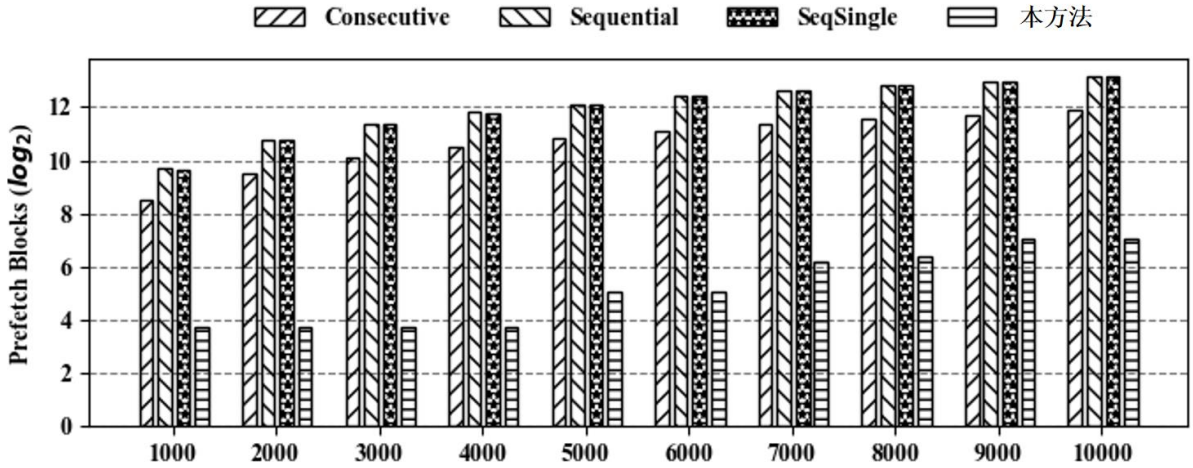


图 38 随机负载数据预取实验结果图

为了测试本方法在应对无规律负载时的表现，本方法通过随机数生成了随机的 IO 负载 trace，图 38 给出了在该负载场景下本方法和其他方法预取数据块的数目随着访问数据块数量的变化值。由于不同方法之间结果的数量级相差较大，此处结果采用了对数的形式展现。从图中可以看出，本方法几乎不进行数据预取，在 10000 个数据块时也仅仅一共发起了 133 个数据块的预取任务，而其他方法多则预取 9000 余个数据块，少则 3000 余个。这是因为本方法在评估两种数据预取方法均无法取得理想的访问加速效果之后，会转而采取不预取的策略。在无规律的负载特征场景下，错误的数据预取操作无法对应用的数据访问过程提供帮助，反而额外的网络传输开销导致系统整体性能下降。

5.5 本章小结

针对广域虚拟数据空间的跨域数据访问通路中面临的性能问题，本章设计并实现了基于跨域应用特征感知的自适应数据预取方法，通过分析应用的数据访问特征，采用自调优的方法，从快速增量顺序预读和关联数据预读中选取合适、准确的数据预取策略，提升了客户端数据缓存的命中率，提升了跨域数据访问性能。主要的工作如下：

(1) 设计并实现了基于跨域应用特征感知的自适应关联数据预取方法。首先，基于广域虚拟数据空间中的边缘缓存系统，构建了客户端侧的多级缓存机制，加速后续跨

域访问请求；其次，面向顺序读和关联读场景，分别构建快速增量预取方法和关联数据预取方法，为特定跨域应用负载提供访问加速；最后，通过虚拟预取队列和滑动窗口机制，实现基于跨域应用负载感知的自适应数据预取方法，综合上述两种数据预取策略的运行效果，动态地为应用选取最佳的预取策略，提升缓存命中率。由此，本方法为跨域数据访问请求带来了显著的性能提升。

（2）构建了实验系统。在实际分布式环境中实现了本章节设计的数据预取策略，并通过实验对比了本方法和其余方法的性能表现。实验结果表明，本方法可以准确地识别出跨域应用特征，在单一负载场景下能始终保持和最佳预取策略一致的性能表现，而在顺序读和关联读混合的复杂负载场景下，相较于其余方法能够取得至少 59% 的缓存命中率提升和 151% 的吞吐量提升。并且，在面对无规律场景时，本方法能快速识别并采取不预取的策略，避免不必要的系统资源浪费。

总结与展望

论文工作总结

本论文面向广域虚拟数据空间的跨域数据访问通路中面临的可靠性、公平性和性能问题,开展面向广域虚拟数据空间的基于网络状况感知与时延预测的可靠跨域重传机制、基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制、基于跨域应用特征感知的自适应数据预取方法等技术的研究,优化跨域数据访问通路中的三大关键路径,提高应用跨域数据访问效率。本文的主要工作和成果总结如下:

(1) 广域网络波动,跨域数据访问请求难以完整执行导致可靠性差。针对以上问题,本文设计并实现了基于网络感知与时延预测的可靠跨域重传机制。该方法构建基于网络感知和时序分析的时延预测机制,通过长短期记忆网络与滑动窗口重传机制预测算法,计算目标节点往返时延预测值;设计快速重定向机制,在网络链路出现异常时,依据预测值做出重定向决策,打断当前数据访问请求,并切换目标节点以完成后续请求。实验结果表明,在面对异常的网络环境时,本方法显著地提升了跨域数据访问的可靠性和数据访问性能。

(2) 广域带宽受限,大块跨域数据访问请求独占带宽资源导致公平性低。针对以上问题,本文设计并实现了基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制。该方法构建多应用 IO 队列,通过 IO 任务拆分,均衡跨域数据访问请求大小,并依据应用标识符进行分组,赋予动态均衡权重;引入虚拟运行 IO 作为带宽分配的决策依据,通过计算实际运行 IO 与应用权重的比值,更新虚拟运行 IO;基于虚拟运行 IO 进行非抢占式和抢占式的跨域数据访问请求调度,实现跨域数据访问带宽公平分配。实验结果表明,本方法相较于传统调度算法显著地提升了带宽分配的公平性,同时相比于其它公平带宽分配方法拥有更高的性能表现。

(3) 广域应用复杂,单一跨域数据预取策略时常失效导致性能不足。针对以上问题,本文设计并实现了基于跨域应用特征感知的自适应数据预取方法。该方法在已有的边缘缓存系统之上,构建多级缓存机制,加速后续跨域访问请求;面向顺序读和关联读场景分别构建了快速增量预取方法和关联数据预取方法,通过虚拟预取队列和滑动窗口

机制，采取自调优的方法综合多种数据预取策略的运行效果，动态地为应用选取最佳的预取策略。实验结果表明，本方法可以准确地识别出跨域应用特征，在单一负载场景下能始终保持和最佳预取策略一致的性能表现，而在顺序读和关联读混合的复杂负载场景下，相较于其余方法能够显著提升缓存命中率与性能。

下一步工作展望

本文解决了广域虚拟数据空间的跨域数据访问通路中的三大关键问题，优化了跨域数据访问效率，但是这些方法仍具有较大的改进空间，其中主要包含以下三方面：

(1) 针对跨域数据访问通路中面临的可靠性问题，本文提出了基于网络状况感知与时延预测的可靠跨域重传机制。其中为了对系统目前的网络状况进行预测，本文采用了长短期记忆网络的机器学习方法对系统采集的数项网络参数和请求参数进行分析和预测，但是在实验中发现其预测准确率与额外开销仍有待优化，偶尔会出现错误重定向的情况，影响系统性能。后续的研究工作需要引入更多维度的网络参数和更准确的计算网络，进一步提升预测的准确率。

(2) 针对跨域数据访问通路中面临的公平性问题，本文提出了基于虚拟运行 IO 的多应用跨域数据访问带宽公平分配机制。该方法依据应用的 IO 规模来为其动态公平分配带宽资源，但未留有人工干涉机制，忽略了实际生产环境中小部分高优先级计算密集型应用的实际需求。后续的研究工作需要设计合适的人工干涉机制，或是在公平调度的基础上构建多级优先队列，以满足各类应用的带宽资源需求。

(3) 针对跨域数据访问通路中面临的性能问题，本文提出了基于跨域应用特征感知的自适应数据预取方法。该方法分别为顺序读和关联读场景设计了数据预取策略，同时使用自调优的手段从中选择合适的预取策略，但两种策略都是较为简单的机制，仍无法完全涵盖实际运行系统中各式各样的应用访问负载。后续的研究工作可以引入更多高效的数据访问策略，使得自适应预取能有更多更好的选择。由于虚拟预取队列的存在，多种数据预取策略也不会产生更多的额外开销。

参考文献

- [1] 肖利民, 宋尧, 秦广军, 等. GVDS: 面向广域高性能计算环境的虚拟数据空间[J]. 大数据, 2021, 7(02): 123-146
- [2] 何小雨, 邓笋根, 栾海晶, 等. 国家高性能计算环境的虚拟数据空间运行支撑技术研究[J]. 大数据, 2021, 7(02): 158-171
- [3] 张晨浩, 肖利民, 秦广军, 等. 面向大数据处理应用的广域存算协同调度系统[J]. 大数据, 2021, 7(05): 82-97
- [4] 钱德沛, 王锐. E 级计算的几个问题[J]. 中国科学: 信息科学, 2020, 50(09): 1303-1326
- [5] Axboe J. Flexible I/O Tester[DB/OL]. <https://github.com/axboe/fio>, 2016
- [6] Huo Z., Guo M., Xiao L., et al. TACD: A Throughput Allocation Method Based on Variant of Cobb–Douglas for Hybrid Storage System[J]. Journal of Parallel and Distributed Computing, 2019, 128: 43-56
- [7] Pabla C. S. Completely Fair Scheduler[J]. Linux Journal, 2009, 2009(184): 4
- [8] Hou B., Chen F. Pacaca: Mining Object Correlations and Parallelism for Enhancing User Experience with Cloud Storage[C]. 2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS). IEEE, 2018: 293-305
- [9] Hsu W. W., Smith A. J., Young H. C. The Automatic Improvement of Locality in Storage Systems[J]. ACM Transactions on Computer Systems (TOCS), 2005, 23(4): 424-473
- [10] Sanfilippo S., Noordhuis P. Redis[DB/OL]. <https://redis.io>, 2009
- [11] Borthakur D. HDFS Architecture Guide[J]. Hadoop apache project, 2008, 53(1-13): 2
- [12] Widenius M., Axmark D. MySQL Reference Manual: Documentation from the Source[M]. " O'Reilly Media, Inc.", 2002
- [13] Gormley C., Tong Z. Elasticsearch: the Definitive Guide: a Distributed Real-time Search and Analytics Engine[M]. " O'Reilly Media, Inc.", 2015
- [14] Wang B., Siarkowicz M., Zala S., et al. Etcd[DB/OL]. <https://etcd.io>, 2013

- [15] Hunt P., Konar M., Junqueira F. P., et al. ZooKeeper: Wait-free Coordination for Internet-scale Systems[C]//2010 USENIX Annual Technical Conference (USENIX ATC 10). 2010
- [16] Ogleari M. A., Miller E. L., Zhao J. Steal But No Force: Efficient Hardware Undo+ Redo Logging for Persistent Memory Systems[C]. 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE, 2018: 336-349
- [17] Ren K., Zheng Q., Patil S., et al. IndexFS: Scaling File System Metadata Performance with Stateless Caching and Bulk Insertion[C]. SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2014: 237-248
- [18] Das S., Agrawal D., El Abbadi A. G-store: a Scalable Data Store for Transactional Multi Key Access in the Cloud[C]. Proceedings of the 1st ACM symposium on Cloud computing. 2010: 163-174
- [19] Sears R., Ramakrishnan R. bLSM: a General Purpose Log Structured Merge Tree[C]. Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data. 2012: 217-228
- [20] Hamilton R., Iyengar J., Swett I., et al. QUIC: a UDP-Based Secure and Reliable Transport for HTTP/2[EB/OL]. <https://datatracker.ietf.org/doc/draft-tsvwg-quic-protocol/02/>, 2016
- [21] Akan O. B., Akyildiz I. F. Event-to-sink Reliable Transport in Wireless Sensor Networks[J]. IEEE/ACM transactions on networking, 2005, 13(5): 1003-1016
- [22] Wan C. Y., Campbell A. T., Krishnamurthy L. PSFQ: a Reliable Transport Protocol for Wireless Sensor Networks[C]. Proceedings of the 1st ACM international workshop on Wireless sensor networks and applications. 2002: 1-11
- [23] Ganesan D., Govindan R., Shenker S., et al. Highly-resilient, Energy-efficient Multipath Routing in Wireless Sensor Networks[J]. ACM SIGMOBILE Mobile Computing and Communications Review, 2001, 5(4): 11-25
- [24] Deb B., Bhatnagar S., Nath B. ReInForM: Reliable Information Forwarding using Multiple Paths in Sensor Networks[C]. 28th Annual IEEE International Conference on Local Computer Networks, 2003. LCN'03. Proceedings. IEEE, 2003: 406-415

-
- [25] Yahya B., Ben-Othman J. RELAX: an Energy Efficient Multipath Routing Protocol for Wireless Sensor Networks[C]. 2010 IEEE International Conference on Communications. IEEE, 2010: 1-6
- [26] Kwon H., Kim T. H., Choi S., et al. A Cross-layer Strategy for Energy-efficient Reliable Delivery in Wireless Sensor Networks[J]. IEEE Transactions on Wireless Communications, 2006, 5(12): 3689-3699
- [27] Braam P. The Lustre Storage Architecture[J]. arXiv preprint arXiv:1903.01955, 2019
- [28] Weil S., Brandt S. A., Miller E. L., et al. Ceph: a Scalable, High-performance Distributed File System[C]. Proceedings of the 7th Conference on Operating Systems Design and Implementation (OSDI'06). 2006: 307-320
- [29] Zhong S., Ye C., Hu G., et al. MadFS:Per-File Virtualization for Userspace Persistent Memory Filesystems[C]. 21st USENIX Conference on File and Storage Technologies (FAST 23). 2023: 265-280
- [30] Gulati A., Merchant A., Varman P. J. mClock: Handling Throughput Variability for Hypervisor IO Scheduling[C]. 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10). 2010
- [31] Gulati A., Shanmuganathan G., Zhang X., et al. Demand Based Hierarchical QoS Using Storage Resource Pools[C]. 2012 USENIX Annual Technical Conference (USENIX ATC 12). 2012: 1-13
- [32] Karlsson M., Karamanolis C., Zhu X. Triage: Performance Differentiation for Storage Systems Using Adaptive Control[J]. ACM Transactions on Storage (TOS), 2005, 1(4): 457-480
- [33] Wong T. M., Golding R. A., Lin C., et al. Zygaria: Storage Performance as a Managed Resource[C]. 12th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'06). IEEE, 2006: 125-134
- [34] Gulati A., Ahmad I., Waldspurger C. A. PARDA: Proportional Allocation of Resources for Distributed Storage Access[C]. FAST. 2009, 9: 85-98
- [35] Ghodsi A., Zaharia M., Hindman B., et al. Dominant Resource Fairness: Fair Allocation

- of Multiple Resource Types[C]. 8th USENIX symposium on networked systems design and implementation (NSDI 11). 2011
- [36] Dolev D., Feitelson D. G., Halpern J. Y., et al. No Justified Complaints: on Fair Sharing of Multiple Resources[C]. proceedings of the 3rd Innovations in Theoretical Computer Science Conference. 2012: 68-75
- [37] Gutman A., Nisan N. Fair Allocation without Trade[J]. arXiv preprint arXiv:1204.4286, 2012
- [38] Wang W., Liang B., Li B. Multi-resource Fair Allocation in Heterogeneous Cloud Computing Systems[J]. IEEE Transactions on Parallel and Distributed Systems, 2014, 26(10): 2822-2835
- [39] Huo Z., Xiao L., Zhong Q., et al. Hybrid Storage Throughput Allocation among Multiple Clients in Heterogeneous Data Center[C]. 2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th International Conference on Embedded Software and Systems. IEEE, 2015: 140-147
- [40] Wang H., Varman P. Balancing Fairness and Efficiency in Tiered Storage Systems with Bottleneck-Aware Allocation[C]. 12th USENIX Conference on File and Storage Technologies (FAST 14). 2014: 229-242
- [41] Fürnkranz J. Round Robin Classification[J]. The Journal of Machine Learning Research, 2002, 2: 721-747
- [42] Greenberg A. G., Madras N. How Fair is Fair Queuing[J]. Journal of the ACM (JACM), 1992, 39(3): 568-598
- [43] Wang W., Casale G. Evaluating Weighted Round Robin Load Balancing for Cloud Web Services[C]. 2014 16th international symposium on symbolic and numeric algorithms for scientific computing. IEEE, 2014: 393-400
- [44] Shreedhar M., Varghese G. Efficient Fair Queuing Using Deficit Round-robin[J]. IEEE/ACM Transactions on networking, 1996, 4(3): 375-385
- [45] Kim H., Han Y. A Proportional Fair Scheduling for Multicarrier Transmission Systems[J].

- IEEE Communications letters, 2005, 9(3): 210-212
- [46] Tan H., Tan Y., He X., et al. A Virtual Multi-channel GPU Fair Scheduling Method for Virtual Machines[J]. IEEE Transactions on Parallel and Distributed Systems, 2018, 30(2): 257-270
- [47] Mahajan K., Balasubramanian A., Singhvi A., et al. Themis: Fair and Efficient GPU Cluster Scheduling[C]. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20). 2020: 289-304
- [48] Gu L., Zeng D., Tao S., et al. Fairness-aware Dynamic Rate Control and Flow Scheduling for Network Utility Maximization in Network Service Chain[J]. IEEE Journal on Selected Areas in Communications, 2019, 37(5): 1059-1071
- [49] Garcia-Garcia A., Saez J. C., Prieto-Matias M. Contention-aware Fair Scheduling for Asymmetric Single-ISA Multicore Systems[J]. IEEE Transactions on Computers, 2018, 67(12): 1703-1719
- [50] Kim C., Huh J. Exploring the Design Space of Fair Scheduling Supports for Asymmetric Multicore Systems[J]. IEEE Transactions on Computers, 2018, 67(8): 1136-1152
- [51] Ghemawat S., Gobioff H., Leung S. T. The Google File System[C]. Proceedings of the nineteenth ACM symposium on Operating systems principles. 2003: 29-43
- [52] Niazi S., Ismail M., Haridi S., et al. HopsFS: Scaling Hierarchical File System Metadata Using NewSQL Databases[C]. 15th USENIX Conference on File and Storage Technologies (FAST 17). 2017: 89-104
- [53] Li S., Lu Y., Shu J., et al. LocoFS: A Loosely-coupled Metadata Service for Distributed File Systems[C]. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2017: 1-12
- [54] Li Z., Chen Z., Srinivasan S. M., et al. C-miner: Mining Block Correlations in Storage Systems[C]. FAST. 2004, 4: 173-186
- [55] Kuenning G. H. The Design of the Seer Predictive Caching System[C]. 1994 First Workshop on Mobile Computing Systems and Applications. IEEE, 1994: 37-43
- [56] Zhang S., Catanese H., Wang A. A. I. The Composite-file File System: Decoupling the

- One-to-One Mapping of Files and Metadata for Better Performance[C]. 14th USENIX Conference on File and Storage Technologies (FAST 16). 2016: 15-22
- [57] Jamilan S., Khan T. A., Ayers G., et al. Apt-get: Profile-guided Timely Software Prefetching[C]. Proceedings of the Seventeenth European Conference on Computer Systems. 2022: 747-764
- [58] Jacobson V. Congestion Avoidance and Control[J]. ACM SIGCOMM computer communication review, 1988, 18(4): 314-329
- [59] Lütkepohl H. Forecasting with VARMA Models[J]. Handbook of economic forecasting, 2006, 1: 287-325
- [60] Horvath C., Leeflang P. S. H., Wieringa J. E., et al. Competitive Reaction-and Feedback Effects Based on VARX Models of Pooled Store Data[J]. International Journal of Research in Marketing, 2005, 22(4): 415-426
- [61] Hochreiter S., Schmidhuber J. Long Short-term Memory[J]. Neural computation, 1997, 9(8): 1735-1780
- [62] Vaswani A., Shazeer N., Parmar N., et al. Attention is All You Need[J]. Advances in neural information processing systems, 2017, 30
- [63] Zhou H., Zhang S., Peng J., et al. Informer: Beyond Efficient Transformer for Long Sequence Time-series Forecasting[C]. Proceedings of the AAAI conference on artificial intelligence. 2021, 35(12): 11106-11115
- [64] Imambi S., Prakash K. B., Kanagachidambaresan G. R. PyTorch[J]. Programming with TensorFlow: Solution for Edge Computing Applications, 2021: 87-104
- [65] 霍建同, 肖利民, 霍志胜, 等. 广域虚拟数据空间中边缘缓存系统的研究与实现[J]. 大数据, 2021, 7(05): 57-81
- [66] Wang C., Sun J., Snir M., et al. Recorder 2.0: Efficient Parallel I/O Tracing and Analysis[C]. 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE, 2020: 1-8

攻读学位期间取得的成果

学术论文

- [1] Shang Jing, Xiao Limin, Wu Zhihui, **Yang Jinqian**, Xiao Zhiwen, Wang Jinquan, Zhang Yifei, Chen Xuguang, Wang Jibin, Li Huiyang. GvdsSQL: Heterogeneous Database Unified Access Technology for Wide-Area Environments[J]. Electronics, 2024(SCI IF=2.9, 已录用)
- [2] Shen Runnan, Wang Liang, Luo Haotian, Yang Rui, **Yang Jinqian**, Wang Jinquan, Sun Qiancheng, Xiao Limin, Dong Jin. Accelerating zk-SNARK with Group and Zone Optimization on GPU[C]. 2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS). IEEE, 2023: 24-31(CCF C)

发明专利

- [1] 肖利民, **杨锦乾**, 王锦权, 沈润楠, 李辉阳, 陈旭光. 一种广域分布式文件系统中的IO任务完全公平调度算法[P]. 中国专利: 202311302613.6(已受理)
- [2] 肖利民, **杨锦乾**, 王锦权, 沈润楠, 李辉阳, 陈旭光. 一种广域环境中异构数据库的代码生成拓展方法[P]. 中国专利: 202311329602.7(已受理)
- [3] 肖利民, 郭桐, 刘禹廷, 沈润楠, **杨锦乾**. 一种基于热力图与风格迁移的对抗补丁隐蔽性增强方法[P]. 中国专利: 202211265098.4(已受理)
- [4] 肖利民, 沈润楠, 王良, 郭桐, 刘禹廷, 张晨浩, 王锦权, **杨锦乾**, 徐向荣. 一种形式化的多标量乘法分析与加速方法[P]. 中国专利: 202310698800.4(已受理)

致 谢

时光荏苒，从我收到北航录取通知书的那刻起，至今已过了三年。回首这一段时光，我深感自己无论是学术探索亦或是个人成长方面都取得了显著的进步，这都得益于在这段时光中遇到的每一个人给予我的帮助和鼓励。在此，我想向所有陪伴我走过这段旅程的人表达最诚挚的感谢。

首先，我由衷感谢我的导师肖利民教授。肖老师深厚的学术造诣、独到的见解和严谨的治学态度，深深地感染了我。对于每个科研项目，大到整体架构设计，小到每个技术细节，肖老师都亲力亲为，与我们展开深入的细致探讨，使我们能够高质量地完成科研工作。非常感谢肖老师对我在科研道路上的指导，让我得到了巨大的成长。同时，也要感谢课题组的王良老师、霍志胜老师、阮利老师、刘磊老师和秦广军老师。各位老师尽职尽责地完成了实验室的组织管理工作，同时为我们的学术研究提供了充足的指导性意见。

其次，我要感谢实验室的同学们对我的关心和帮助。非常感谢王锦权博士对我在科研工作和工程实践上的指导，锦权和我同级，但学术能力和工程能力比起我们都要强上不少，让我十分钦佩，成为了我入学以来学习和效仿的榜样。在和锦权一同完成的各项科研工作中，锦权给我提供了许多帮助，教会了我许多工程技术上的学习技巧，使我的工程能力得到了巨大提升。同样也要感谢宋尧师兄和沈润楠师兄，两位师兄为我的学术探索提供了宝贵的思路。感谢贾志斌、孟令达、闭家铭、李辉阳、陈旭光、杨睿等各位同窗好友们，与你们共度的时光充满欢乐和收获。

此外，我要特别感谢我的家人。感谢我的父母对我的关心和支持，让我能够心无旁骛地完成学业。在我求学的道路上，您们始终是最坚实的后盾。同时也要感谢我的女朋友周盼同学，感谢你在这段时间里的支持和鼓励，希望我们在以后的时间里能一同努力，一同进步。

最后，我要感谢参与本论文评审和答辩的各位专家教授，您的宝贵意见对我论文质量的提高起到了至关重要的作用。我将认真对待每一条建议，不断完善自己的学术成果。

再次向所有给予我帮助的人致以最诚挚的谢意！